

```
myBlob.drawSprite();
```

Objects First with Processing

Your character, your world, your rules.

Jason Cox
written with Claude, an AI writing assistant
Draft, October 2026

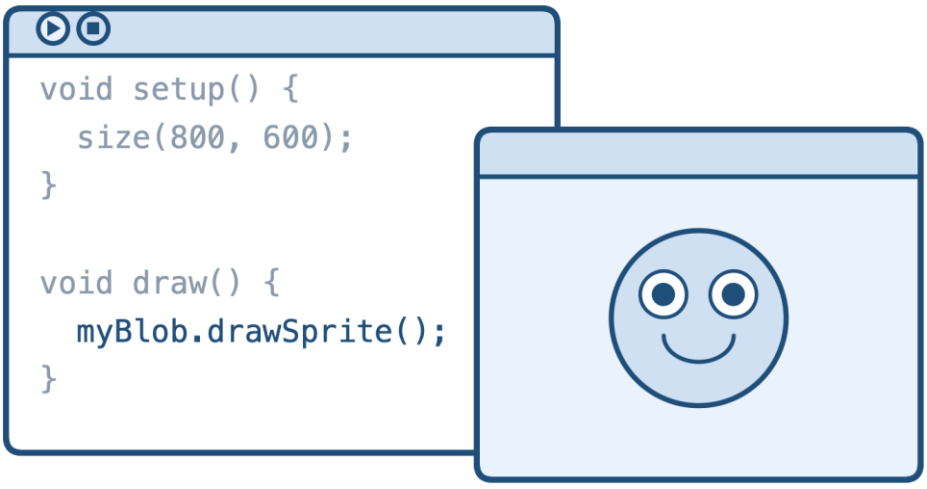
Contents

Prologue - How This Book Works 5
Unit 1 - Your Character 10
Chapter 1 - Hello, Blob! 11
Part One - A Blob is an object 13
Part Two - Once, and over and over 18
Part Three - Make it yours 21
Chapter 2 - Draw Your Character 29
Part One - A Blob with a new face 31
Part Two - Your drawing replaces Blob's 38
Part Three - Measure from the belly button 40
Chapter 3 - Bring It to Life 47
Part One - Two numbers and a step 49
Part Two - Make it go 50
Part Three - Make it yours 56
Chapter 4 - You're in Control 59
Part One - Two kinds of input 61
Part Two - Option A: follow the mouse 62
Part Three - Option B: the arrow keys 64
Part Four - Make it yours 68
Unit 2 - Your Game 72
Chapter 5 - Build Your World 73
Part One - Sketch it first 75
Part Two - Ground, and the order you draw in 76
Part Three - Things in the world 79
Part Four - Make it yours 82
Chapter 6 - Collisions: If This, Then That 87
Part One - Are they touching? 89
Part Two - Keep score 91
Part Three - A game that can end 95
Part Four - Make it yours 101
Chapter 7 - Build Your Game 105
Part One - You already know all of this 107

Part Two - One full loop 115
Part Three - Make it yours 117
Part Four - Play-test, and show it 120
Bonus Chapters 122
Bonus A - Images 123
Part One - Pictures live in a folder 125
Part Two - Scenery: `ImageSprite` 126
Part Three - Your character as a picture 129
Bonus B - Sounds 135
Part One - A second library 137
Part Two - Declare, load, play 138
Part Three - Where the sound goes 140
Bonus C - Animate Your Character 143
Part One - A flipbook 145
Part Two - One class, several poses 146
Part Three - Build the flipbook 148
Bonus D - Lists and Loops 155
Part One - A bag 157
Part Two - Two loops that both say `for` 158
Part Three - Taking things out 161
For Teachers 166
Teacher Reference - The SpriteGame Library 167
Part One - The shape of the library 169
Part Two - `Sprite`, the base 170
Part Three - `Blob`, and why the course builds on it 174
Part Four - The rest of the family 177
Part Five - Known issues 184

Prologue - How This Book Works

Your character, your world, your rules.



In this book you build a video game: an actual game, with a character you draw yourself, that you can hand to somebody else and watch them play.

You start on the first day by putting something on the screen. In Chapter 2 you draw your character. After that, every chapter gives that character something new. It moves, then you control it, then it gets a world to live in, then things to collect and things to avoid. By Chapter 7 the game is yours: your character, your world, your rules.

You can start this book having never seen a line of code. It begins at the very beginning.

Objects first

The first thing you make in this book is an **object**: a thing in your program that has its own information and knows how to do things. Plenty of programming courses spend weeks on other things before they get to objects. This one starts with an object on the first day, and every chapter after that is about objects: using them, then making your own, then making them do more.

You'll use objects before you know everything inside them. That's on purpose. A Blob already knows how to draw itself, how to move, and how to tell when it's touching something. You'll ask it to do those things, and see them work, a good while before you'd know how to build them yourself. Curiosity about how it works is the bridge to building your own.

When I was in college, I had a textbook called *Objects First with Java*. I didn't realize it at the time, but that book settled my belief that objects are the right place to start, and this book starts with an object for the same reason.

Where this came from

The objects in this book come from a library called **SpriteGame**. I wrote it for this class, Coding for Embedded Systems, the sixth-grade class I teach at Winsor. The class runs for two quarters. The first quarter is Processing, and that's this book. The second is Arduino, on the Circuit Playground board.

SpriteGame comes from the same belief about objects first that I teach my high school class with. The version that went into its repository in 2021 was already about its third, and in the spring of 2026 it became a library you can install into Processing.

The first quarter is Processing because Processing draws. You write a few lines, press Run, and something shows up on your screen, fast. That's a good way to start.

How the chapters got written

The chapters came after the lessons. Each lesson has a plan, a page on Canvas, and the code you start from. Each chapter got written from those three things.

So the code in every chapter is real. It's copied from what the lesson gives you, or it's code the chapter asks you to write yourself. Every picture of a sketch window in this book is a real one: the code on the page was run, and the picture is what it drew. When a chapter shows an error message, that's the message Processing printed, word for word.

I wrote this book with a writing assistant: Claude, an AI model made by Anthropic. Claude helped me turn each lesson, its code, and its Canvas page into the chapter you're reading. This book could not exist without that help.

This is the first year the course runs this way, and the book is being written while it runs. Like the course, it'll change the next time.

How a chapter runs

Most chapters take one class meeting. Chapter 6 takes two, and Chapter 7 takes three. All of the building happens in class.

When you get stuck, there's **Sparky**, an AI helper on Flint. Sparky helps you with syntax: the exact spelling, a missing semicolon, what an error message means. It can help you come up with ideas when you're stuck on what to make. And it can do the math for you. If you want a hat sitting right on top of your character's head, Sparky can work out the numbers, and you get back to building.

Sparky usually hands you code with a few blanks in it. You fill them in, type the code yourself, and run it. The first students to take this course said that was the help that worked best, so it's how every Sparky works now. If you're still stuck, ask again, and Sparky shows you the whole thing.

Sparky works best when you tell it which chapter you're on and paste in your code. Ask for one step at a time. For a shape, describe it in plain words: where it starts, where it ends, and which way it curves. Before you paste in a big change, save a copy with **File → Save As** and a number on the end of the name, so your old version is still there if something breaks. And if Sparky gets something wrong twice, or says it isn't sure, take it to your teacher.

Your character

From Chapter 2 on, there's one tab in your sketch that belongs entirely to you: `MyCharacter`. Every chapter does its new idea once with an example, so everybody has the same working code and the same words for it. Then it hands the idea to you, to try on your character. That's where you find out whether you can do it alone.

Run it after every change

Each chapter's building is cut into small Steps, and your sketch runs at the end of every one. Run it after every change. That's how you find out what you broke, while it's still one line.

The chapters come in two units. Unit 1, **Your Character**, is Chapters 1 to 4: you make a character, make it move, and take the controls. Unit 2, **Your Game**, is Chapters 5 to 7: a world, things to collect and avoid, and a game that ends.

Unit	Chapter	Meetings
1	1 - Hello, Blob!	1
1	2 - Draw Your Character	1
1	3 - Bring It to Life	1
1	4 - You're in Control	1
2	5 - Build Your World	1
2	6 - Collisions: If This, Then That	2
2	7 - Build Your Game	3

Four bonus chapters come at the back of the book: images, sounds, animation, and lists. Each one says when it's best taken. After them, at the very back, there's a reference for teachers, about how the library works on the inside.

How to read a chapter

The shape

Every chapter has the same parts, in the same order.

- **An opening** that says where the last chapter left you, and what's still missing. A table names the chapter's two or three moves.
- **Part One** reads the code you start from. Its sections are numbered by chapter, 1.1, 1.2, 1.3, and each one takes apart one piece of it.
- **Part Two and after** build. The building comes in numbered Steps, and each Step changes one or two things.
- **The end of the chapter** gathers it up: a summary of what you built, a table of new code and where you met it, the vocabulary, a few questions to check your understanding, and a look at what comes next.

The marks

Build it marks code to type into your sketch. Type it by hand when you can. Your fingers learn the code faster than your eyes do.

Your turn marks code you write yourself. The chapter says what it has to do, and leaves the how to you.

Optional marks a **Your turn** that nothing later depends on. Skip it if you're short on time, and the steps after it still work. Do it if you're curious.

The rest come in boxes, and each kind has its own color.

Aside: what an Aside is

An Aside explains a new piece of code the first time it shows up in the book. If you already know it, skim the box and keep going.

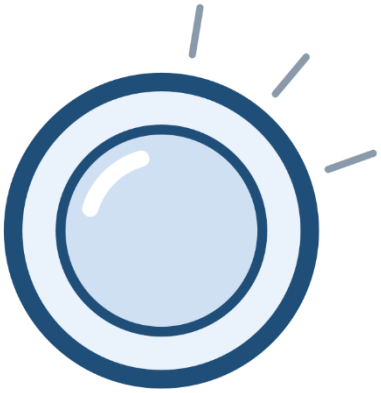
Check. A Check marks a place to stop, run your sketch, and make sure it does what the chapter says. It tells you what you should see, and most Checks come with a picture of the window. If yours is different, fix that before you go on, because the next step builds on this one.

Watch out. A Watch out box is the mistake nearly everybody makes at that spot, and what it looks like when you make it.

Argue it. A question with more than one good answer. Pick one, and be ready to say why.

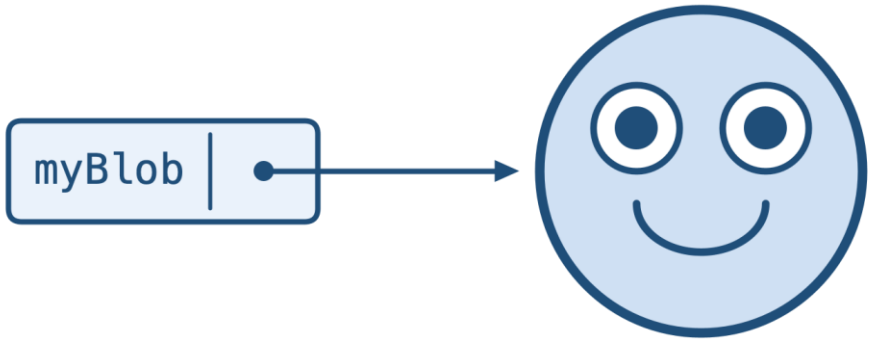
Unit 1 - Your Character

Chapters 1 to 4



Chapter 1 - Hello, Blob!

Run your first program and put something alive on the screen.



This window has a red blob in it. You'll have to take my word for it.


```
void setup() {
  size(800, 600);
  myBlob = new Blob(this, 400, 300, 40, color(255, 80, 80));
  myBlob.drawSprite();
}
```

By the end of today there's something on your screen, and you put it there. It's a circle. The library we're using calls it a **Blob**, and it's the first object you'll make in this course. Every chapter after this one gives you more say over what's on the screen, and by Chapter 7 it's a game.

The program that makes the circle is ten lines of code. You'll type all ten. Then you'll run it, break it on purpose, and change it until the blob is yours.

This chapter does it in three moves.

Part	The idea
One	**A Blob is an object.** Type the sketch, run it, and read every line.
Two	**Once, and over and over.** `setup()` runs one time, and `draw()` runs about sixty times a second. Moving one line proves it.
Three	**Make it yours.** Its color, where it sits, how big it is, what's behind it, and maybe a friend.

Part One is mostly reading. Parts Two and Three are five small steps, and the sketch runs at the end of every one.

Part One - A Blob is an object

1.1 Type it, then run it

Open Processing. You get a window with a big space for code, a ▶ **Run** button, and a ■ **Stop** button. A program in Processing is called a **sketch**.

Save before you type anything: **File → Save As**, and name it `HelloBlob`. Saving first means you'll know where your sketch lives when you come back to it.

Build it. Type this in. Type it by hand. Your fingers learn the code faster than your eyes do.

```
// Lesson 1 - Hello, Blob!
import coxprogramming.processing.sprites.*;

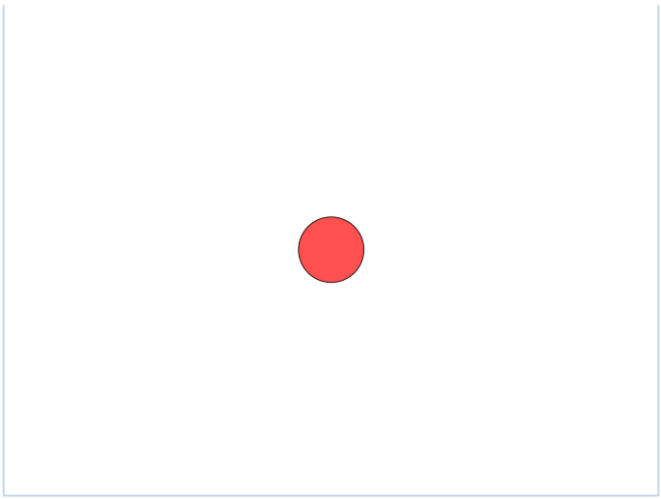
Blob myBlob;

void setup() {
  size(800, 600);
  myBlob = new Blob(this, 400, 300, 40, color(255, 80, 80));
}

void draw() {
  background(255);       // white background erases the previous frame
  myBlob.drawSprite();   // draw the blob at its position
}
```

Get it exactly right. One missing semicolon is enough to stop the whole thing. Then press ▶ **Run**.

> **Check.** A new window opens with a red circle in the middle. Look closely and the circle has a thin black outline. Remember the outline, because it comes back in Step 3. Press ■ **Stop** to close the window.



Your first sketch. The red circle is a Blob.

If you get an error instead, look at the line Processing highlights, and check it letter by letter against the code above. Step 2 shows you the three most common mistakes, on purpose.

1.2 Objects have properties and behaviors

The red circle is an **object**. An object is a thing in your program that has its own information and knows how to do things.

Programmers have a word for each of those. **Properties** are the things that describe an object: where it is, how big it is, what color it is. **Behaviors** are the things an object can do. A Blob can draw itself on the screen.

You'll find both in the code. The properties are the numbers you hand a Blob when you make it. The behavior is `drawSprite()`.

1.3 Reading it, line by line

Ten lines of code, and one note at the top. Here's what each one does.

```
// Lesson 1 - Hello, Blob!
```

> **Aside: comments**
>
> A line that starts with `//` is a **comment**, a note for people. Processing skips it. A comment can also sit at the end of a line, like the two in `draw()`.

```
import coxprogramming.processing.sprites.*;
```

This tells Processing you want to use the library's tools. A **library** is a set of tools somebody else built, so you can use them. `Blob` is one of them. This line goes first, always.

```
Blob myBlob;
```

This makes a **variable**: a box with a name on it. The box is called `myBlob`, and the word in front of it, `Blob`, says what kind of thing the box holds. Right now the box is empty. The next part of the sketch puts a blob in it.

```
void setup() {
  size(800, 600);
  myBlob = new Blob(this, 400, 300, 40, color(255, 80, 80));
}
```

`setup()` is a **function**: a set of instructions with a name. Its instructions go between the curly braces, `{` and `}`. Processing runs `setup()` one time, when your sketch starts. Part Two is about exactly that.

> **Aside: the semicolon**
>
> Every instruction ends in a semicolon, `;`. That's how Processing knows where one instruction stops and the next one starts, the way a period ends a sentence. Lines that end in a brace, like `void setup() {`, are the exception.

`size(800, 600)` makes the window 800 pixels wide and 600 pixels tall. A **pixel** is one tiny dot of your screen.

> **Aside: x and y**
>
> Every spot in the window has two numbers. **x** is how far across it is from the left edge. **y** is how far down it is from the top. The top-left corner is `(0, 0)`, and in this window the bottom-right corner is `(800, 600)`.
>
> Watch the y. In math class, up is bigger. On a screen, down is bigger.

Then the line that makes the blob:

```
myBlob = new Blob(this, 400, 300, 40, color(255, 80, 80));
```

`new Blob(...)` makes a brand-new Blob, and the `=` puts it in the `myBlob` box. The five things inside the parentheses are what a Blob needs to know about itself, always in this order:

	What it is
`this`	the sketch the blob belongs to. Copy it exactly, every time.
`400`	x: how far across the blob's center is
`300`	y: how far down the blob's center is
`40`	the **radius**: how far it is from the center to the edge. A radius of 40 makes a circle 80 pixels across.
`color(255, 80, 80)`	its color. Section 1.4 is about these three numbers.

Those are its properties. `(400, 300)` is the exact middle of an 800-by-600 window, which is why your blob sits in the middle.

```
void draw() {
  background(255);       // white background erases the previous frame
  myBlob.drawSprite();   // draw the blob at its position
}
```

`draw()` is the other function. `background(255)` paints the whole window white. Then `myBlob.drawSprite()` asks your blob to draw itself.

> **Aside: the dot**
>
> `myBlob.drawSprite()` reads left to right: *myBlob, draw yourself.* First the object's name, then a dot, then the behavior you want. The parentheses on the end mean *do it now*.
>
> A behavior written in code is called a **method**. `drawSprite()` is your blob's method for drawing itself.

Part Two - Once, and over and over

Every sketch in this course has these two functions.

`setup()` is like setting up an art table. It happens once, before anything else. You put out your paper and your paints, and you're done setting up.

`draw()` is like drawing one page of a flipbook. Processing runs it again, and again, and again, about **60 times a second**. Flip the pages fast enough and it looks like motion. Each page is called a **frame**.

- Something that happens one time goes in `setup()`.
- Something that has to keep happening goes in `draw()`.

Your blob sits perfectly still, though. So how would you know `draw()` is running sixty times a second?

Step 1 - Move one line

Build it. Cut the whole line `myBlob.drawSprite();` out of `draw()`, comment and all. Paste it at the end of `setup()`, under the line that makes the blob:

```
void setup() {
  size(800, 600);
  myBlob = new Blob(this, 400, 300, 40, color(255, 80, 80));
  myBlob.drawSprite();
}

void draw() {
  background(255);       // white background erases the previous frame
}
```

Run it.

> **Check.** A white window, empty.

The blob's somewhere in this picture.

Where did the blob go? Make a guess before you read on.

The blob got drawn. `setup()` drew it, once, right at the start. Then `draw()` ran, and its first instruction is `background(255)`, which paints the whole window white, right over the blob. Then `draw()` ran again, and again, sixty times a second, painting white each time. The blob's under the paint.

That's why `drawSprite()` belongs in `draw()`. Anything you want to see has to be drawn again on every frame, after the background.

Build it. Put the line back at the end of `draw()`, where it started, and run.

> **Check.** The red blob is back in the middle of a white window.

Step 2 - Break it on purpose

Sooner or later, every sketch you write stops with an error. The quickest way to get good at errors is to make a few on purpose, while you know exactly what you broke.

> **Aside: reading an error**
>
> When Processing can't run your sketch, it highlights a line and shows a message under your code. The message says what went wrong, as well as Processing can tell. The highlighted line is where it noticed. Those can be two different lines.

Make each break, run, read the message, and then fix it before the next one.

Build it. Delete the `import` line, and run.

Check. The sketch stops, and the message reads:

```
Cannot find a class or type named “Blob”
```

Processing can't find Blob, because the line that tells it where to look is gone. This is the most common mistake in the course. The message only names `Blob`, so when you see it, check the import line first. Put it back.

Build it. Delete the semicolon at the end of `size(800, 600);`, and run.

Check. The message reads `Syntax Error - Missing ';'?`, and the highlighted line is the one *after* `size`, the `myBlob = new Blob(...)` line.

Processing noticed the problem when it got to the next instruction. When an error points at a line that looks fine, look at the line above it. Put the semicolon back.

Build it. Change `drawSprite` to `drawsprite`, with a small `s`, and run.

Check. The message reads `The function drawsprite() does not exist.`

To Processing, `drawsprite` and `drawSprite` are two different words. Capital letters count, everywhere. Fix it, run, and make sure your red blob is back.

Part Three - Make it yours

1.4 Color is three numbers

`color(255, 80, 80)` means a lot of red, a little green, and a little blue. Each number goes from **0**, none of that color, up to **255**, as much as the screen can make. These are **RGB** numbers: Red, Green, Blue, always in that order.

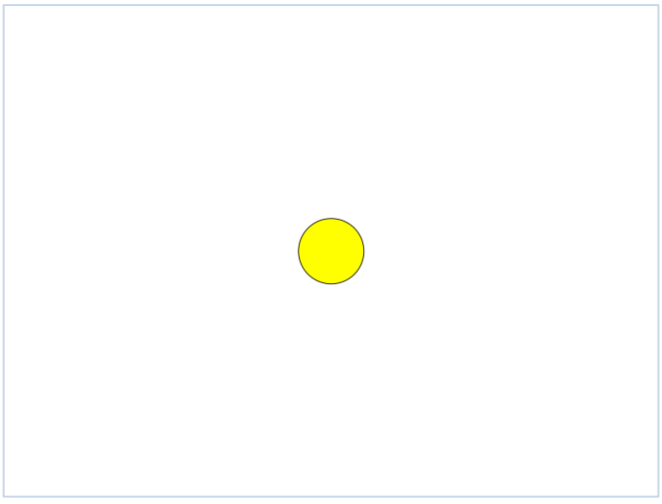
You write	You get
`color(0, 0, 0)`	black
`color(255, 255, 255)`	white
`color(255, 0, 0)`	red
`color(255, 80, 80)`	the starter's pink-red

Now predict one. Red all the way up, green all the way up, and blue at zero: `color(255, 255, 0)`. Mix red paint and green paint and you get a muddy brown. Write down what you think the screen will make.

Step 3 - Your color

Build it. Change `color(255, 80, 80)` to `color(255, 255, 0)`, and run.

> **Check.** A yellow blob.



Red light and green light make yellow.

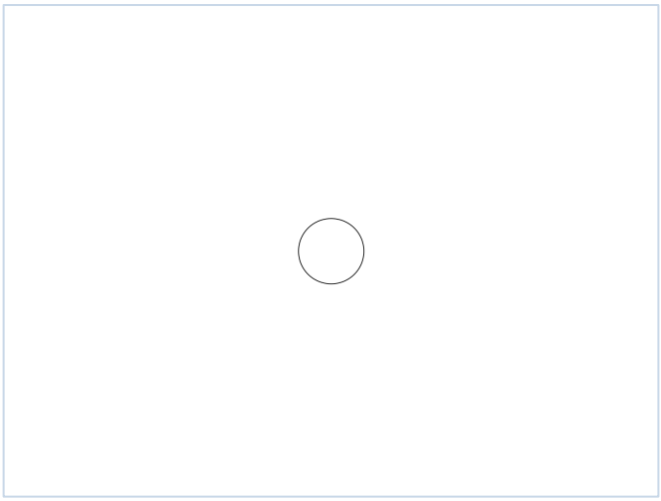
Your screen makes every color out of light, and paint and light mix by different rules.

The Color Selector saves you the guessing. Open **Tools → Color Selector**, click around until you find a color you like, and read the R, G, and B numbers right off the panel.

Your turn. Find a color you actually like in the Color Selector, and put its three numbers in your blob's `color( , , )`. "Because I like green" is a perfectly good reason.

One more, for that outline from 1.1. Try `color(255, 255, 255)`, a white blob on a white background.

> **Check.** You can still see it, as a thin black ring.

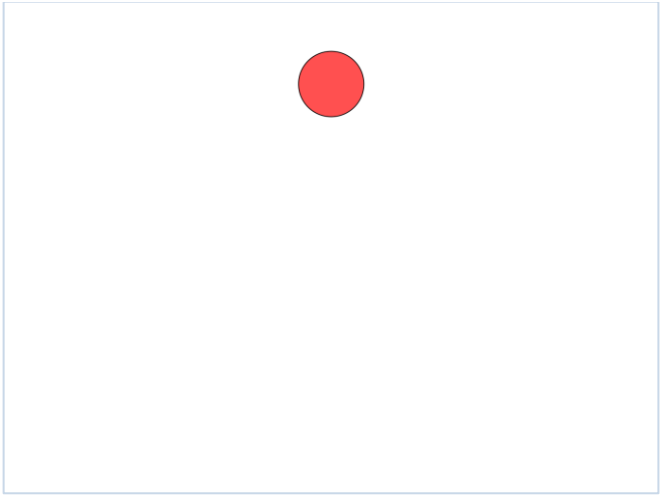


A white blob on white. The outline gives it away.

Processing draws an outline around every shape until you tell it to stop. In Chapter 2 you will. Put your own color back.

Step 4 - Where, and how big

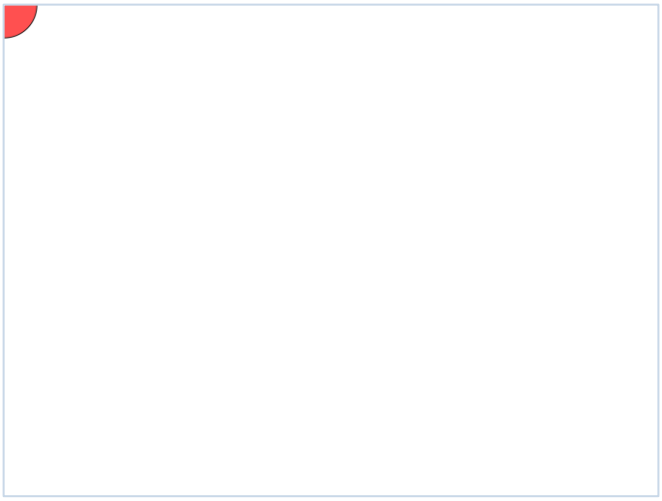
Build it. In the `new Blob` line, change `400, 300` to `400, 100`, and run. The blob moves up, because a smaller y is closer to the top.



y is 100: near the top.

Now predict where `(0, 0)` will put it, and try it.

Check. A quarter of a blob, tucked into the top-left corner.



The blob's center is on the corner.

`x` and `y` are where the blob's **center** goes. At `(0, 0)` the center sits right on the corner, and three quarters of the blob hang off the edges of the window.

Then the size. Change the radius, the `40`, to `80` and run. Then try `15`.

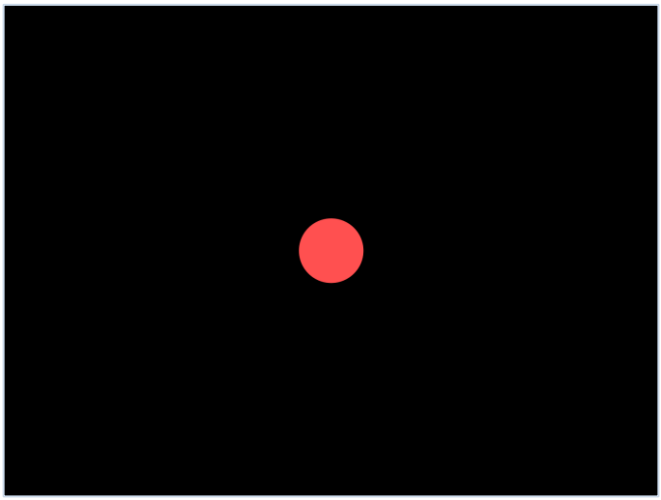
Your turn. Put your blob where you want it, at the size you want.

Step 5 - What's behind it

`background()` takes the same kind of numbers as `color()`. Give it one number for a grey, from `0` for black up to `255` for white, or three for any color at all.

Build it. Change `background(255)` to `background(0)`, and run.

> **Check.** Your blob on a black window, with its outline gone. The outline is black too.



background(0).

Your turn. Give your blob a background you chose. The Color Selector's three numbers work here too: `background(30, 40, 90)` is a night sky.

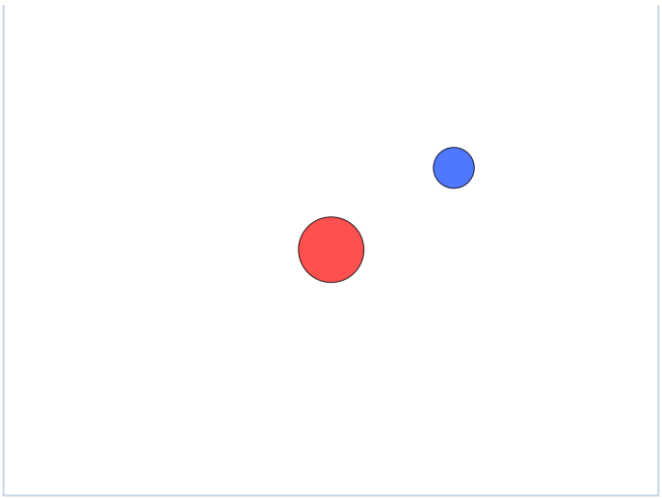
1.5 A friend

Your turn. Optional: give your blob a friend, a second Blob with its own color and its own spot. A second blob takes three lines, one in each of the three places your first blob uses:

1. At the top, a second box: `Blob friend;`
2. In `setup()`, a second `new Blob(...)`, to put a blob in that box.
3. In `draw()`, a second `drawSprite()`, after the background.

You pick its name, and all five of its numbers.

> **Check.** Two blobs. Here's one friend. Yours will be wherever you put it.



`myBlob`, and a friend at `(550, 200)`.

> **Watch out.** Skip the second of those three lines, and the sketch starts, then stops with `NullPointerException`, on your friend's `drawSprite()` line. The box at the top is still empty when `draw()` asks it to draw.

If one friend goes well, keep going: four or five blobs make a small scene. A face, a snowman, a planet with moons, whatever you see in them.

Chapter summary

You started with an empty window and ten lines to type. You end with a blob that's yours: your color, in your spot, at your size, on a background you chose.

You built	It's an example of
`Blob myBlob;`	a **variable**, a box with a name
`new Blob(this, 400, 300, 40, color(255, 80, 80))`	making an **object**, and handing it its **properties**
`myBlob.drawSprite()`	asking an object for a **behavior**, called a **method**
`drawSprite()` moved into `setup()`, and back	`setup()` runs once, and `draw()` runs every **frame**
three errors, made on purpose	reading an error: what it says, and where it noticed
your own `color( , , )`	**RGB**, three numbers mixed as light
a friend, if you made one	the same three places, a second time

New syntax in this chapter

Syntax	Where you met it
`//` comments	1.3
`import`	1.3
a variable: `Blob myBlob;`	1.3
`void setup() { }` and `void draw() { }`	1.3, Part Two
the semicolon	1.3
`size(800, 600)`	1.3
x and y	1.3
`new Blob(this, x, y, radius, color)`	1.3
the dot: `myBlob.drawSprite()`	1.3
`background()`	1.3, Step 5
`color(r, g, b)`	1.4

Vocabulary

behavior -- a verb: something an object can do. A Blob can draw itself. (1.2)

comment -- a note for people, after `//`. Processing skips it. (1.3)

frame -- one page of the flipbook. `draw()` makes one frame each time it runs, about 60 a second. (Part Two)

function -- a set of instructions with a name, like `setup()` and `draw()`. (1.3)

library -- a set of tools somebody else built, so you can use them. SpriteGame is the library that has `Blob`. (1.3)

method -- a behavior, written in code. You ask for one with a dot: `myBlob.drawSprite()`. (1.3)

object -- a thing in your program that has its own information and knows how to do things. (1.2)

pixel -- one tiny dot of your screen. (1.3)

property -- a noun: something that describes an object. Where it is, how big, what color. (1.2)

radius -- how far it is from the center of a circle to its edge. Half the circle's width. (1.3)

RGB -- the three numbers, Red, Green, and Blue, that make a color. Each goes from 0 to 255. (1.4)

sketch -- a program in Processing. (1.1)

variable -- a box with a name on it. `Blob myBlob;` makes one. (1.3)

Check your understanding

1. Name three properties of your blob, and one behavior.
2. Which function runs once, and which runs over and over? About how many times a second?
3. In Step 1 the blob disappeared. Where was it?
4. Your sketch stops with `Cannot find a class or type named "Blob"`. What do you check first?
5. What does `color(0, 0, 255)` make? What about `color(0, 0, 0)`?
6. Where in the window is `(800, 0)`?

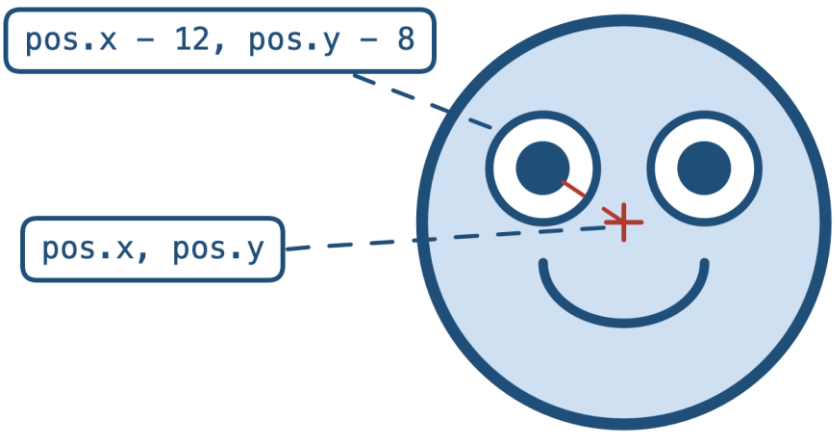
What's next

Your blob's a circle, and a circle is all a Blob knows how to look like. In Chapter 2 you make your own kind of object: a character you draw yourself, out of as many shapes as you want, that still knows everything a Blob knows.

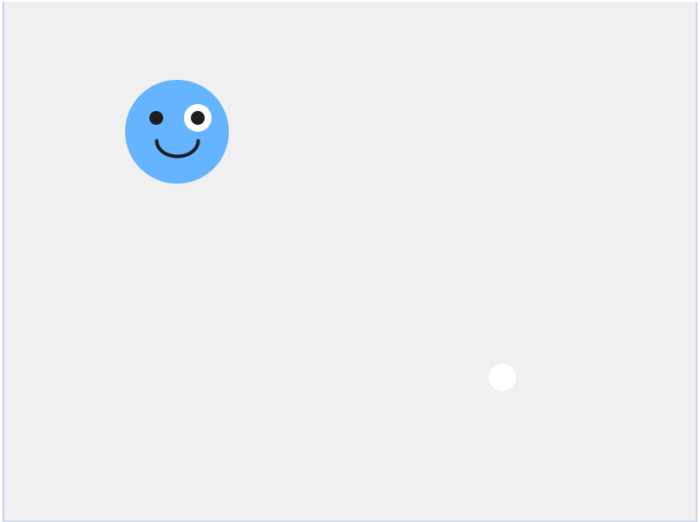
And yes, it's going to move. That's Chapter 3.

Chapter 2 - Draw Your Character

Build the character that will star in everything you make from here on.



The whole face moved to the corner. One eye stayed where it was told.



```
ellipse(388, 292, 16, 16);
```

Chapter 1 ended with a blob that was yours: your color, your spot, your size. It's still a circle, though. A circle is the only way a Blob knows how to look.

A plain Blob already knows a lot. It has a position, it can move, it can wrap around the screen, and later it can tell when it's touching something. What it's missing is a face you care about.

So today you build a character **on top of** Blob. It keeps everything a Blob knows, and you decide what it looks like. This is the character you'll use for the rest of the course, so make it one you actually want to look at.

This chapter does it in three moves.

Part	The idea
One	**A Blob with a new face.** Make a new tab, put a character in it, and read every line.
Two	**Your drawing replaces Blob's.** Break it two ways, and see the plain Blob underneath.
Three	**Measure from the belly button.** Every shape goes where it goes because of `pos.x` and `pos.y`. Moving the character proves it, and then you make it yours.

Part One - A Blob with a new face

2.1 Two tabs

Open your sketch from Chapter 1. Or start a new one, and save it as `MyCharacter` before you type anything.

Your character gets a tab of its own.

> **Aside: tabs**
>
> A **tab** is a separate file inside your sketch. They sit in a row along the top of your code, and the first one is the main tab. Big sketches are easier to read when each kind of object lives in its own tab.

Build it. Click the small **arrow at the far right of the tab bar**, and choose **New Tab**. Name it `MyCharacter`. Processing adds the `.pde` on the end for you.

The arrow is small, and it's all the way at the right edge. Look for it twice.

Build it. Paste this into the new `MyCharacter` tab:

```
/**
 * MyCharacter - this is YOUR character. Change everything!
 */
class MyCharacter extends Blob {

  /**
   * The constructor: called once when you create the character.
   * "super" connects your character to the Blob it's built on.
   * The radius (30) is the invisible collision circle - make it match your art.
   */
  MyCharacter(PApplet p, float x, float y) {
    super(p, x, y, 30, color(100, 180, 255));
  }

  /**
   * drawSprite: called every frame to draw your character on screen.
   * Use pos.x and pos.y as the center - all other shapes are measured from
there.
   */
  void drawSprite() {
    // Body
    fill(100, 180, 255);
    noStroke();
    ellipse(pos.x, pos.y, 60, 60);

    // Eyes
    fill(255);
    ellipse(pos.x - 12, pos.y - 8, 16, 16);
    ellipse(pos.x + 12, pos.y - 8, 16, 16);
    fill(30);
    ellipse(pos.x - 12, pos.y - 8, 8, 8);
    ellipse(pos.x + 12, pos.y - 8, 8, 8);

    // Smile
    stroke(30);
    strokeWeight(2);
    noFill();
    arc(pos.x, pos.y + 5, 24, 18, 0, PI);
  }
}
```

Build it. Click back to your first tab. Replace everything in it with this:

```
// Lesson 2 - Draw Your Character
import coxprogramming.processing.sprites.*;

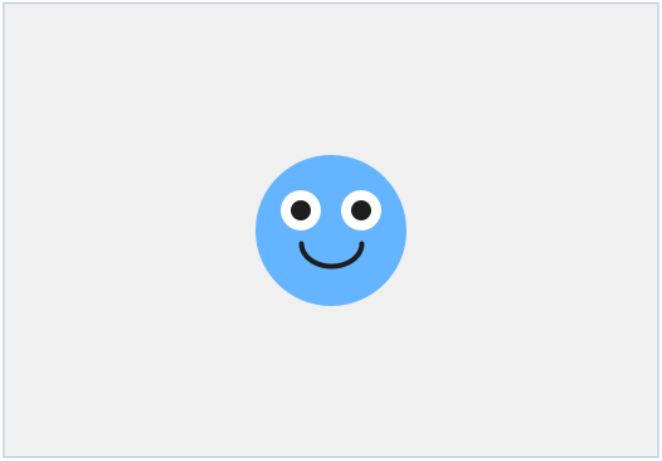
MyCharacter player;

void setup() {
  size(800, 600);
  player = new MyCharacter(this, width/2, height/2);
}

void draw() {
  background(240);
  player.drawSprite();
}
```

Run it.

> **Check.** A friendly blue face in the middle of a light grey window, with two eyes and a smile.



The starter character. It won't look like this for long.

2.2 The main tab

The main tab is Chapter 1's sketch with three changes.

`MyCharacter player;` makes a box called `player`, for holding a `MyCharacter`. In Chapter 1 the box was for a `Blob`. `MyCharacter` is a kind of thing you just invented, in the new tab.

`new MyCharacter(this, width/2, height/2)` makes your character and puts it in the box. It takes three things: the sketch, then x, then y.

> **Aside: `width` and `height`**
>
> Processing remembers the size of your window. `width` is 800 here, and `height` is 600, because that's what `size(800, 600)` asked for. The `/` means divide, so `width/2` is 400: halfway across. `(width/2, height/2)` is the middle of the window, whatever size the window is.

`background(240)` is a light grey. And `player.drawSprite()` is the same dot as Chapter 1: *player, draw yourself.*

2.3 The class, and the handshake

Now the `MyCharacter` tab. It starts with the line that does the work:

```
class MyCharacter extends Blob {
```

A **class** is a blueprint for making a new kind of object. This one says how to make a `MyCharacter`, and everything about it goes between this brace and the last one in the tab.

`extends Blob` means your character is built on top of Blob. It already knows how to move, wrap around the screen, and detect collisions. All you add is what it looks like.

Your character gets all of Blob's properties and behaviors, for free. It has a position, a radius, and a color, and it can do everything a Blob can do. In a minute you'll swap out one behavior, `drawSprite()`, for your own.

Next is the **constructor**, the part that runs once, when `new MyCharacter(...)` makes your character:

```
MyCharacter(PApplet p, float x, float y) {
  super(p, x, y, 30, color(100, 180, 255));
}
```

The three things in the first line catch the three things the main tab sent. `p` is the sketch, which was `this`. `x` and `y` are where to start.

> **Aside: `float`**
>
> `float` is the kind of number that can have a decimal point, like `2.5`. Positions are floats, because a character can sit partway between two pixels. `PApplet` is the kind of thing a sketch is. Every character class you write has both words, right in this spot.

The line starting with `super` is the **handshake**. It connects your character to the library. Copy it exactly.

Look at what's inside it, though: the sketch, x, y, a radius, and a color. That's the five things a Blob needs, from Chapter 1. The handshake hands them to the Blob your character is built on.

- The `30` is your character's radius: an invisible circle the library uses later, for collisions. Keep it about the size of your character's body.
- The color is a backup. You're about to draw over it.

> **Watch out.** Delete the `super` line, and the sketch stops with this:
>
> ```
> Implicit super constructor Blob() is undefined. Must explicitly invoke
> another constructor
> ```
>
> All of that means the handshake is missing. Put it back exactly as it was.

2.4 drawSprite(), yours

The rest of the tab is one method:

```
void drawSprite() {
```

You met `drawSprite()` in Chapter 1. A Blob already has one, and it draws a circle. When you write your own `drawSprite()` in your class, yours replaces Blob's. Programmers call that **overriding** a method. Every frame, `player.drawSprite()` runs your drawing.

> **Aside: `void`**
>
> `void` at the start says the method just does its job. It draws, and that's the end of it. Every method you write for a while starts with `void`.

Inside are three groups of shapes.

The body.

```
fill(100, 180, 255);
noStroke();
ellipse(pos.x, pos.y, 60, 60);
```

`fill()` picks the color for the shapes that come after it. `noStroke()` turns off the outline you saw on the white blob in Chapter 1. `ellipse()` draws an oval: where its center goes, then how wide, then how tall. Sixty wide and sixty tall is a circle.

The eyes. White ovals first, then dark pupils on top of them.

```
fill(255);
ellipse(pos.x - 12, pos.y - 8, 16, 16);
ellipse(pos.x + 12, pos.y - 8, 16, 16);
fill(30);
ellipse(pos.x - 12, pos.y - 8, 8, 8);
ellipse(pos.x + 12, pos.y - 8, 8, 8);
```

Shapes stack. Whatever you draw **later** sits on top of what you drew earlier. The pupils come after the whites, so they sit on top.

The smile.

```
stroke(30);
strokeWeight(2);
noFill();
arc(pos.x, pos.y + 5, 24, 18, 0, PI);
```

`stroke()` turns the outline back on, in a dark grey, and `strokeWeight(2)` makes it two pixels thick. `noFill()` means the next shape is only an outline.

> **Aside: `arc`**
>
> `arc()` draws part of an oval. The first four numbers are the same as an `ellipse`. The last two say where to start and where to stop, measured so that `PI` is half a turn. The turn starts at the right side of the oval and goes down first. `0` to `PI` is the bottom half of the oval, which is a smile.
>
> `PI` to `TWO_PI` is the top half. It curves the other way: a frown, a rainbow, or a raised eyebrow. `TWO_PI` is a whole turn.



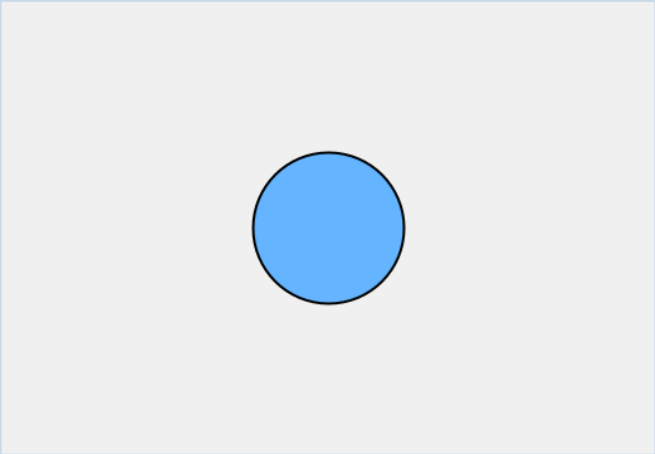
Part Two - Your drawing replaces Blob's

Step 1 - Misspell it

In Chapter 1, `drawsprite` with a small `s` stopped your sketch with an error. Try it here, on the method in your class.

Build it. In the `MyCharacter` tab, change `void drawSprite()` to `void drawsprite()`. Predict what happens, then run.

> **Check.** It runs fine. The face is gone, and in its place is a plain blue circle with a thin black outline.



The Blob under your character.

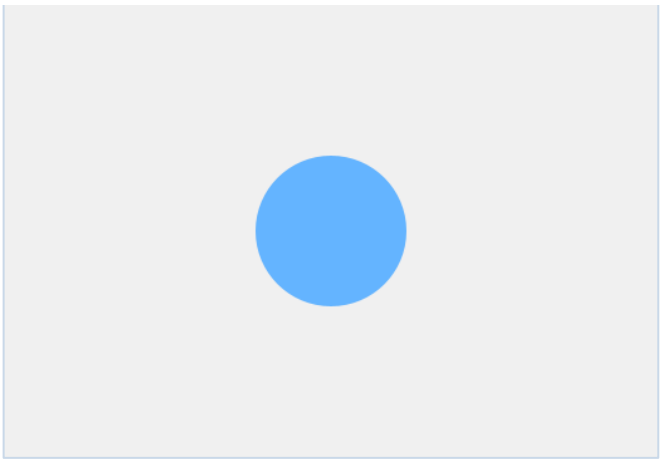
`drawsprite` is a brand-new method, with a name of its own. The main tab calls `drawSprite()`, with a capital `S`, and now the only `drawSprite()` your character has is Blob's. Blob's draws a plain circle, at the radius and in the backup color from your handshake, with Processing's outline. That circle was under your face the whole time. Your `drawSprite()` was covering it up.

Build it. Put the capital `S` back, and run. The face is back.

Step 2 - Draw the body last

Build it. In your `drawSprite()`, cut the three body lines, from `// Body` to the body's `ellipse`. Paste them at the bottom of the method, after the `arc`. Run.

> **Check.** A plain blue circle again, and this time the outline's gone too.

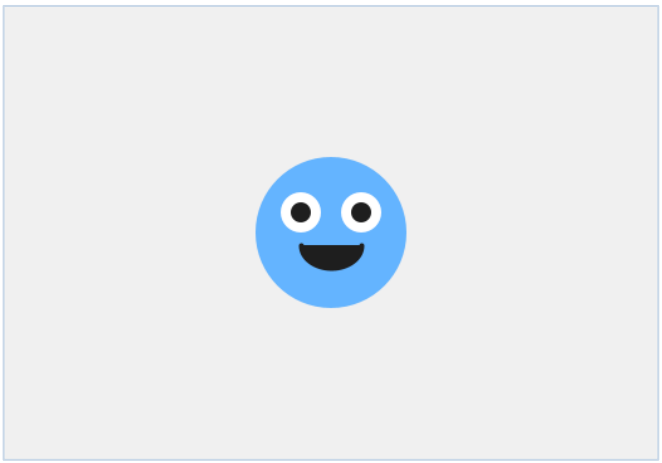


The body, painted over the face.

The face is still there, under the body. Shapes stack, so the body you drew last covered everything you drew before it. The outline is missing because the body's own `noStroke()` comes right before it.

Build it. Put the body lines back at the top, and run.

> **Watch out.** Leave out `noFill()` before the `arc`, and the smile fills in with the pupils' dark grey. You get an open mouth. Maybe you wanted one.



The smile without `noFill()`.

Part Three - Measure from the belly button

Every shape in your character is placed from `pos.x` and `pos.y`. `pos` is your character's position, one of the properties it got from Blob, and it's the center of your character.

Think of `pos.x, pos.y` as your character's belly button. Every shape you draw is measured from there.

- **Minus x** moves a shape left. **Plus x** moves it right.
- **Minus y** moves a shape up. **Plus y** moves it down.

Yes, up is minus. Screens are strange that way.

So `ellipse(pos.x - 12, pos.y - 8, 16, 16)` is an eye 12 pixels left of the belly button and 8 pixels up.

Step 3 - A new body color

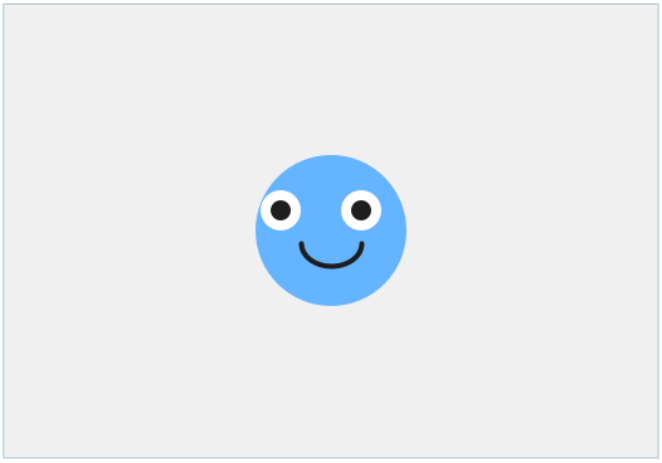
Build it. Find `fill(100, 180, 255)`, right above the body's `ellipse`, and put in a color of your own. The Color Selector works the same as in Chapter 1. Run.

> **Check.** The body changes color. The eyes and the smile have their own `fill` and `stroke`, so they look just as they did.

Step 4 - Slide an eye

Build it. The left eye is the two lines with `pos.x - 12`: its white and its pupil. Change both to `pos.x - 20`, and run.

> **Check.** The left eye slides out toward the edge of the face.



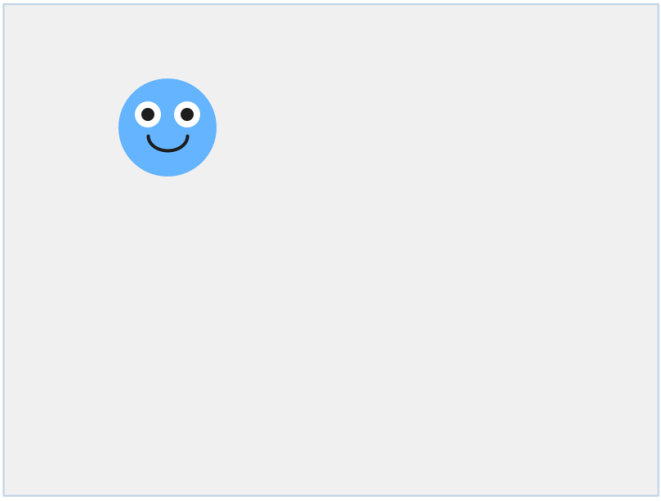
The left eye, 20 pixels left of center.

Put it wherever you like it.

Step 5 - Start somewhere else

Build it. In the main tab, change `width/2, height/2` to `200, 150`, and run.

> **Check.** The whole face is up near the top-left now, all in one piece.



Your character, starting at `(200, 150)`.

Every shape moved, because every shape is measured from the belly button, and the belly button moved.

Step 6 - The eye at fixed numbers

Here's an eye written another way. Before you moved it, the character's belly button was at `(400, 300)`. So the left eye's white was at `400 - 12` and `300 - 8`, which is `(388, 292)`.

Build it. First, put the main tab back to `width/2, height/2`. Then, in the `MyCharacter` tab, change the left eye's white, the first `ellipse` under `fill(255)`, to plain numbers:

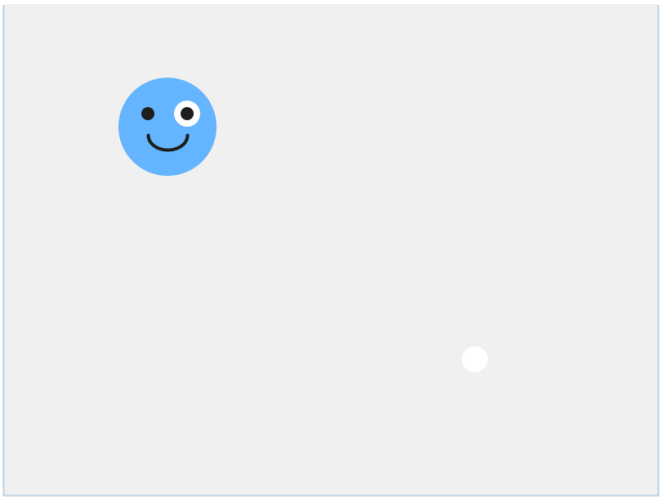
```
ellipse(388, 292, 16, 16);
```

Run it.

> **Check.** Your character looks exactly the same as before.

Now change the main tab to `200, 150` again, and run.

> **Check.** The face moves up to the top-left. One white eye stays behind in the middle of the window.



One eye didn't get the message.

Why did that eye stay behind, when every other part of the face moved? Make a guess before you read on.

Plain numbers nail a shape to one spot in the window. It looks fine as long as the character sits still, and today it does. In Chapter 3 your character starts moving, and a shape at plain numbers gets left behind. So measure **every** shape from `pos.x` and `pos.y`.

Build it. Put the eye back to `pos.x - 12, pos.y - 8`. Put the main tab back to `width/2, height/2`, or anywhere you like.

2.5 Make it yours

This chapter **is** the make-it-yours. Spend the rest of your time here.

The shapes you can use, all measured from the belly button:

You write	You get
`ellipse(x, y, w, h)`	an oval, centered at `x, y`
`rect(x, y, w, h)`	a rectangle. Its `x, y` is the **top-left corner**, so `rect(pos.x - 20, pos.y - 20, 40, 40)` is a square centered on the belly button.
`triangle(x1, y1, x2, y2, x3, y3)`	a triangle, from its three corners
`line(x1, y1, x2, y2)`	a line, from one end to the other. Its color is the `stroke`.
`arc(x, y, w, h, start, stop)`	part of an oval

Your turn. Make your character yours. Some places to start:

- Change the **body shape**. `rect()`, `triangle()`, or several overlapping `ellipse()` calls all work.
- Add hair, a hat, wings, antennae, a tail, ears, a beak, horns. Anything.
- Change every color, with the Color Selector.
- Add detail. Eyebrows change the whole mood. So do freckles, a nose, or a mouth with teeth.
- Give it an expression. Angry eyebrows are two short `line()` calls.

Order matters as you go. Draw the body first, then what sits on it.

You can keep adding detail in every chapter from here on. Today's job is to get a character on the screen that's recognizably yours. Goodbye, starter blob~

Your turn. When your character is done, check the `30` in the `super(...)` line. If your character came out much bigger or smaller than the starter, change the `30` to about half your character's width.

Try a big change first, like `100`, and run. The screen looks exactly the same. The radius is invisible. In Chapter 6 it's the circle the library uses to decide whether your character is touching something, so set it to fit your character's body now.

Your turn. Optional: make a **second** character class in another new tab: a sidekick, a pet, a rival. Same pattern, different art. Give the class its own name, like `class Sidekick extends Blob`, and use that name everywhere `MyCharacter` appears in the copy.

Chapter summary

You started with a Blob that could only be a circle. You end with a character of your own, built on top of Blob: it knows everything a Blob knows, and it looks the way you drew it.

You built	It's an example of
the `MyCharacter` tab	a **class**, a blueprint for a new kind of object, in a **tab** of its own
`class MyCharacter extends Blob`	a new kind of object **built on top of** another
`super(p, x, y, 30, color(...))`	the **handshake**, handing Blob the five things it needs
your own `void drawSprite()`	**overriding** a method: your drawing replaces Blob's
`drawsprite`, misspelled	the plain Blob that was underneath all along
the body drawn last	shapes stack, and later shapes sit on top
every shape at `pos.x` and `pos.y` plus or minus	measuring from the belly button, so the whole face moves together

New syntax in this chapter

Syntax	Where you met it
tabs	2.1
`width`, `height`, and `/`	2.2
`class MyCharacter extends Blob { }`	2.3
the constructor, `PApplet`, `float`	2.3
`super(...)`	2.3
`void drawSprite() { }`	2.4
`fill()`, `noFill()`, `stroke()`, `noStroke()`, `strokeWeight()`	2.4
`ellipse()`, `arc()`, `PI`, `TWO_PI`	2.4
`pos.x`, `pos.y`	2.4, Part Three
`rect()`, `triangle()`, `line()`	2.5

Vocabulary

class -- a blueprint for making a new kind of object. (2.3)

constructor -- the part of a class that runs once, when `new` makes an object. (2.3)

`extends` -- *built on top of*. `class MyCharacter extends Blob` means everything a Blob can do, your character can do too. (2.3)

handshake -- the `super(...)` line. It connects your character to the library. Copy it exactly, and change only the radius. (2.3)

overriding -- writing your own version of a method your object already had. Yours replaces the old one. (2.4)

`pos.x`, **`pos.y`** -- your character's position, its belly button. Measure every shape from there. (Part Three)

tab -- a separate file inside your sketch. (2.1)

Check your understanding

1. What does `extends Blob` give your character? Say it in one sentence.
2. What are the five things inside the `super(...)` handshake, and where have you seen those five before?
3. Why did misspelling `drawSprite` give you a plain circle, and why that color?
4. Your eyes vanished after you added a new shape. What's the first thing you check?
5. An ear is at `pos.x + 30, pos.y - 25`. Is it on the left or the right? Above the belly button, or below?
6. Why is `ellipse(388, 292, 16, 16)` going to cause trouble in Chapter 3?

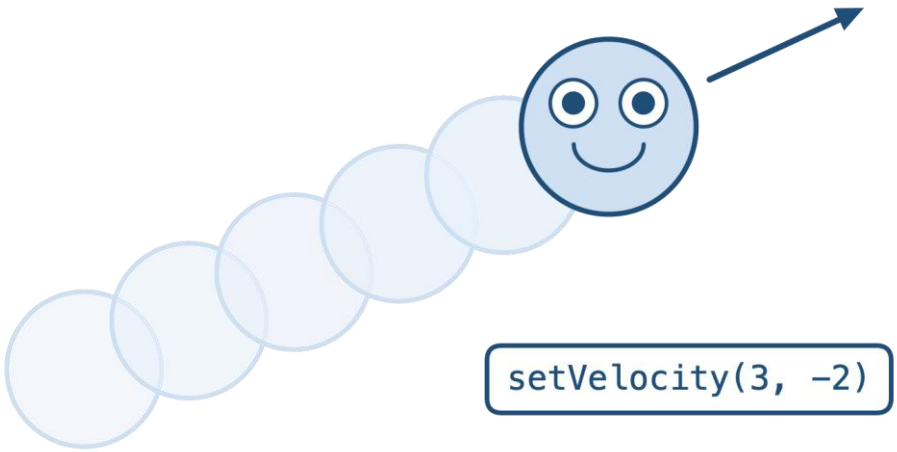
What's next

Your character can move. It always could: it's a Blob, and moving is one of the things a Blob knows how to do. All it needs is for your sketch to ask.

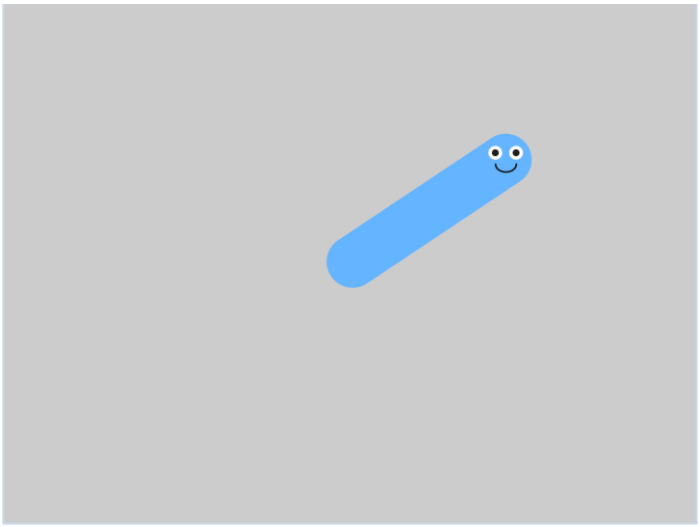
In Chapter 3 you give it a speed and a direction, and it slides across the screen and off the edge, and comes back around the other side. Every shape you measured from the belly button goes with it.

Chapter 3 - Bring It to Life

Your character stops sitting there and starts moving on its own.



Here is your character, and everywhere it has been for the last second.



```
//background(240);
```

Chapter 2 ended with a character of your own, sitting in the middle of the window. It can move. It always could, because it's built on top of Blob, and moving is one of the things a Blob knows how to do. All it needed was for your sketch to ask.

Today it asks. It takes two lines, one in `setup()` and one in `draw()`, and you leave your `MyCharacter` tab exactly as it is.

This chapter does it in three moves.

Part	The idea
One	**Velocity is two numbers.** How fast across, and how fast up or down.
Two	**`move()` is the step.** Give your character a velocity, watch nothing happen, then make it go: off the edge and back.
Three	**Make it yours.** A speed and a direction that suit your character, and a world with a mood.

Part One is reading. Part Two is six small steps, and the sketch runs at the end of every one.

Part One - Two numbers and a step

3.1 Velocity is two numbers

"Speed" in a game is really two questions. How fast am I going sideways? And how fast am I going up or down? So a character's **velocity** is two numbers.

- `vx` is left and right. **Positive** goes right. **Negative** goes left.
- `vy` is up and down. **Positive** goes **down**. **Negative** goes up.

```
              -vy  up
                |
 -vx  left  ----+----  right  +vx
                |
              +vy  down
```

You met this in Chapter 1: on a screen, y counts down from the top. So a positive `vy` moves your character down the screen, and that surprises nearly everybody.

The line that sets a velocity looks like this:

```
player.setVelocity(3, -2);  // 3 pixels right, 2 pixels up per frame
```

That's 3 pixels to the right and 2 pixels up, **every frame**. At sixty frames a second, it adds up fast.

Velocity is one more of your character's properties. Blob gave it one when it was made, set to zero across and zero down. That's why your character has been sitting still.

3.2 `move()` is the step

Setting a velocity is like telling someone how fast to walk. Until they take a step, they stay right where they are.

```
player.move();
```

`move()` is the step. It takes the velocity and adds it to the position, once. So it goes in `draw()`, where it happens sixty times a second.

Part Two - Make it go

Step 1 - A velocity, and nothing else

Open your sketch from Chapter 2. Leave the `MyCharacter` tab alone today. Everything happens in the main tab.

Build it. In `setup()`, right after the line that makes your character, add the velocity:

```
void setup() {
  size(800, 600);
  player = new MyCharacter(this, width/2, height/2);
  player.setVelocity(3, -2);  // 3 pixels right, 2 pixels up per frame
}
```

Run it.

> **Check.** Your character sits in the middle of the window, exactly the way it did in Chapter 2.

You told it how fast to go. Taking the step is a separate instruction, and that's Step 2.

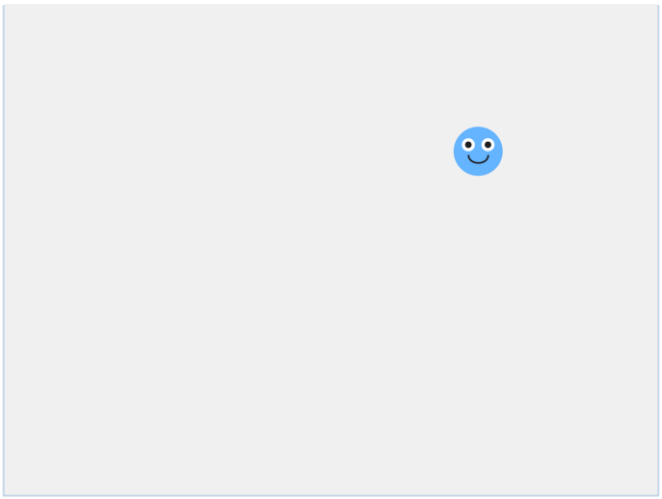
Step 2 - Take the step

Build it. In `draw()`, add `player.move();` on the line **before** `player.drawSprite();`:

```
void draw() {
  background(240);
  player.move();        // update position (do this BEFORE drawing)
  player.drawSprite();  // draw at new position
}
```

Run it, and watch.

> **Check.** Your character drifts up and to the right.



One second in: 60 frames, 180 pixels right and 120 up.

> **Watch out.** A character with a velocity and no `move()` sits perfectly still. This is the most common mistake in the chapter. When your character is frozen, look for `move()` in `draw()` first.

Step 3 - See every frame

`background(240)` paints over the whole window at the start of every frame. Here's what your character looks like without it.

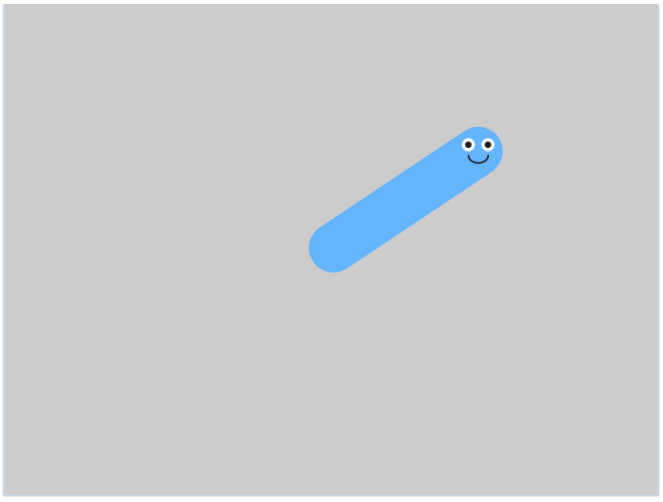
Build it. Put `//` at the front of the `background` line, so it reads:

```
  //background(240);
```

> **Aside: commenting out**
>
> `//` turns a whole line into a comment, so Processing skips it. That's called **commenting out** a line. It's the quickest way to switch something off and keep it handy: take the `//` away, and the line is back.

Run it.

> **Check.** A long blue stripe, with your character's face at the front end.



Every frame, still on the screen.

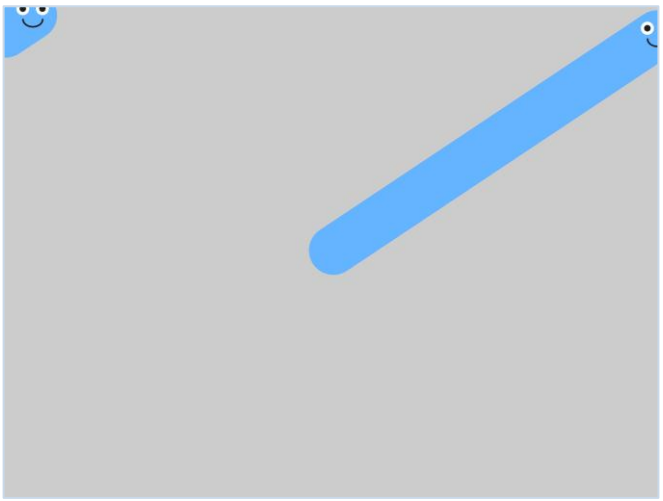
That stripe is every frame of the flipbook from Chapter 1, at once. Each frame drew your character one step further along, and with nothing painting over the old ones, they're all still there. The grey is the color Processing starts a window with.

Leave the `//` there for one more step.

Step 4 - Off the edge

Let it run, and keep watching when your character reaches the edge of the window.

> **Check.** It slides off one edge and comes back in on the opposite side.



Off the right edge, back in on the left. The stripe starts over on the other side.

That's called **wrapping**. Off the right, back in on the left. Off the top, back in at the bottom. You didn't write that. The library does it for you, inside `move()`.

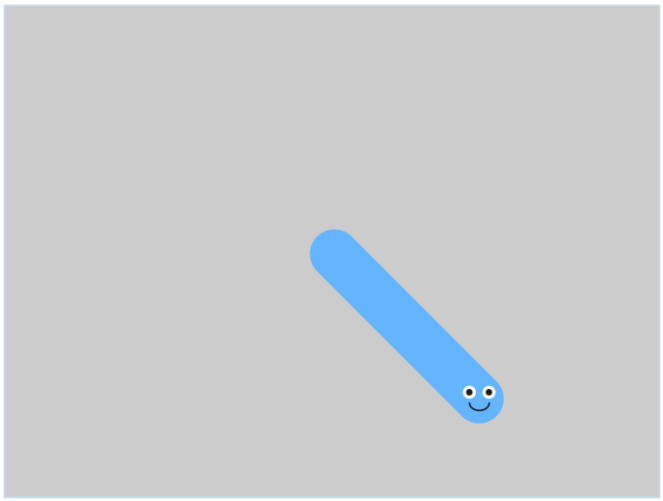
Step 5 - Faster, slower, and which way

Keep the `//` in front of `background` for this step, so each try draws its own path.

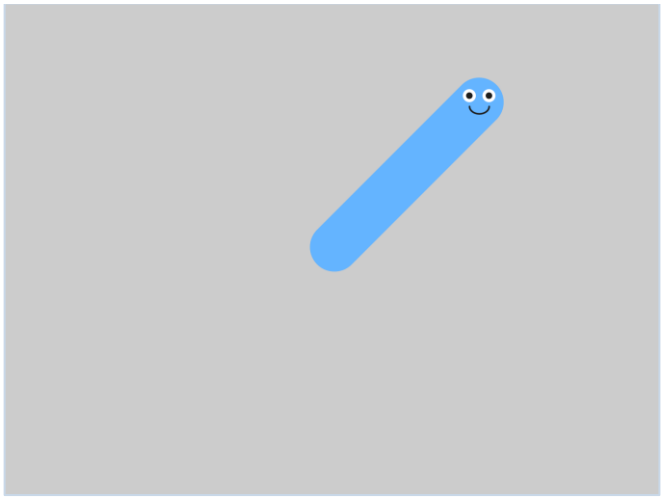
Build it. Change the velocity to `(8, 0)` and run. Then `(0.5, 0)` and run. Fast, and slow.

Now predict before you run. Which way does `setVelocity(3, 3)` go? Which way does `setVelocity(3, -3)` go?

> **Check.** `(3, 3)` heads down and to the right. `(3, -3)` heads up and to the right.



`setVelocity(3, 3)`: positive y, so down.



`setVelocity(3, -3)`: negative y, so up.

When `vx` and `vy` are the same size, the path is an exact diagonal.

Build it. Take the `//` off `background(240)`, and run. The stripes are gone, and your character is one face again.

Step 6 - A different direction every run

Build it. Replace both numbers in `setVelocity()` with `random(-5, 5)`:

```
player.setVelocity(random(-5, 5), random(-5, 5));
```

Press ▶ **Run** five times in a row.

Check. Your character takes off in a different direction, at a different speed, every time.

Aside: `random(a, b)`

`random(-5, 5)` picks a surprise number between `-5` and `5`, fresh every time it runs. It's usually not a whole number: it might be `2.7`, or `-0.4`. That's fine, because velocities are floats, like positions. Put `random()` anywhere a number goes, and you get a different number every run.

3.3 Move first, then draw

Here's a question to think about. What happens if you call `drawSprite()` **before** `move()`?

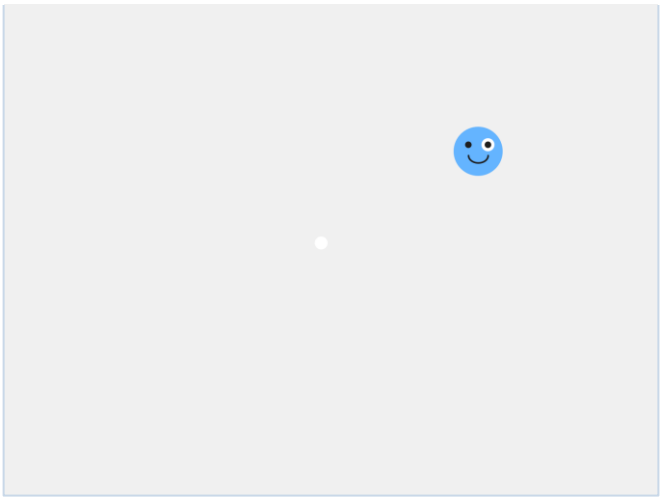
```
void draw() {
  background(240);
  player.drawSprite();
  player.move();
}
```

Make a guess before you read on.

Each frame, that version draws your character, and only then moves it. So it's always drawing your character where it *used to be*: one frame behind, forever.

Try it, and you'll see it looks the same. It is the same, one frame late. At sixty frames a second, one frame is too short to see. It's still a habit worth getting right now: **move, then draw**.

Watch out. If part of your character gets left behind when it starts moving, look at Chapter 2's Step 6. A shape drawn at plain numbers, like `ellipse(388, 292, 16, 16)`, sits still while the rest of the face moves. Measure it from `pos.x` and `pos.y`, and it comes along.



An eye at plain numbers, left behind in the middle of the window.



Part Three - Make it yours

3.4 A speed that fits

Your turn. Tune your character's velocity until it feels right for **your** character. A heavy monster lumbers. A bug darts. A ghost might drift slowly up. Pick numbers, or keep `random()`, and run it until it looks like your character moving.

Your turn. Change the background color to set a mood. A character drifting across dark blue looks completely different from the same character on white. Three numbers from the Color Selector work here, just as in Chapter 1.

3.5 Something else alive

Your turn. Optional: add a second blob with a velocity of its own, so there's something else moving on the screen. It goes in the same three places as the friend in Chapter 1, plus the two new lines from today:

1. At the top: `Blob friend;`
2. In `setup()`: a `new Blob(...)`, and then `friend.setVelocity(...)`.
3. In `draw()`: `friend.move();`, then `friend.drawSprite();`.

Check. Two things moving, each at its own speed. Here's one friend, slow, drifting left and a little down.



Your character, and a friend with `setVelocity(-1, 0.5)`.

Your turn. Optional: put the `//` back in front of `background()`, on purpose. Some of the best-looking sketches leave their trails on the screen.

Chapter summary

You started with a character that could move, waiting to be asked. You end with one that slides across the screen at a speed you chose, and wraps around the edges on its own.

You built	It's an example of
`player.setVelocity(3, -2)`	a **velocity**: two numbers, across and down, per frame
a velocity and no `move()`	a velocity is a plan, and `move()` carries it out
`player.move()` in `draw()`	taking one step every frame
`//background(240);`	**commenting out** a line, and every frame on the screen at once
your character, off the edge and back	**wrapping**, which the library does inside `move()`
`setVelocity(random(-5, 5), random(-5, 5))`	**`random()`**, a different number every run
`move()` before `drawSprite()`	move, then draw

New syntax in this chapter

Syntax	Where you met it
`setVelocity(vx, vy)`	3.1, Step 1
`move()`	3.2, Step 2
`//` in front of a line of code	Step 3
`random(a, b)`	Step 6

Vocabulary

commenting out -- putting `//` in front of a line of code, so Processing skips it. Take the `//` away to switch it back on. (Step 3)

`move()` -- the step. It adds your character's velocity to its position, once. It goes in `draw()`. (3.2)

`random(a, b)` -- a surprise number between `a` and `b`, fresh every time it runs. (Step 6)

velocity -- how fast something moves, and which way. Two numbers: `vx`, left and right, and `vy`, up and down. (3.1)

wrapping -- going off one edge of the window and coming back in on the opposite edge. (Step 4)

Check your understanding

1. `setVelocity(0, 4)`. Which way does your character go?
2. Your character has a velocity, and it won't move. What's missing?

3. Why does `move()` go in `draw()`, and `setVelocity()` go in `setup()`?
4. Where did the blue stripe in Step 3 come from?
5. What does `random(1, 3)` give you? Will it give you the same number next time?
6. You call `drawSprite()` before `move()`. What's different, and can you see it?

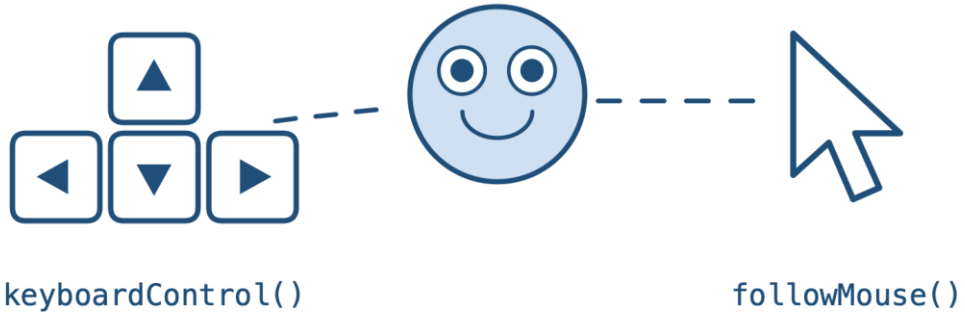
What's next

Your character moves now, all on its own, in whatever direction you gave it at the start. Once it's going, it goes where you pointed it.

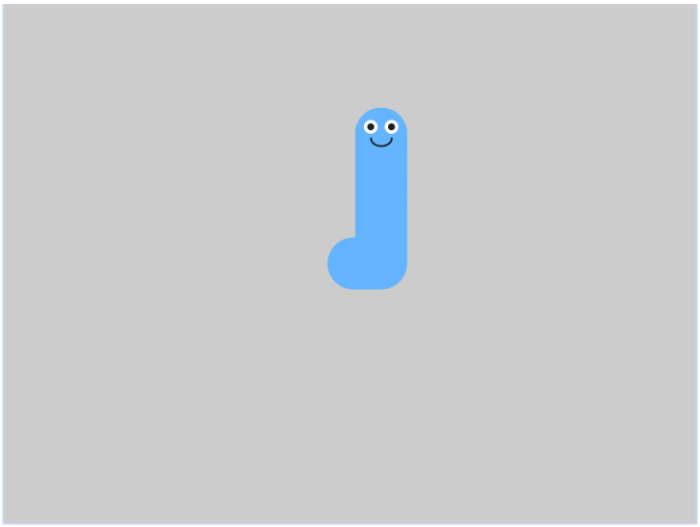
In Chapter 4 you take the controls. Your character follows your mouse, or goes where the arrow keys send it, and your game gets its player.

Chapter 4 - You're in Control

Your character stops doing its own thing and starts doing what you tell it.



Every key on the keyboard is a brake, except four.



```
void keyPressed() {
  player.keyboardControl();
}
```

Chapter 3 ended with a character that moves on its own, in whatever direction you gave it at the start. Once it's going, it goes where you pointed it.

Today you take the controls. There are two ways to do it, and you'll try both: your character can chase your mouse, or go where the arrow keys send it. Then you pick the one that fits your game.

This chapter does it in four moves.

Part	The idea
One	**Two kinds of input.** Some things you can ask about every frame. Some things happen once.
Two	**Option A: follow the mouse.** One line in `draw()`, and a speed in `setup()`.
Three	**Option B: the arrow keys.** A new kind of function, one Processing calls for you when a key goes down.
Four	**Make it yours.** Pick one, tune it, and add a click.

Leave your `MyCharacter` tab exactly as it is. Everything happens in the main tab.

Part One - Two kinds of input

4.1 All the time, or once

Input is information coming into your program from you: the mouse, the keyboard. There are two very different kinds, and telling them apart is the real idea of this chapter.

Some questions have an answer **all the time**. "Where is the mouse right now?" You can ask that on every frame, and there's always an answer. That's **continuous** input, and it belongs in `draw()`.

Other things happen **once**. "Did a key just go down?" That's an **event**. It has an answer only at the instant it happens, so Processing comes and tells you, by running a function you wrote for it.

Option A uses the first kind. Option B uses the second.

Part Two - Option A: follow the mouse

Step 1 - Chase the cursor

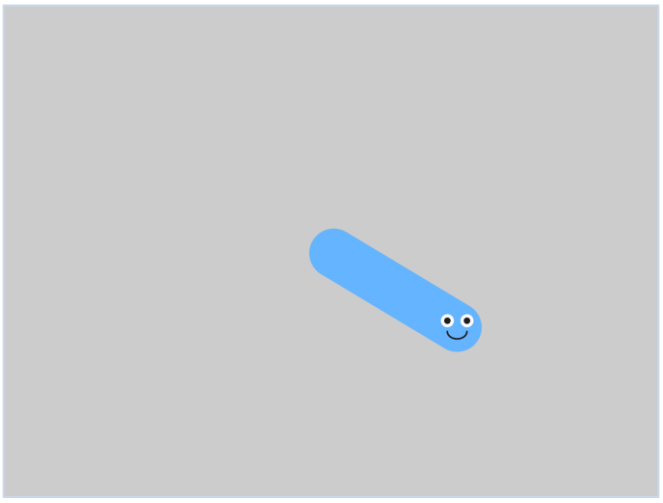
Build it. Open your sketch from Chapter 3. In `setup()`, set the velocity to `(4, 0)`. In `draw()`, replace `player.move();` with `player.followMouse();`:

```
void setup() {
  size(800, 600);
  player = new MyCharacter(this, width/2, height/2);
  player.setVelocity(4, 0);   // REQUIRED - this is the speed it chases at
}

void draw() {
  background(240);
  player.followMouse();
  player.drawSprite();
}
```

Run it, and move your mouse around the window.

> **Check.** Your character chases the mouse. When you hold the mouse still, your character slides straight to it.



The mouse held still at `(650, 450)`, with `background()` switched off, as in Chapter 3, to show the path.

`followMouse()` pulls your character a little way toward the cursor, every frame. Since it happens every frame, it goes in `draw()`.

> **Aside: `mouseX` and `mouseY`**
>
> Processing always knows where your mouse is. `mouseX` is how far across the window it is, and `mouseY` is how far down, measured the same way as everything else since Chapter 1. `followMouse()` reads them every frame. You'll use them yourself in 4.5.

Step 2 - The chase speed

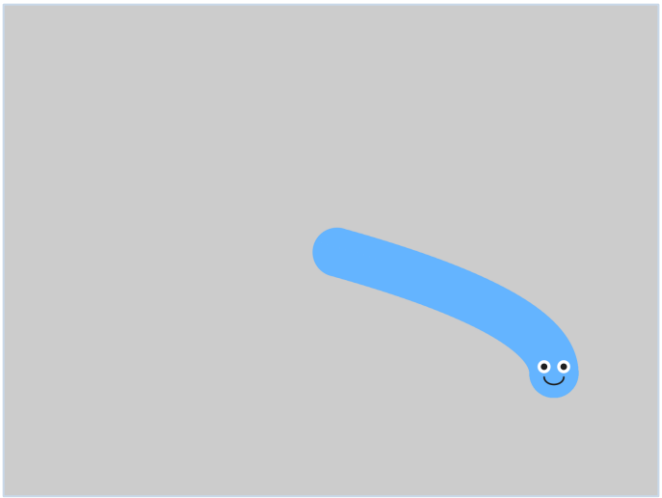
Build it. Change the `4` in `setVelocity` to `1`, and run. Then try `12`.

> **Check.** At `1` your character creeps after the mouse. At `12` it jumps to it.

`followMouse()` uses the **size** of your velocity as its chase speed, and ignores which way the velocity points. `setVelocity(5, 0)` and `setVelocity(0, 5)` chase at exactly the same speed.

> **Watch out.** With a velocity of `(0, 0)`, or no `setVelocity()` at all, `followMouse()` moves your character zero pixels, and it looks broken. When your character ignores the mouse, check `setup()` for a velocity first.

> **Watch out.** Use `followMouse()` *or* `move()`, one or the other. `followMouse()` moves the character itself. With both, `followMouse()` pulls it toward the mouse, and `move()` pushes it to the right by its velocity, every frame. Your character curves over and parks just to the right of the mouse, where the pull and the push cancel out.



`followMouse()` and `move()` together. It never quite gets to the mouse at `(650, 450)`.

Part Three - Option B: the arrow keys

Step 3 - Back to `move()`

Build it. Comment out the `followMouse()` line, and put `player.move();` back, above `drawSprite()`:

```
void draw() {
  background(240);
  // player.followMouse();
  player.move();
  player.drawSprite();
}
```

Keep the `setVelocity(4, 0)` in `setup()`.

Step 4 - A function for when a key goes down

Build it. At the very bottom of your sketch, after the closing brace of `draw()`, add a new function:

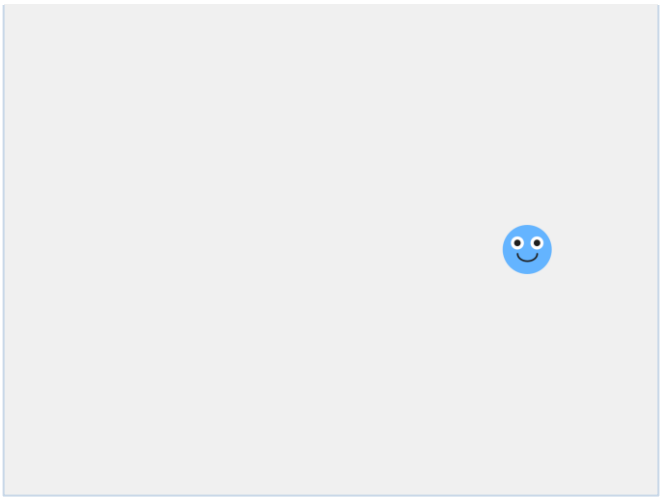
```
// Arrow keys control the character
void keyPressed() {
  player.keyboardControl();
}
```

> **Aside: event functions**
>
> `keyPressed()` is an **event function**. You write it, and Processing calls it for you, every time a key goes down. Its name has to be exactly `keyPressed`, the same way `setup` and `draw` have to be spelled exactly. There's one for the mouse, too: `mousePressed()` runs once, each time you click.

> **Watch out.** Click inside the sketch window before you press any keys. Until you do, the keys go to the code editor.

Run it, and wait a second before you touch anything.

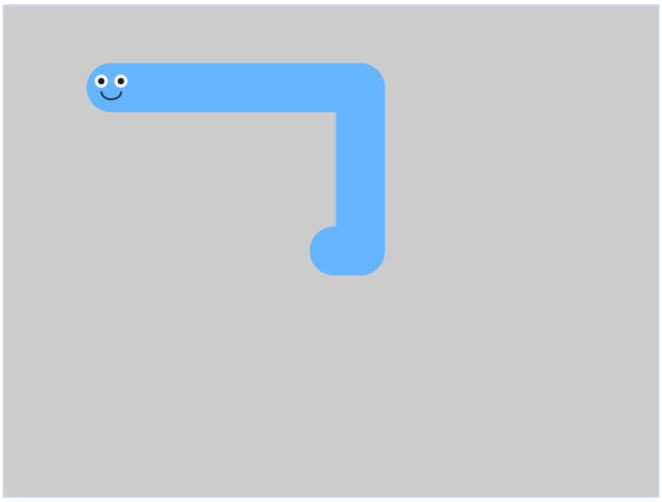
> **Check.** Your character drifts slowly to the right on its own.



One second in, before any key.

That's the `setVelocity(4, 0)` from Step 1, and `move()` is using it, the same as in Chapter 3. Now click in the window and use the arrow keys.

> **Check.** Each arrow sends your character that way.



Up, then left. A trail, as before.

Step 5 - Let go

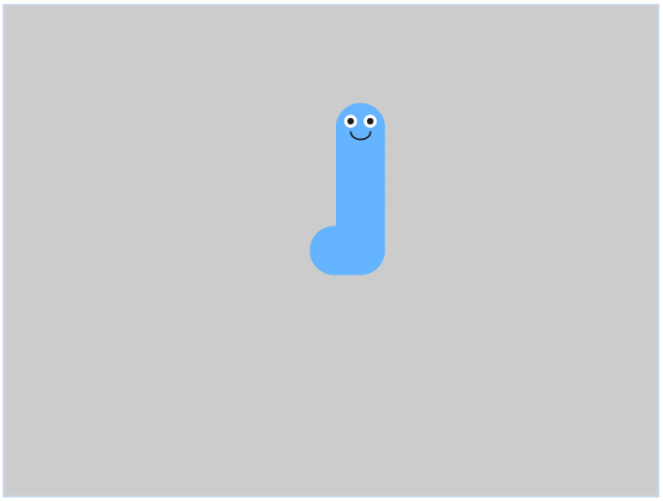
Build it. Press an arrow key, and let go.

> **Check.** Your character keeps going.

That's how it's meant to work. `keyboardControl()` **sets** your character's velocity, and letting go of the key leaves the velocity just as it was. So your character coasts, like it's on ice. Some games want exactly that.

Now press the space bar.

> **Check.** Your character stops dead.



Up, and then the space bar.

`keyboardControl()` looks at which key went down. An arrow sets a speed of 5 in that direction. Any other key sets the velocity to zero. So every key that isn't an arrow is a brake: the space bar, a letter, anything.

That 5 is fixed. Changing the numbers in `setVelocity()` changes how your character drifts before the first key. After that, it's always 5.

4.4 The two halves

Option B has two halves, and they're the two kinds of input from 4.1.

- `keyboardControl()`, in `keyPressed()`, runs once per key press. It **changes** the velocity. That's an event.
- `move()`, in `draw()`, runs every frame. It **uses** the velocity, to take a step.

Watch out. Switch to the keys and forget to put `move()` back, and your character sits still. Every key press sets a velocity, and only `move()` takes the step.

Part Four - Make it yours

4.5 Pick one

> **Argue it.** Mouse or keys, for your game? `followMouse()` is easier and more forgiving. `keyboardControl()` feels more like a real game. Each one gives you something the other one gives up, and that's a **trade-off**. Pick one, and be ready to say why it fits the game you want to make.

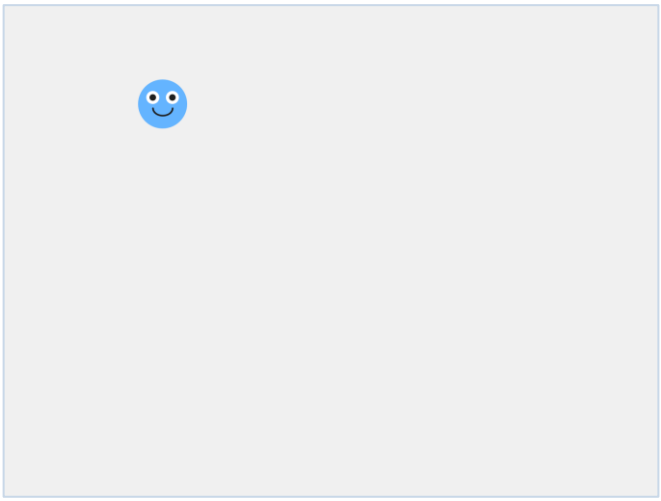
Your turn. Keep the control style you picked, and comment out or delete the other one. Then tune it: the chase speed for the mouse, or how you feel about the coasting for the keys. Whatever you choose today, you can change it later.

4.6 Something extra

Your turn. Optional: add a click ability. Put this at the bottom of your sketch:

```
void mousePressed() {
  player.moveTo(mouseX, mouseY);   // teleport to wherever you clicked
}
```

> **Check.** Click anywhere, and your character jumps straight there.



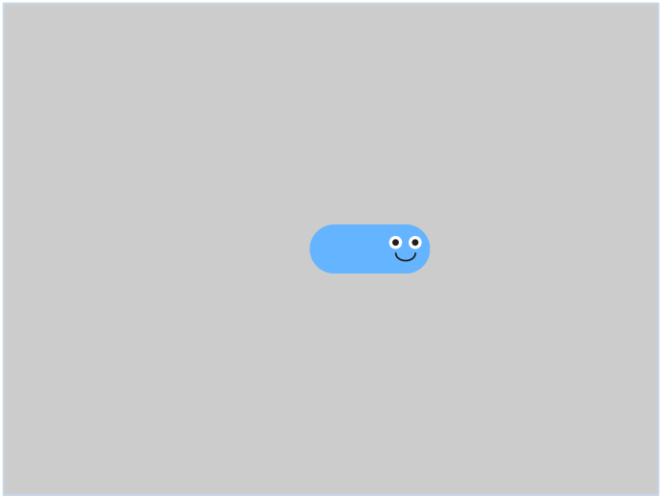
A click at `(150, 120)`.

`moveTo()` picks your character up and puts it down at the spot you give it. It works with either control style. With the keys, it gives you a teleport.

Your turn. Optional: make the character stop when you let go of the key. Add one more event function:

```
void keyReleased() {
  player.setVelocity(0, 0);
}
```

Check. Press an arrow and let go. Your character goes, then stops.



Right, and then let go.

Try it both ways, coasting and stopping. Which one suits your character?

Your turn. Optional: add a second blob that moves on its own, the way you did in Chapter 3, so there's something else on the screen while you steer.

Chapter summary

You started with a character that went where you pointed it, once. You end with one you steer, with the mouse or the keys, and a reason for the one you picked.

You built	It's an example of
`player.followMouse()` in `draw()`	**continuous** input, asked about every frame
`setVelocity(4, 0)` for the chase	`followMouse()` uses the size of the velocity as its speed
`void keyPressed() { player.keyboardControl(); }`	an **event function**, run once per key press
`move()` in `draw()`, with the keys	one half changes the velocity, the other half uses it
letting go, and the space bar	coasting, and a brake on every other key
`mousePressed()` and `moveTo(mouseX, mouseY)`	a second event function, and the mouse's position
mouse or keys, picked	a **trade-off**

New syntax in this chapter

Syntax	Where you met it
`followMouse()`	Step 1
`mouseX`, `mouseY`	Step 1
`void keyPressed() { }`	Step 4
`keyboardControl()`	Step 4
`void mousePressed() { }`	Step 4, 4.6
`moveTo(x, y)`	4.6
`void keyReleased() { }`	4.6

Vocabulary

continuous input -- something your program can ask about every frame, like where the mouse is. It goes in `draw()`. (4.1)

event -- something that happens once, at one instant, like a key going down or a click. (4.1)

event function -- a function you write and Processing calls for you when an event happens: `keyPressed()`, `mousePressed()`, `keyReleased()`. (Step 4)

input -- information coming into your program from you: the mouse, the keyboard. (4.1)

trade-off -- a choice where each option gives you something the other one gives up. (4.5)

Check your understanding

1. Why does `followMouse()` go in `draw()`, and `keyboardControl()` go in `keyPressed()`?
2. Your character won't chase the mouse. What do you check first?
3. With the arrow keys, what does `keyboardControl()` change, and what does `move()` do every frame?
4. Why does your character keep going after you let go of the key?
5. What happens when you press the space bar, and why?
6. You chose mouse or keys. What does your choice give you, and what does it give up?

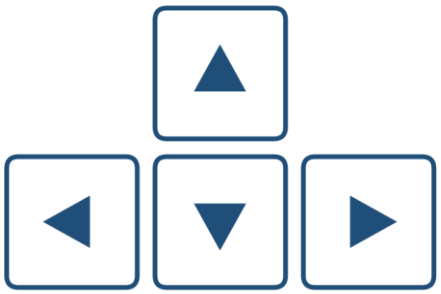
What's next

Your character goes where you tell it now, across an empty grey window.

In Chapter 5 you build it a world: ground, scenery that drifts past, and a coin floating through it, waiting for Chapter 6.

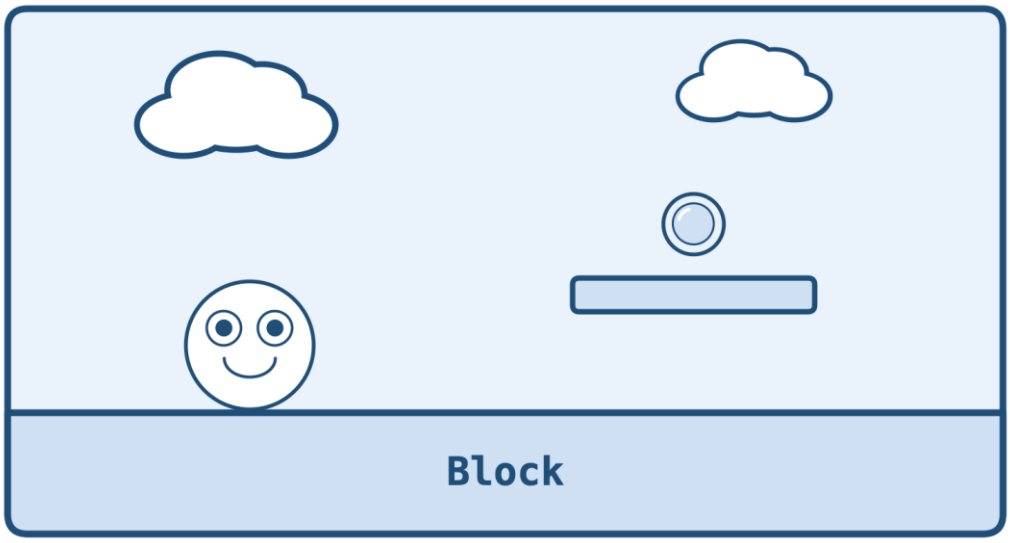
Unit 2 - Your Game

Chapters 5 to 7



Chapter 5 - Build Your World

Your character needs somewhere to be. Today you build the place your game happens in.



Your character walked down to the ground. It's still there.



```
player.drawSprite();
ground.drawSprite();
```

Chapter 4 ended with a character you can steer, across an empty grey window.

Today it gets a world: ground to stand on, a sky, clouds drifting past, and a coin floating through, waiting for Chapter 6. There's very little new code today. Most of the chapter is you, deciding what your world looks like.

The one new idea is **draw order**: the order of your lines decides what appears in front.

This chapter does it in four moves.

Part	The idea
One	**Sketch it first.** Three minutes on paper, and the one new kind of sprite.
Two	**Ground, and the order you draw in.** A `Block` for the ground, and the rule: background first, player last.
Three	**Things in the world.** Clouds in a tab of their own, and a coin.
Four	**Make it yours.** A world with a theme that's yours.

You keep everything from Chapters 1 to 4, and you leave your `MyCharacter` tab exactly as it is.

Part One - Sketch it first

5.1 On paper

Your turn. Before any code, sketch the world your game takes place in, on paper. What's in the background? Is there ground? Sky? What drifts past? Three minutes, then show a neighbor.

It earns its three minutes. You'll build a better world with the picture in front of you.

5.2 `Block`, the rectangle sprite

Your character is a Blob, and so far everything in your sketch has been one. The library has another kind of sprite for scenery: a **`Block`**, a rectangle.

A `Block` starts with zero velocity, so it sits exactly where you put it. That makes it good for ground, walls, a platform, a ceiling.

```
ground = new Block(this, 0, height - 70, width, 70, color(80, 160, 60));
```

Six things, in this order:

	What it is
`this`	the sketch, the same as a Blob
`0`	x of the **top-left corner**: the left edge
`height - 70`	y of the **top-left corner**: 70 pixels up from the bottom
`width`	how wide: the whole window
`70`	how tall
`color(80, 160, 60)`	its color, a grassy green

So this one fills the bottom 70 pixels of the window, edge to edge.

> **Watch out.** For a `Block`, `x` and `y` are the **top-left corner**. For a Blob they were the **center**. This catches everyone once.

A `Block` is scenery. Your character passes straight through it, the same as it passes through everything else in your world right now.

Part Two - Ground, and the order you draw in

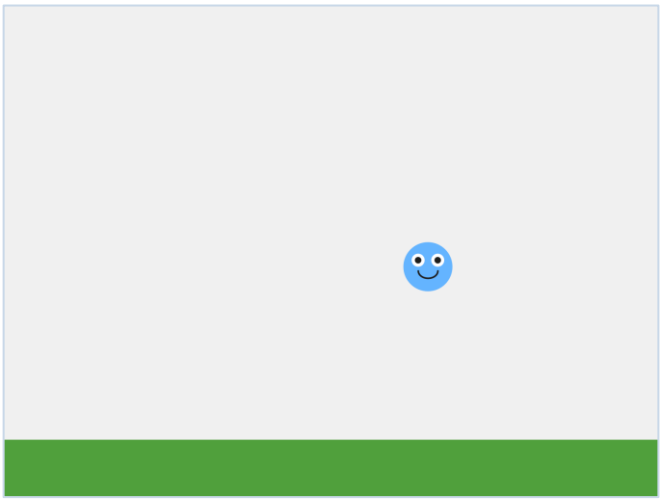
Step 1 - Ground

Build it. Open your sketch from Chapter 4. Add the ground in three places, the same three as every object since Chapter 1:

```
Block ground;                    // at the top, with player
// in setup():  x, y is the TOP-LEFT corner
ground = new Block(this, 0, height - 70, width, 70, color(80, 160, 60));
// in draw(), right after background()
ground.drawSprite();
```

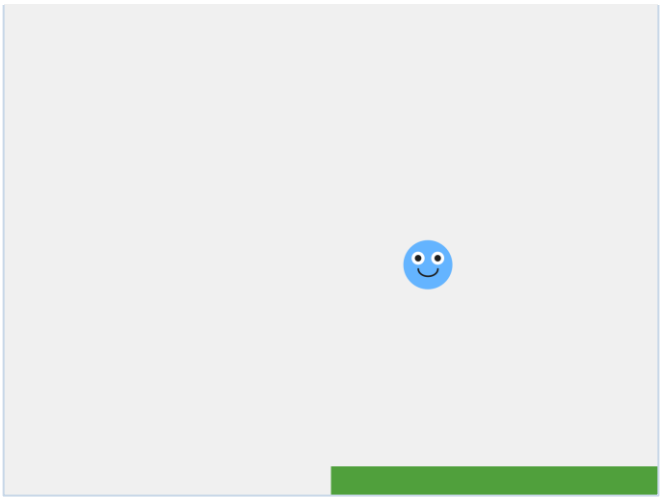
Run it.

Check. A strip of green along the bottom of the window.



Ground, on Chapter 4's grey.

Watch out. Use the ground's center for its `x` and `y`, the way you would for a Blob, and the ground lands in the wrong spot. Here it is with `width/2, height - 35`, the middle of where the ground should be.



The corner, put where the center should be. Half the ground hangs off the window.

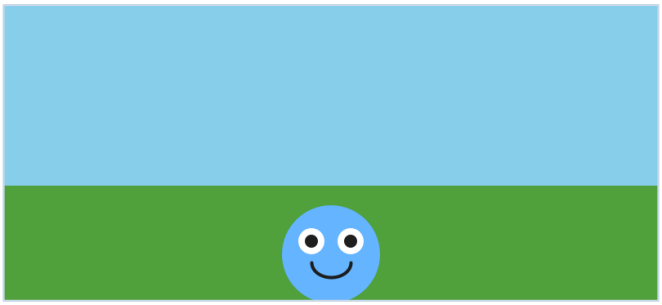
Step 2 - A sky

Build it. Change `background(240)` to a color that fits your world. The Canvas starter uses a sky blue, `background(135, 206, 235)`. Deep space black, ocean teal, or lava red work too.

Step 3 - Draw order, on purpose

Build it. Move `ground.drawSprite();` to the very last line of `draw()`, after your player. Run it, and steer your player down to the ground.

> **Check.** Your player disappears underneath the ground.



Ground drawn first, and the player is in front.



Ground drawn last, and the player is behind it. Same spot.

Move the line back, right after `background()`, and say out loud what happened.

Processing draws like paint on a canvas. Whatever you draw **later** covers whatever you drew earlier. You saw it inside your character in Chapter 2, when the body drawn last painted over the face. Now it's the whole world. The order of the calls in `draw()` *is* the layering.

Rule of thumb: background first, player last.

> **Watch out.** `background()` paints over the whole window, so it goes at the **top** of `draw()`. Call it halfway down, and it erases everything you drew above it that frame. Move it below your clouds, in Step 4, and the frame comes out exactly like one with no clouds in it at all.

Part Three - Things in the world

Step 4 - Clouds, in a tab of their own

Build it. Make a new tab called `Cloud`, the way you made `MyCharacter` in Chapter 2. Paste this in:

```
// Cloud - a second sprite class.  Redesign it to fit your world!
class Cloud extends Blob {

  Cloud(PApplet p, float x, float y, float speed) {
    super(p, x, y, 40, color(255));  // radius 40 used for collision if ever
needed
    setVelocity(speed, 0);           // drift slowly to the right; no vertical
movement
  }

  void drawSprite() {
    noStroke();

    // Bottom puffs - the wide base of the cloud
    fill(250, 250, 255);
    ellipse(pos.x,      pos.y,      72, 46);
    ellipse(pos.x - 31, pos.y + 13, 50, 33);
    ellipse(pos.x + 33, pos.y + 13, 55, 33);

    // Top puffs - slightly greyer for a sense of depth
    fill(235, 238, 248);
    ellipse(pos.x - 13, pos.y - 17, 54, 38);
    ellipse(pos.x + 17, pos.y - 11, 46, 33);
  }
}
```

Read it against your `MyCharacter` tab. A class line with `extends Blob`, a `super(...)` handshake, and a `drawSprite()` full of shapes measured from `pos.x` and `pos.y`. You've written every piece of this already.

Two things are new, both in the constructor.

> **Aside: a constructor with a number of its own**
>
> `Cloud(PApplet p, float x, float y, float speed)` takes a fourth thing, the cloud's speed. Then the constructor uses it: `setVelocity(speed, 0)`. So every cloud sets its own velocity when it's made, right in its own constructor.

Build it. In the main tab, add two clouds, in the same three places:

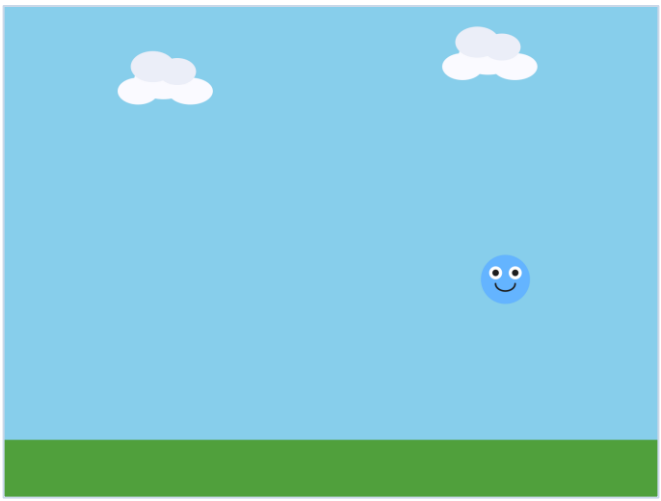
```
Cloud cloud1;                              // 1. declare it at the top
Cloud cloud2;
```

```
cloud1 = new Cloud(this, 150, 90, 0.5);    // 2. create it in setup()
cloud2 = new Cloud(this, 520, 60, 0.8);
cloud1.move();                             // 3. move and draw it in draw()
cloud1.drawSprite();
cloud2.move();
cloud2.drawSprite();
```

The move-and-draw lines go **right after** `background()`, before the ground. Clouds are far away, so they're drawn first.

Run it.

> **Check.** Two clouds drift slowly to the right, each at its own speed, and wrap around like everything else.



Sky, two clouds, ground, and your player.

> **Watch out.** Make a cloud in `draw()` by mistake, and it freezes. The `new Cloud(...)` line runs every frame, so the cloud is built again at its starting spot sixty times a second. Making things goes in `setup()`.

Each cloud is its own variable. Want a third? Declare `cloud3`, create it, move it, draw it. If you find yourself wanting fifty of something, ask your teacher about Bonus D.

Step 5 - A coin

Build it. Add a coin. Declare `Blob coin;` at the top. Create it in `setup()`:

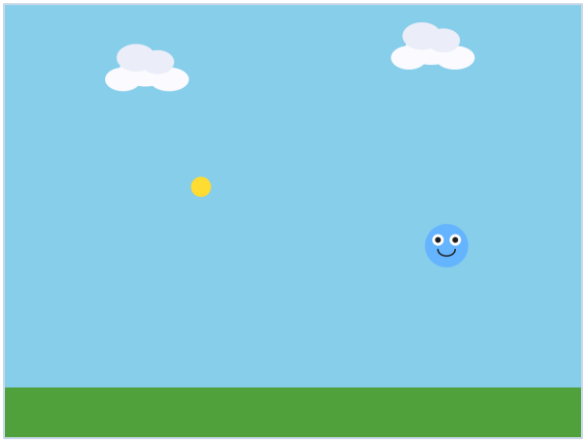
```
// A coin drifting through the world.  You collect it in Lesson 6.
coin = new Blob(this, random(width), random(100, height - 150),
                14, random(1, 2.5), random(-1.5, 1.5),
                color(255, 220, 50));
```

Aside: a Blob with seven numbers

This Blob gets seven things where Chapter 1's got five: the sketch, x, y, the radius, then **two more**, its velocity, and then its color. It's the same as making a Blob and calling `setVelocity()` on it, in one line. Every number here except the radius and color is `random()`, so the coin starts somewhere different, going a different way, every run.

Move and draw it in `draw()`, **after** the ground and **before** the player. Run it.

Check. A gold coin drifting through your world. Steer your player into it. Nothing happens. That's Chapter 6.



The finished world, as the Canvas starter builds it. Your coin will be somewhere else.

Here's the whole `draw()`, back to front:

```
void draw() {
  background(135, 206, 235);   // 1. sky - furthest back

  cloud1.move();               // 2. clouds drift behind everything
  cloud1.drawSprite();
  cloud2.move();
  cloud2.drawSprite();

  ground.drawSprite();         // 3. ground in front of the sky

  coin.move();                 // 4. things in the world
  coin.drawSprite();

  player.followMouse();        // 5. player last, so it is always in front
  player.drawSprite();
}
```

If you drive with the arrow keys, your step 5 is `player.move();` instead.

Part Four - Make it yours

5.5 A place

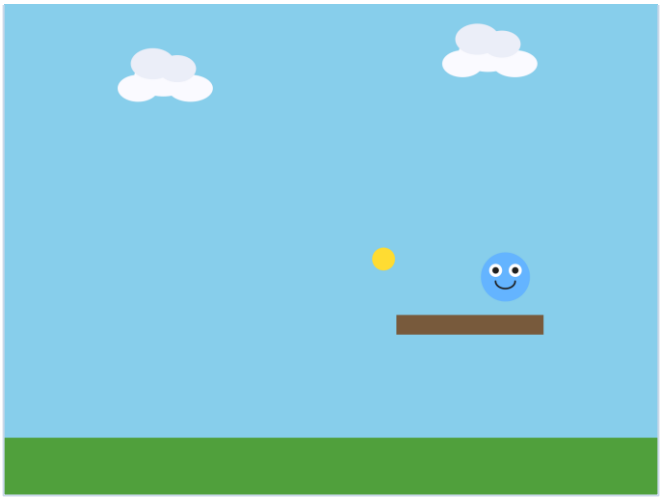
The rest of the time is yours. Play your world as you go. Does it look like a place? Keep going until it does.

Your turn. Give your world a theme. Space? Underwater? Lava? A cave? A candy world? Start with the background.

Your turn. Make everything belong together. A gold coin in a space world looks wrong. Make it a star, or a gem, or an asteroid.

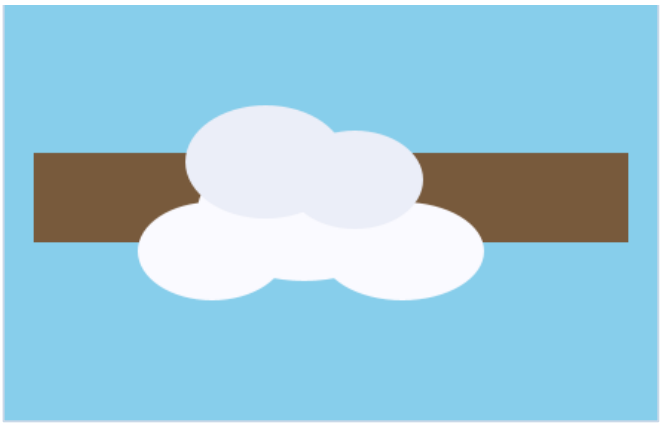
Your turn. Redesign the `Cloud` tab completely: a star, a bubble, a bird, a fish. Rename the class if you like. Or write a brand-new decoration in a new tab, a `Tree`, a `Rock`, with the same `extends Blob` pattern as your character.

Your turn. Add more `Block`s: a platform, a wall, a border around your scene. Pick where each one goes in `draw()`.

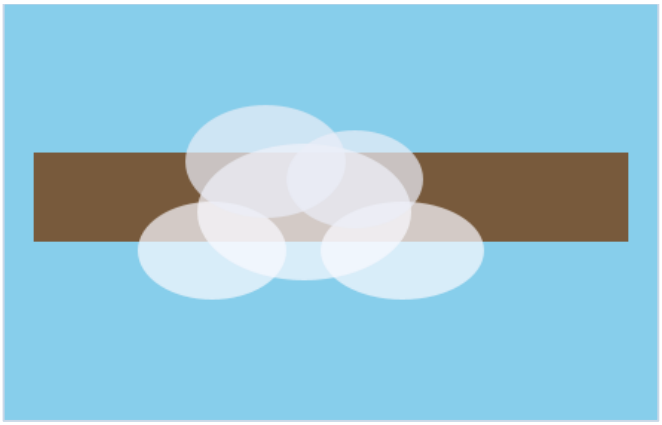


A platform, `new Block(this, 480, 380, 180, 24, color(120, 90, 60))`.

Your turn. Make far-away things look far away. `fill()` and `color()` take a fourth number, **alpha**, for how see-through something is: `0` is invisible, and `255` is solid. In your `Cloud` tab, try `fill(250, 250, 255, 180)`.



A solid cloud in front of a block.



The same cloud with an alpha of `180`. You can see the block through it.

Speed does the same work. A world feels deep when the far-away things move slowly and the close things move fast. Ground sits still. Clouds drift. The coin darts. Your player goes wherever you send it.

Your turn. Optional: build a real parallax effect. Three clouds at three different speeds: the slowest one faintest and drawn first, the fastest one solid and drawn last. It's the oldest trick in 2D games, and it still works.

Chapter summary

You started with a character in an empty window. You end with a world: a sky, ground, things drifting past, and a coin, all in the order you chose to draw them.

You built	It's an example of
`new Block(this, 0, height - 70, width, 70, ...)`	a `Block`: a rectangle, placed by its **top-left corner**
the ground drawn last	**draw order**: later covers earlier
background first, player last	the order of `draw()` is the layering
the `Cloud` tab	a second class, built exactly like your character
`Cloud(PApplet p, float x, float y, float speed)`	a constructor with a number of its own
`new Blob(this, x, y, 14, vx, vy, color)`	a Blob made with its velocity
`fill(250, 250, 255, 180)`	**alpha**, how see-through

New syntax in this chapter

Syntax	Where you met it
`Block`, `new Block(this, x, y, w, h, color)`	5.2, Step 1
`height - 70`	5.2
a constructor's own extra number, and `setVelocity()` inside it	Step 4
`new Blob(this, x, y, radius, vx, vy, color)`	Step 5
`fill(r, g, b, alpha)`	5.5

Vocabulary

alpha -- a fourth number in `fill()` or `color()` for how see-through something is. `0` is invisible, `255` is solid. (5.5)

`Block` -- the library's rectangle sprite. Good for ground, walls, and platforms. Its `x` and `y` are its top-left corner. (5.2)

draw order -- the order you draw things in. Whatever is drawn later appears in front. (Step 3)

Check your understanding

1. A `Block` at `new Block(this, 100, 100, 50, 50, ...)`. Where is its top-left corner? Where is its middle?
2. Your player disappears when it walks onto the ground. What do you check?
3. Why do the clouds get drawn before the ground?
4. Your cloud won't move. It's in the right place in `draw()`. Where else do you look?

5. What do the seven numbers in the coin's `new Blob(...)` mean?
6. Name one draw-order decision you made in your world, and what it changed.

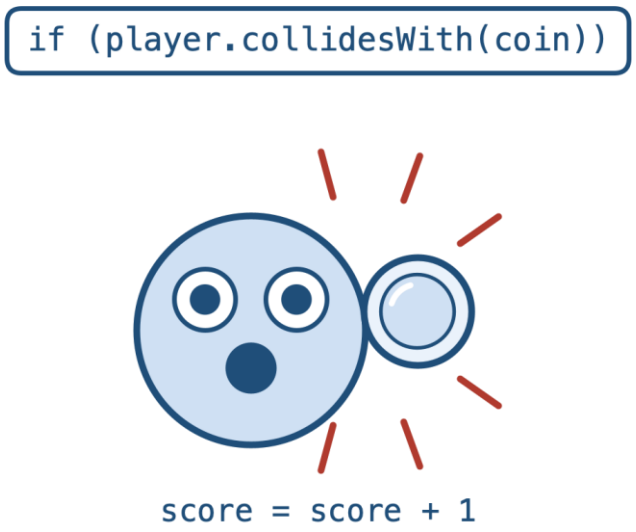
What's next

There's a coin drifting through your world, and touching it does nothing. What *should* happen?

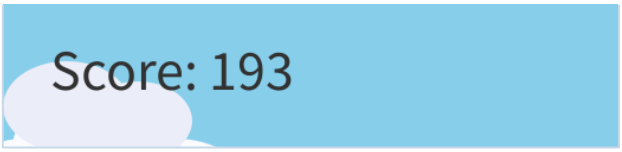
Chapter 6 is where your game starts paying attention. Touch the coin and score a point. Touch something dangerous, and the game ends.

Chapter 6 - Collisions: If This, Then That

Things finally touch each other, and your program decides what happens next. This is where it becomes a game.



One coin. A few seconds. A very good score.




```
if (player.collidesWith(coin)) {
  score = score + 1;   // add a point
}
```

Chapter 5 ended with a coin drifting through your world, and touching it does nothing. What *should* happen?

Today your sketch starts paying attention. Touch the coin, and you score a point. Touch something dangerous, and the game ends.

The new idea is `if`: making something happen only when it's true. The library can tell when two things are touching, and that gives `if` something to be true about.

This chapter takes two meetings, and does it in four moves.

Part	The idea
One	**Are they touching?** A yes-or-no question, and an `if` to act on the answer.
Two	**Keep score.** A number that remembers, on the screen, one point per touch.
Three	**A game that can end.** A hazard, a true-or-false, and two screens.
Four	**Make it yours.** Harder, easier, and yours.

Parts One and Two are the first meeting. Parts Three and Four are the second.

Part One - Are they touching?

6.1 A yes-or-no question

```
player.collidesWith(coin)
```

`collidesWith()` asks one question: *are these two touching?* The library checks whether their circles overlap, and hands back `true` or `false`.

Your character has `collidesWith()` because you wrote `class MyCharacter extends Blob` back in Chapter 2. It's one of the things a Blob can do, so your character got it for free, and you never had to write it.

Remember the `30` in your `super(...)` line? In Chapter 2 you changed it, and nothing on the screen changed. This is what it's for. That number is the radius of an invisible circle around your character, called its **hitbox**, and `collidesWith()` uses that circle and only that circle. Your drawing can be any size you like.

6.2 `if`

An `if` makes something happen only when a question's answer is yes:

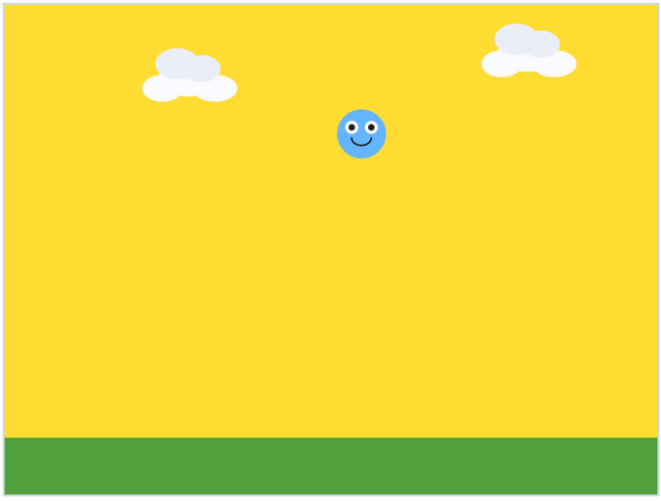
```
if (player.collidesWith(coin)) {
  background(255, 220, 50);   // only happens while they are touching
}
```

- The part in the parentheses is the **condition**, the yes-or-no question.
- The part in the braces is what happens when the condition is true.
- When it's false, Processing skips straight past the braces.

Step 1 - The gold flash

Build it. Open your sketch from Chapter 5. In `draw()`, right after your `background()` line, add the `if` above. Run it, and touch the coin.

> **Check.** The sky flashes gold while you touch the coin, and goes back to blue when you let go.



Touching the coin.

`draw()` runs sixty times a second, so it asks *are they touching?* sixty times a second. On every frame the answer is yes, the sky is gold.

Step 2 - Generous or precise

Your turn. If touching the coin feels too hard or too easy, open your `MyCharacter` tab and change the radius in the `super(...)` line. A bigger radius touches things from further away, which feels generous. A smaller one means you have to be right on top of something, which feels precise. Which do you want your game to be?

Part Two - Keep score

Step 3 - A number that remembers

Build it. Delete the gold flash. Then, at the very top of your sketch, next to your other variables, add:

```
int score = 0;      // a whole number, starts at zero
```

> **Aside: `int`**
>
> `int` is the kind of variable that holds a whole number: `0`, `7`, `-3`. No decimal point. `int score = 0;` makes a box called `score` and puts a `0` in it, on the same line.

A score has to remember something from one frame to the next. So it goes at the very **top** of your sketch, outside `setup()` and outside `draw()`. Up there, every function in your sketch can see it.

> **Watch out.** Put `int score = 0;` inside `setup()`, and the box only exists while `setup()` runs. `draw()` can't see it, and the sketch stops with:
>
> ```
> score cannot be resolved to a variable
> ```
>
> Move it back to the top.

Step 4 - The score on the screen

Build it. At the bottom of `draw()`, add three lines:

```
fill(50);                          // the text color
textSize(24);                      // how big
text("Score: " + score, 20, 36);   // what, then x, then y
```

Run it.

> **Check.** `Score: 0` in the top-left corner.



The score.

> **Aside: `text()`**
>
> `text()` draws words on the screen. You give it what to say, then where: x, then y. The `+` glues the words `"Score: "` and the number in `score` together into one piece of text. Set `fill()` **before** `text()`, or your words come out in whatever color the last shape used.

Build it. Test it by cheating. Change `int score = 0;` to `int score = 7;`, and run.

> **Check.** `Score: 7`.



The cheat.

Setting a value by hand to check that the display works is a real technique. Programmers do it all the time. Set it back to `0`.

Step 5 - A point per touch

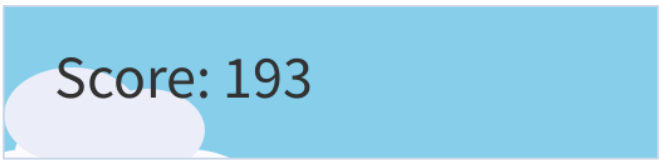
Build it. Above the three score lines, add the collision `if` again, this time with a point inside:

```
if (player.collidesWith(coin)) {
  score = score + 1;   // add a point
}
```

> **Aside: `score = score + 1`**
>
> Read it right to left. Take what's in `score`, add one, and put the answer back in `score`. The `=` means *make this equal to that*.

Run it, and sit your player on the coin. Watch the score.

> **Check.** The score races upward, as long as you're touching the coin.



A few seconds of touching.

Why is it doing that? Make a guess before you read on.

`draw()` asks *are they touching?* every frame. For as long as you sit on the coin, the answer is yes, and every yes is a point. That's a point for every frame: sixty a second, if you sit right on it.

Step 6 - One touch, one point

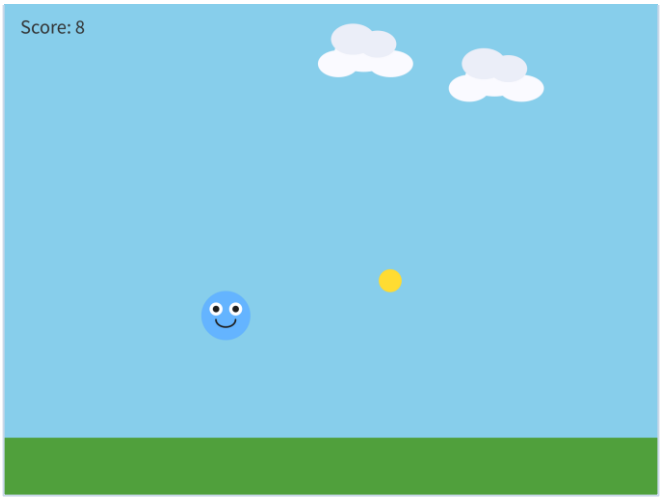
The fix is to end the touch the moment you count it. Move the coin somewhere else.

Build it. Inside the same `if`, after the point, add:

```
coin.moveTo(random(width), random(100, height - 150));  // coin jumps somewhere new
```

Run it.

> **Check.** Touch the coin, and you score one point, and the coin jumps somewhere new.



Eight coins in fifteen seconds.

You met `moveTo()` in Chapter 4, teleporting your character to a click. Here it teleports the coin, and `random()` picks the spot, so it lands somewhere new every time.

Your turn. Change the coin's speed and size, and style the score so it fits your world. The score's `fill()` and `textSize()` are yours.

Part Three - A game that can end

Last meeting your character started scoring points. Today it gets something to avoid, and your game gets a way to end.

Build it. Before you change anything today, save a copy: **File → Save As**, and put a number on the end of the name, like `MyGame2`. Your Day 1 game stays safe in the old one. Do the same any time you're about to make a big change, or paste in code from Sparky. If something breaks, the old version is still there.

If your sketch from last time is broken, the Canvas page has a Day 1 checkpoint. Paste it into your main tab, keep your own `MyCharacter` and `Cloud` tabs, and you're caught up.

Step 7 - A hazard

Build it. Add one more Blob, in the same three places as everything else. Declare `Blob hazard;` at the top. Create it in `setup()`:

```
// The hazard starts in the top-left corner, away from the player
hazard = new Blob(this, 80, 80, 22, 3, 2, color(220, 60, 60));
```

In `draw()`, move and draw it, right after the coin.

It starts in a corner on purpose. A hazard made right on top of your player would end the game the instant it started.

Step 8 - True or false

Build it. At the top of your sketch, add:

```
boolean gameOver = false;   // true or false, starts false
```

> **Aside: `boolean`**
>
> A **`boolean`** is the simplest variable there is. It's `true`, or it's `false`, and that's all. Here it remembers one thing: is the game over?

Then, after the coin's `if`, add one for the hazard:

```
if (player.collidesWith(hazard)) {
  gameOver = true;
}
```

Run it, and touch the hazard.

> **Check.** Nothing seems to happen. The game keeps going.

The variable changed. `gameOver` is `true` now, and nothing in your sketch reads it yet. The next two Steps give it a reader.

Step 9 - A function of your own

`draw()` is getting long. Before it gets longer, you're going to give the game part of it a name of its own.

Build it. Below the closing brace of `draw()`, make a new function called `playGame()`. Move into it your coin, your hazard, your player, both collision `if`s, and the score display. Leave the sky, clouds, and ground in `draw()`.

```
void playGame() {
  coin.move();
  coin.drawSprite();

  hazard.move();
  hazard.drawSprite();

  player.followMouse();
  player.drawSprite();

  // Touching the coin:  score a point, and the coin jumps somewhere new
  if (player.collidesWith(coin)) {
    score = score + 1;
    coin.moveTo(random(width), random(100, height - 150));
  }

  // Touching the hazard:  the game is over
  if (player.collidesWith(hazard)) {
    gameOver = true;
  }

  fill(50);
  textSize(24);
  text("Score: " + score, 20, 36);
}
```

In `draw()`, after the scenery, call it:

```
  playGame();
```

Aside: writing your own function

`void playGame() { ... }` makes a function, the same shape as `setup()` and `draw()`. The difference is who calls it. Processing calls `setup()` and `draw()` for you. This one runs when your sketch says its name: `playGame();`. You can name a function anything, so name it what it does.

Run it.

Check. Everything looks exactly the same as before.

That means it worked. Moving code into a function of its own is called **refactoring**, and a refactor that leaves the screen exactly the same is a success.

Step 10 - Two screens

Build it. Below `playGame()`, add a second function, for the end of the game:

```
void showGameOver() {
  textAlign(CENTER, CENTER);
  fill(220, 60, 60);
  textSize(52);
  text("Game Over", width/2, height/2 - 20);
  fill(50);
  textSize(22);
  text("Final score: " + score, width/2, height/2 + 26);
  textAlign(LEFT, BASELINE);   // put text alignment back to normal
}
```

Then, in `draw()`, replace `playGame();` with this:

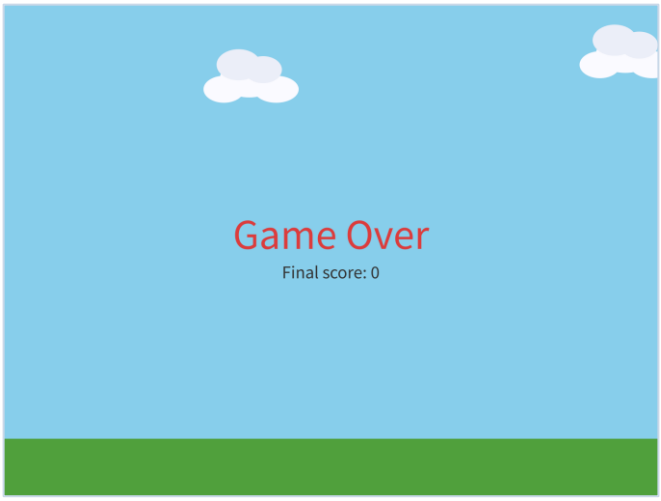
```
  if (!gameOver) {
    playGame();       // still playing
  } else {
    showGameOver();   // it's over
  }
```

Aside: `!` and `else`

`!` means **not**. So `!gameOver` reads *not game over*: still playing. `else` is the other half of an `if`. When the condition is true, the first braces run. When it's false, the braces after `else` run instead. Exactly one of the two runs, every frame.

Run it, and touch the hazard.

Check. The game stops, and *Game Over* comes up with your final score.



Game over. The sky, clouds, and ground are still there, because they're still in `draw()`.

This is the big idea of today. Your game is always in a **mode**. Right now there are two: playing, and game over. The `boolean` remembers which mode you're in, and the `if` / `else` in `draw()` picks which one to run.

Build it. Test the other branch by cheating. Change `boolean gameOver = false;` to `true`, and run.

> **Check.** The game opens straight onto the game over screen, with a score of 0.

Set it back to `false`.

6.5 Comparing numbers

A condition can compare numbers, too:

```
if (score >= 10) {
  background(255, 0, 0);   // only happens once score is 10 or more
}
```

You write	It asks
`score > 10`	greater than 10?
`score < 10`	less than 10?
`score >= 10`	10 or more?
`score <= 10`	10 or less?
`score == 10`	exactly 10?

Watch out. `==` is a question: *are these equal?* A single `=` is a command: *make this equal to that*. Write `if (score = 10)` by mistake, and the sketch stops with:

```
Type mismatch: cannot convert from int to boolean
```

An `if` needs a yes or a no, and `score = 10` is a command. Make it `==`.

Part Four - Make it yours

Your turn. Rewrite the game over screen so it sounds like your game: different words, different colors.

> **Watch out.** Keep the last line, `textAlign(LEFT, BASELINE);`. It puts text back to normal after the centered game over text, and without it every `text()` after it comes out centered too.

Your turn. Tune the hazard: its speed, its size, where it starts. A harder game, or an easier one?

Your turn. Optional: any of these.

- A second hazard, `hazard2`, with its own `if`. Give it a different speed or size.
- A second coin, `coin2`, worth more: `score = score + 5;`
- Change what ends the game. A hazard? Reaching a score of 20? Both?
- A background that changes as the score goes up. Stack a few `if`s right after the sky, one for each score, so the later ones paint over the earlier:

```
if (score >= 5) {
  background(255, 170, 80);    // sunset
}
if (score >= 10) {
  background(30, 30, 70);      // night
}
```

Your turn. Optional: give your player lives. Start with `int lives = 3;` at the top. When the hazard hits, take one away, and move the hazard somewhere else, exactly like the coin. Set `gameOver` to `true` only when `lives` reaches `0`. Show the lives on the screen, under the score.

Then play your game. This is where it becomes one, so leave time to play it.

Chapter summary

You started with a coin that did nothing when you touched it. You end with a game: a score that counts, something to avoid, and a screen that says it's over.

You built	It's an example of
`player.collidesWith(coin)`	a yes-or-no question, using the **hitbox** from Chapter 2
`if (...) { ... }`	doing something only when a **condition** is true
`int score = 0;` at the top	a variable every function can see
`text("Score: " + score, 20, 36)`	words and a number, glued together, on the screen
`score = 7`, then back	testing by cheating
the racing score, and `moveTo()`	`draw()` asks every frame, so the fix is to end the touch
`boolean gameOver`	a `true` or `false` that remembers which mode you're in
`playGame()`	**refactoring** into a function of your own
`if (!gameOver) { ... } else { ... }`	two modes, and `draw()` picking one

New syntax in this chapter

Syntax	Where you met it
`collidesWith()`	6.1
`if (condition) { }`	6.2
`int`	Step 3
`fill()`, `textSize()`, `text()` for words	Step 4
`+` between text and a number	Step 4
`score = score + 1`	Step 5
`boolean`, `true`, `false`	Step 8
`void playGame() { }`, and calling it	Step 9
`!`, `else`	Step 10
`textAlign()`	Step 10
`>`, `<`, `>=`, `<=`, `==`	6.5

Vocabulary

`boolean` -- a variable that's only ever `true` or `false`. (Step 8)

condition -- the yes-or-no question in the parentheses of an `if`. (6.2)

hitbox -- the invisible circle that decides whether something is touching. For your character, it's the radius in the `super(...)` line. (6.1)

`if` statement -- "if this is true, do this block of code." (6.2)

`int` -- a variable that holds a whole number. (Step 3)

mode -- which part of the game your program is in right now: playing, or game over. (Step 10)

refactoring -- moving code around so it's easier to read, while it does exactly the same thing. (Step 9)

Check your understanding

1. What does `collidesWith()` hand back? What decides whether two things are touching?
2. Why does `int score = 0;` go at the very top of the sketch?
3. Before you added `moveTo()`, why did the score race upward?
4. What's the difference between an `int` and a `boolean`? Give an example of each from your own game.
5. Why did touching the hazard in Step 8 seem to do nothing?
6. `=` and `==`. Which one asks a question?

What's next

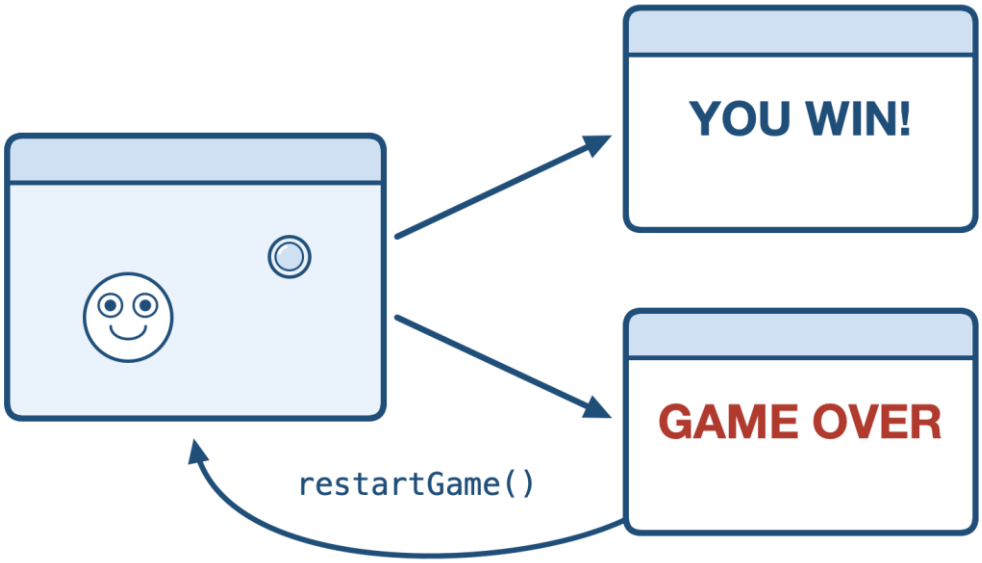
Your game can end. Once it's over, though, the only way to play again is to press Stop and Run.

In Chapter 7 you finish it. A way to win as well as lose, a click to play again, and three meetings to make it a game somebody else wants to play.

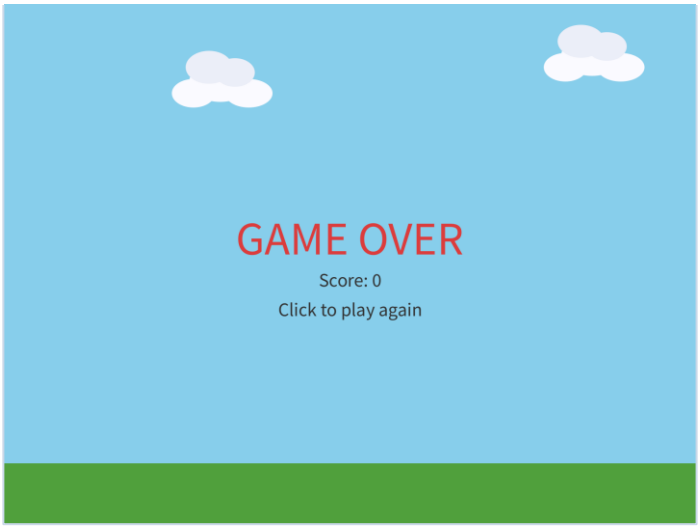


Chapter 7 - Build Your Game

Everything you have learned, in one game that is yours. This is the one you show people.



Restarting a game sounds hard. You just start it again.




```
void mousePressed() {
  if (gameOver || playerWon) {
    restartGame();
  }
}
```

Chapter 6 ended with a game that can end. Once it's over, though, the only way to play again is to press Stop and Run.

Today you finish it. A way to win as well as lose, a click to play again, and then the rest of the time to make it a game somebody else wants to play.

There's exactly **one** new idea in this chapter: restarting. Everything else you've already built.

This chapter takes three meetings, and does it in four moves.

Part	The idea
One	**You already know all of this.** Read the starter, and find every piece.
Two	**One full loop.** Win, lose, click, and play again, before you change anything.
Three	**Make it yours.** Your character, your world, your rules.
Four	**Play-test, and show it.** Somebody else plays your game.

Parts One and Two are the first meeting. Part Three is the second. Part Four is the third.

Part One - You already know all of this

7.1 One sentence first

Your turn. Before any code, write one sentence describing your game. Who's your character? What are they collecting, and what are they avoiding? Two minutes.

If you can say what your game is, you'll build a much better one.

7.2 The starter

You can start from your Chapter 6 sketch, or from the starter below, whichever is closer to what you want. Either way, keep your own `MyCharacter` and `Cloud` tabs.

Build it. Open your Chapter 6 sketch and save a copy first: **File → Save As**, with a number on the end of the name. Your Chapter 6 game stays safe in the old one. Then paste this into your main tab:

```
// Lesson 7 - Build Your Game.  Replace everything marked CHANGE with your own
choices!
import coxprogramming.processing.sprites.*;

MyCharacter player;
Cloud cloud1;
Cloud cloud2;
Block ground;

Blob coin1;      // things to collect
Blob coin2;
Blob coin3;
Blob hazard1;    // things to avoid
Blob hazard2;

int score = 0;
int goal = 15;               // CHANGE: the score you need to win
boolean gameOver = false;
boolean playerWon = false;

void setup() {
  size(800, 600);

  // Scenery never changes during play, so it is built once, here.
  ground = new Block(this, 0, height - 70, width, 70, color(80, 160, 60));  //
CHANGE: your ground
  cloud1 = new Cloud(this, 150, 90, 0.5);
  cloud2 = new Cloud(this, 520, 60, 0.8);

  restartGame();
}

void draw() {
  background(135, 206, 235);  // CHANGE: your sky / background color

  cloud1.move();
  cloud1.drawSprite();
  cloud2.move();
  cloud2.drawSprite();

  ground.drawSprite();
```

```
  if (!gameOver && !playerWon) {
    playGame();
  } else if (playerWon) {
    showWin();
  } else {
    showGameOver();
  }
}

void playGame() {
  // --- Collectibles ---
  coin1.move();
  coin1.drawSprite();
  coin2.move();
  coin2.drawSprite();
  coin3.move();
  coin3.drawSprite();

  // --- Hazards ---
  hazard1.move();
  hazard1.drawSprite();
  hazard2.move();
  hazard2.drawSprite();

  // --- Player ---
  player.followMouse();  // or keyboardControl() in keyPressed()
  player.drawSprite();
```

```
  // --- Collecting:  score a point, and the coin jumps somewhere new ---
  if (player.collidesWith(coin1)) {
    score = score + 1;
    coin1.moveTo(random(width), random(80, height - 150));
  }
  if (player.collidesWith(coin2)) {
    score = score + 1;
    coin2.moveTo(random(width), random(80, height - 150));
  }
  if (player.collidesWith(coin3)) {
    score = score + 1;
    coin3.moveTo(random(width), random(80, height - 150));
  }

  // --- Lose condition ---
  if (player.collidesWith(hazard1)) {
    gameOver = true;
  }
  if (player.collidesWith(hazard2)) {
    gameOver = true;
  }

  // --- Win condition ---
  if (score >= goal) {
    playerWon = true;
  }

  // --- Score display ---
  fill(50);
  textSize(22);
  text("Score: " + score + " / " + goal, 20, 32);
}
```

```
void showWin() {
  textAlign(CENTER, CENTER);
  fill(80, 200, 100);
  textSize(54);
  text("YOU WIN!", width/2, height/2 - 30);
  fill(50);
  textSize(22);
  text("Score: " + score, width/2, height/2 + 18);
  text("Click to play again", width/2, height/2 + 52);
  textAlign(LEFT, BASELINE);
}

void showGameOver() {
  textAlign(CENTER, CENTER);
  fill(220, 60, 60);
  textSize(54);
  text("GAME OVER", width/2, height/2 - 30);
  fill(50);
  textSize(22);
  text("Score: " + score, width/2, height/2 + 18);
  text("Click to play again", width/2, height/2 + 52);
  textAlign(LEFT, BASELINE);
}

void mousePressed() {
  if (gameOver || playerWon) {
    restartGame();
  }
}

void restartGame() {
  score = 0;
  gameOver = false;
  playerWon = false;

  // CHANGE: your player's starting position
  player = new MyCharacter(this, width/2, height/2);
  player.setVelocity(4, 0);
  // Collectibles - CHANGE: color, size, speed
  coin1 = new Blob(this, random(width), random(80, height - 150), 14, random(1,
2.5), random(-1.5, 1.5), color(255, 220, 50));
  coin2 = new Blob(this, random(width), random(80, height - 150), 14, random(1,
2.5), random(-1.5, 1.5), color(255, 220, 50));
  coin3 = new Blob(this, random(width), random(80, height - 150), 14, random(1,
2.5), random(-1.5, 1.5), color(255, 220, 50));

  // Hazards - CHANGE: color, size, speed.  They start in the top corners, away
from the player.
  hazard1 = new Blob(this, 80, 80, 22, 3, 2, color(220, 60, 60));
  hazard2 = new Blob(this, width - 80, 80, 22, -2, 3, color(220, 60, 60));
}
```

Look through it. Every piece of it is something you've written before:

Piece	What it does	You met it in
`setup()`	builds the scenery once, then calls `restartGame()`	Chapter 1
`draw()`	the sky, clouds, and ground, then picks a screen	Chapters 1 and 5
`MyCharacter`, `Cloud`	your character, and your scenery	Chapters 2 and 5
`setVelocity()`, `followMouse()`	a speed, and the mouse	Chapters 3 and 4
`playGame()`	three coins, two hazards, your player, and the score	Chapter 6
`showWin()`, `showGameOver()`	the two end screens	Chapter 6
`mousePressed()`	a click	Chapter 4
`restartGame()`	**new**	today

The coins and hazards are plain colored blobs. They're placeholders, there so the code runs. Replacing them with things that belong in your world is the assignment.

If you drive with the arrow keys, put your `keyPressed()` from Chapter 4 at the bottom, and change `player.followMouse();` in `playGame()` to `player.move();`.

7.3 Three modes

Chapter 6 had two modes: playing, and game over. This game has three, because winning and losing should look different. `playerWon` joins `gameOver`, and `draw()` picks one of three screens:

```
  if (!gameOver && !playerWon) {
    playGame();
  } else if (playerWon) {
    showWin();
  } else {
    showGameOver();
  }
```

> **Aside: `&&` and `else if`**
>
> `&&` means **and**. `!gameOver && !playerWon` reads *not lost, and not won*: still playing. `else if` adds another question to an `if`: if the first one was false, try this one. Processing goes down the list, and the first one that's true wins. The last `else` catches whatever's left.

You win when `score >= goal`. The `goal` is at the top, `15` to start. Change it to make the game longer or shorter.

7.4 Restarting

To restart a game, you might think you need to undo everything that happened. That's much harder than it sounds, and it's easy to miss something.

You just run the starting steps again. Make a new player, make new coins and hazards, set the score back to `0`, and set the modes back to `false`. That's `restartGame()`:

```
void restartGame() {
  score = 0;
  gameOver = false;
  playerWon = false;

  player = new MyCharacter(this, width/2, height/2);
  // ... brand-new coins, and brand-new hazards
}
```

Look at `setup()`. It builds the ground and the clouds, and then it calls `restartGame()`. Starting the game and restarting it are the same thing, so they're the same code. The ground and clouds are built once, in `setup()` alone, because nothing in the game ever changes them.

The restart happens on a click:

```
void mousePressed() {
  if (gameOver || playerWon) {
    restartGame();
  }
}
```

Aside: ||

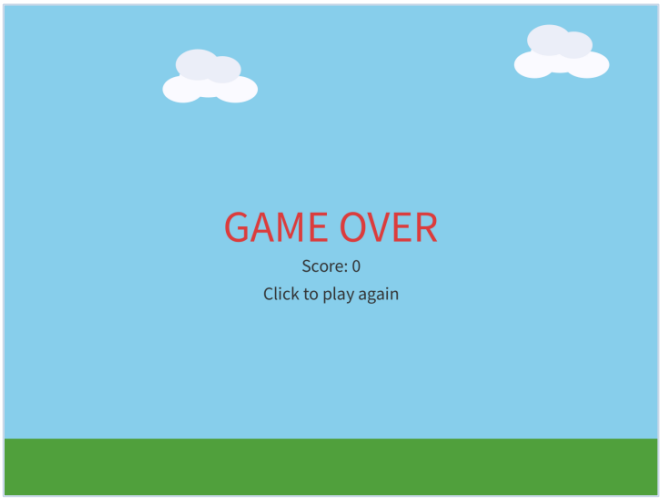
`||` means **or**. `gameOver || playerWon` is *we lost, or we won*. The `if` guards the restart, so a click while you're playing leaves your game alone.

Part Two - One full loop

Step 1 - Lose, and play again

Build it. Run the starter. Play it until you lose.

> **Check.** *GAME OVER*, your score, and *Click to play again*.



Lost.

Now click.

> **Check.** A fresh game: the score is `0`, your player is back in the middle, and the hazards are back in the top corners.



Ten frames after the click.

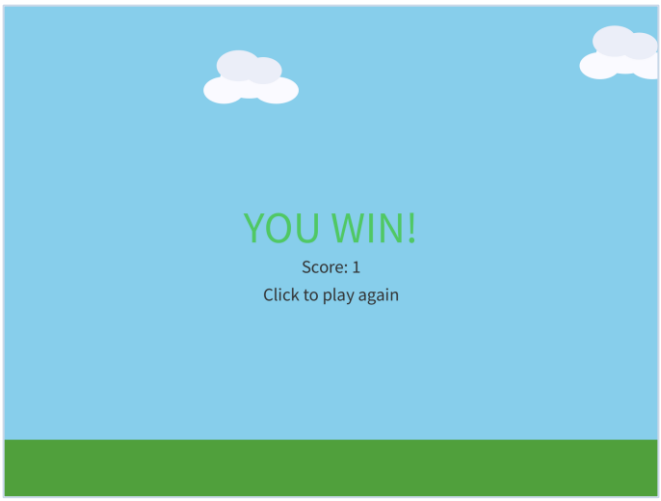
Make sure that works before you change anything.

Step 2 - Win it, by cheating

Winning takes 15 coins, which is a lot of playing just to check one screen. So test it by cheating, the way you did in Chapter 6.

Build it. Change `int goal = 15;` to `int goal = 1;`, and run. Touch one coin.

> **Check.** *YOU WIN!*



One coin, and a win.

Click to play again, and make sure the win restarts too. Then set the goal back.

> **Watch out.** `restartGame()` has to put back **every** variable that the game changes. Leave one out, and the restart misses it. Take out `gameOver = false;`, lose, and click: the game rebuilds everything else, and you're still on the game over screen. Check every variable at the top of your sketch against `restartGame()`, one at a time.

Step 3 - One full loop, in your world

Build it. Bring in your own world from Chapter 5: your ground, your background, your decorations. Then replace the first placeholder coin with something that belongs there.

Get one full loop working first: your player, your coins, and one way to win or lose. Play it, properly, before you add anything else. The games that go wrong in this chapter are almost always the ones with five new features and no loop that plays.

Part Three - Make it yours

Last meeting you got one full loop playing. Today it becomes yours.

Start with five minutes of play-testing. Let a neighbor play your game, and keep your hands off the keyboard. Watching somebody else play is the fastest way to find out what's broken.

Step 4 - Replace the placeholders

Your turn. Your character, your collectibles, your hazards, your world. Every color, every name, every shape the starter marks `CHANGE`. Space, ocean, forest, city, a dream. Whatever your sentence from 7.1 said.

Step 5 - Your screens, in your words

Your turn. Rewrite the win and lose screens. *"YOU WIN!"* is the starter talking. Make it sound like you, and like your game.

Keep the `textAlign(LEFT, BASELINE);` at the end of each one, as in Chapter 6.

Step 6 - Tune it

Your turn. Play it. Too easy? Faster hazards, or a higher `goal`. Too hard? Slower hazards, or a bigger hitbox in your `super(...)` line.

You built your game, so you're the best player it'll ever have. That means it's probably too hard for everybody else. Your neighbor is the one to tune it for.

7.5 Three coins, three `if`s

The starter has three coins and two hazards. Each one has its own variable and its own `if`. Yes, that's repetitive. It's also code you can read from top to bottom and change one piece at a time. That's a trade-off, and for a first game it's the right one.

Want a fourth coin? It goes in four places: declare `coin4`, create it in `restartGame()`, move and draw it in `playGame()`, and give it an `if`.

> **Watch out.** Copy the `coin3` block to make `coin4`, and every `coin3` in the copy has to become `coin4`. Miss the one in `moveTo()`, and touching `coin4` moves `coin3` away. `coin4` sits still under your player, scoring a point every frame, and you win the whole game in under a second. Read the copied lines out loud, name by name.

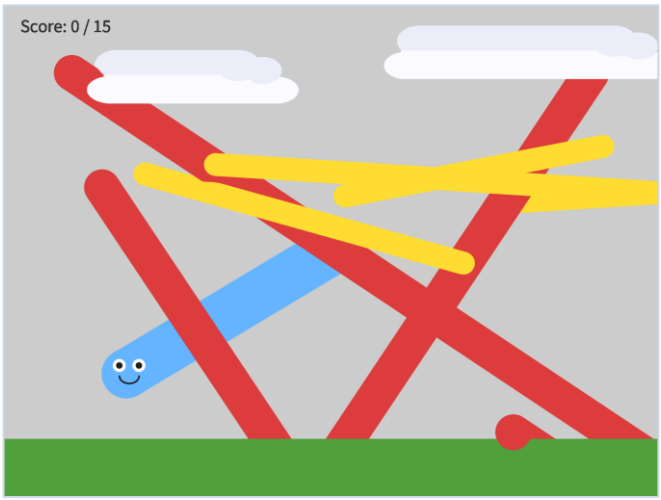
Want twenty coins? That's what Bonus D is for.

Step 7 - One thing that's your idea

Your turn. Add one thing that's your idea. Some places to start:

- **Lives**, from Chapter 6, in place of instant game over.
- **Hazards that speed up** as the score climbs.
- **A special ability** on a key: a speed burst, a shield, a teleport. It goes in `keyPressed()`.
- **A custom hazard**, in its own `extends Blob` tab: a shark, a storm cloud, a meteor.
- **A title screen** before the game starts. That's a fourth mode.
- **Bouncing.** One line in `setup()`, right before `restartGame()`:

```
  Sprite.screenBehavior = ScreenBehavior.BouncyWalls;   // everything bounces
off the edges
```



`BouncyWalls`, with `background()` switched off to show the paths. The hazards bounce off the bottom edge, behind the ground, and the coins bounce off the right.

It changes every sprite in your game at once. That's why the clouds bounce too.

> **Argue it.** Play your game with bouncing on, then with it off. It's the same code, and a different game. Which one is better? Write down which one you kept, and why.

- **A bonus chapter.** Pictures for your character (Bonus A), sounds (Bonus B), an animated character (Bonus C), or lots of coins (Bonus D). They're at the back of the book.

Your turn. Optional: a second level. When the player wins, rebuild the world harder, with faster hazards, a different background, and a higher goal, and keep the score. That's `restartGame()` with a level number, and it turns a demo into a game people want to keep playing.

Part Four - Play-test, and show it

Today somebody else plays your game.

7.6 Play-testing

Spend the first half of the meeting on what your play-testers found. Watch them play, with your hands off the keyboard. Whatever confuses them is a real problem. Go fix it, then watch somebody play it again.

That cycle, build, test, change, test again, is what **play-testing** is.

7.7 Show it

Play a neighbor's game. Tell them one thing you liked, and one thing you would add.

Then be ready to say two things about your own game: what it is, in one sentence, and one decision you made on purpose.

Chapter summary

You started with a game that could end, once. You end with one that can be won, lost, and played again, that's yours from the character to the rules, and that somebody else has played.

You built	It's an example of
`playerWon`, and three screens	three **modes**, and `draw()` picking one
`!gameOver && !playerWon`	`&&`, **and**
`else if`	a list of questions, and the first true one wins
`restartGame()`, called from `setup()`	**restarting** by running the starting steps again
`if (gameOver || playerWon)` in `mousePressed()`	`||`, **or**, guarding the restart
`goal = 1`	testing by cheating, again
three coins, three `if`s	repetitive and readable: a trade-off
your neighbor, playing	**play-testing**

New syntax in this chapter

Syntax	Where you met it
`&&`	7.3
`else if`	7.3
`restartGame()`	7.4
`||`	7.4
`Sprite.screenBehavior = ScreenBehavior.BouncyWalls;`	Step 7

Vocabulary

game loop -- `draw()` running over and over. It's the engine of every game you've ever played. (7.2)

play-testing -- playing your own game, or watching somebody else play it, to find out whether it's fun, and then changing it. (7.6)

restart -- putting every variable back to its starting value, so the game can be played again. (7.4)

win / lose condition -- the rules that decide when the game ends, and how. (7.3)

Check your understanding

1. What does `restartGame()` do, and why does `setup()` call it?
2. Why are the ground and clouds built in `setup()` and not in `restartGame()`?
3. What does `!gameOver && !playerWon` mean, in plain words?
4. Why does `mousePressed()` check `gameOver || playerWon` before it restarts?
5. You restart, and the score goes back to `0`, and the game stays on the game over screen. What's missing?
6. What's your game, in one sentence? What's one decision you made on purpose?

What's next

That's the course. Seven chapters ago there was an empty window, and now there's a game in it that somebody else has played.

If you want more, the bonus chapters are at the back: pictures, sounds, animation, and lists. Next quarter, the code leaves the screen, with Arduino and the Circuit Playground board.

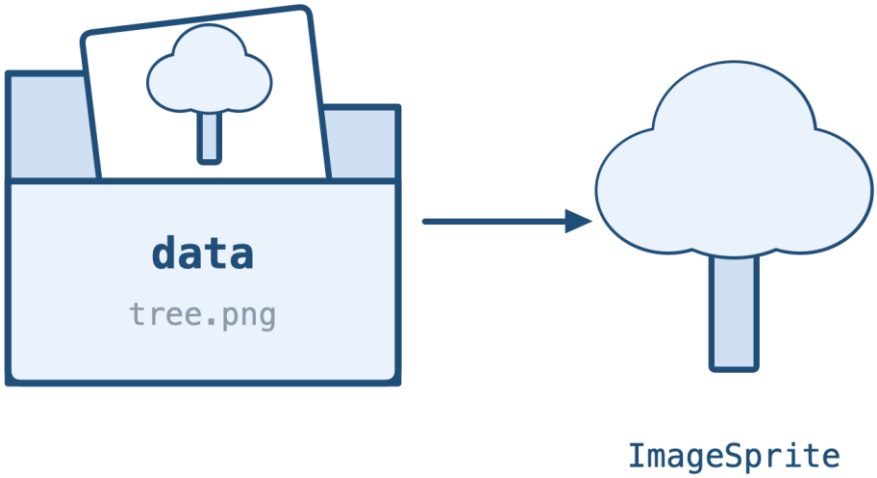
Bonus Chapters

Images, sounds, animation, and lists. Each one says when it's best taken.

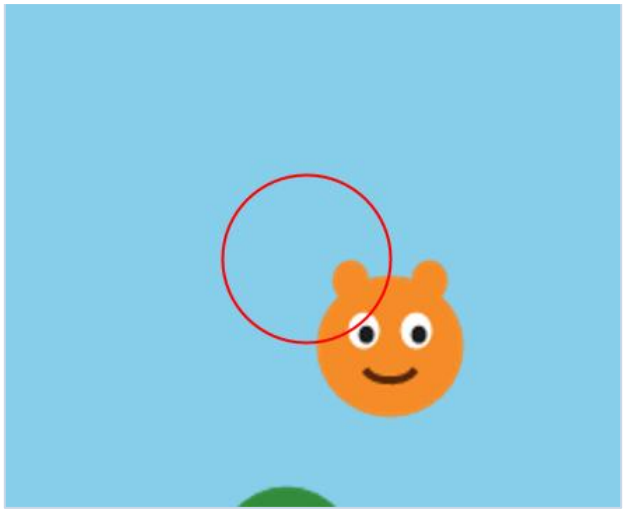


Bonus A - Images

Swap your shapes for actual pictures.



The picture is here. The character is over there. They used to be the same place.



```
    parent.image(img, pos.x, pos.y, 60, 60);
```

So far, everything on your screen is made of shapes you drew in code: circles, rectangles, arcs. This chapter puts real pictures in your game. A character you drew in a paint program, a tree you found online, a whole background.

You can take it any time after Chapter 4. It works well between Chapters 4 and 5, or during Chapter 7, when you're making your world look finished. It takes one meeting.

This chapter does it in three moves.

Part	The idea
One	**Pictures live in a folder.** The `data` folder, and how a sketch finds a file.
Two	**Scenery: `ImageSprite`.** A sprite that's a picture.
Three	**Your character as a picture.** Keep `extends Blob`, and draw a picture in `drawSprite()`.

Part One - Pictures live in a folder

A.1 Your picture

Your turn. Find or draw a small picture for your character. PNG is the kind of file you want. A PNG can have a **transparent** background, so the world shows through around your character. You can draw a 32-by-32 character in any paint program, or find free art at opengameart.org.

A.2 The `data` folder

Processing looks for pictures in exactly one place: a folder called `data`, inside your sketch's folder.

Build it.

1. Open your sketch, and choose **Sketch → Show Sketch Folder**.
2. Make a new folder in there called `data`. Exactly that, all lowercase.
3. Drag your picture into `data`.
4. Write down its name exactly, capitals and the ending (`.png`) included.

Part Two - Scenery: ImageSprite

A.3 A sprite that's a picture

An **ImageSprite** is a sprite from the library that shows a picture. It has the sprite behaviors you know: `move()`, `setVelocity()`, `followMouse()`, `drawSprite()`. It takes the sketch, where its **center** goes, the file's name, and a size:

```
new ImageSprite(this, x, y, "tree.png")            // 100 x 100 pixels
new ImageSprite(this, x, y, "tree.png", 80)        // 80 x 80 square
new ImageSprite(this, x, y, "tree.png", 120, 80)   // 120 wide, 80 tall
```

It has no `collidesWith()`. An `ImageSprite` is a sprite, and it isn't a Blob, so it can't collide. Save it for scenery: the things in the world your player just looks at.

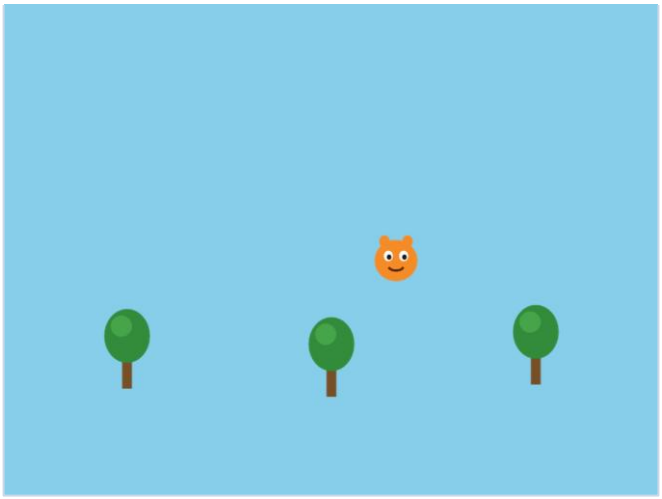
Step 1 - A picture in the world

Build it. Start with the easy one. Add an `ImageSprite` to your sketch, in the same three places as every object since Chapter 1. Here are three trees, from the Canvas page's sample sketch:

```
ImageSprite tree1;
ImageSprite tree2;
ImageSprite tree3;
  // in setup().  Replace "tree.png" with your own file name.
  tree1 = new ImageSprite(this, 150, 420, "tree.png", 60, 100);
  tree2 = new ImageSprite(this, 400, 430, "tree.png", 60, 100);
  tree3 = new ImageSprite(this, 650, 415, "tree.png", 60, 100);
  // in draw(), before your player:  decorations go behind
  tree1.drawSprite();
  tree2.drawSprite();
  tree3.drawSprite();
```

Run it.

> **Check.** Your pictures, in your world.



Three trees, and a picture character from Part Three. The art is stand-in art. Yours is yours.

Change the size numbers until your picture looks the right shape. A tall tree squashed into a square looks wrong.

Step 2 - When the picture doesn't show

Three things go wrong here, and each one looks different.

Watch out. The name doesn't match. Ask for `"pine.png"` when the file is `tree.png`, and Processing prints this under your code, then stops:

```
The file "pine.png" is missing or inaccessible, make sure the URL is valid
or that the file has been added to your sketch and is readable.
```

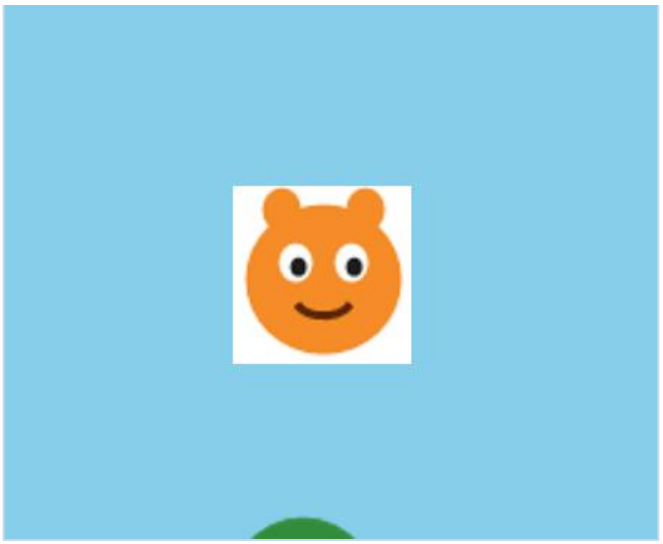
Check that the file is in `data`, and check the name, letter by letter.

Watch out. The capitals don't match. Ask for `"Tree.PNG"` when the file is `tree.png`, and Processing stops and tells you exactly what's wrong:

```
This file is named tree.png not Tree.PNG. Rename the file or change your
code.
```

Capitals count here too, the same as in your code.

Watch out. A white box around your picture. The PNG was saved without a transparent background. The picture really is a rectangle, and its background just happens to be white. Your code is fine. Find or make a PNG with a transparent background. Free tools like remove.bg can take a background out for you.



The same character, saved on a white background.

Part Three - Your character as a picture

A.4 Keep the Blob

Your player needs to collide with things, and an `ImageSprite` can't. So for your character, keep `extends Blob`, exactly as you wrote it in Chapter 2, and change `drawSprite()` to draw a picture in place of shapes.

Step 3 - `MyImageCharacter`

Build it. Make a new tab called `MyImageCharacter`, and paste this in:

```
// MyImageCharacter - like MyCharacter, but draws a picture instead of shapes.
// extends Blob means it still has collidesWith() for collision detection.
class MyImageCharacter extends Blob {

  PImage img;

  MyImageCharacter(PApplet p, float x, float y, String fileName) {
    super(p, x, y, 30, color(0));  // radius 30 - the invisible collision circle
    img = p.loadImage(fileName);
  }

  void drawSprite() {
    parent.pushStyle();
    parent.imageMode(CENTER);
    parent.image(img, pos.x, pos.y, 60, 60);  // drawn centered on the sprite's
position
    parent.popStyle();
  }
}
```

Read it against your `MyCharacter` tab. The class line, the handshake, and a `drawSprite()`. Three things are new.

> **Aside: `PImage` and `loadImage()`**
>
> A **`PImage`** is Processing's kind of variable for a picture. `PImage img;` makes a box for one. `loadImage(fileName)` reads the file out of your `data` folder and puts it in the box. The constructor does that once, when the character is made. `String` is the kind of thing a piece of text is, like `"hero.png"`.

Aside: `imageMode(CENTER)`, and `pushStyle()`

`image(img, pos.x, pos.y, 60, 60)` draws the picture, 60 by 60. Normally a picture is placed by its top-left corner, like a `Block`. `imageMode(CENTER)` places it by its middle, so it sits on the belly button like every other part of your character. `pushStyle()` and `popStyle()` save and put back Processing's settings around it, so `imageMode` only changes your character's drawing.

Build it. In your main tab, make your player a `MyImageCharacter` in place of a `MyCharacter`, in the declaration at the top and in `setup()`:

```
  player = new MyImageCharacter(this, width/2, height/2, "hero.png");
```

Use your own file's name. Run it, and move your character around.

Check. Your picture, moving and colliding the way your shape character did.

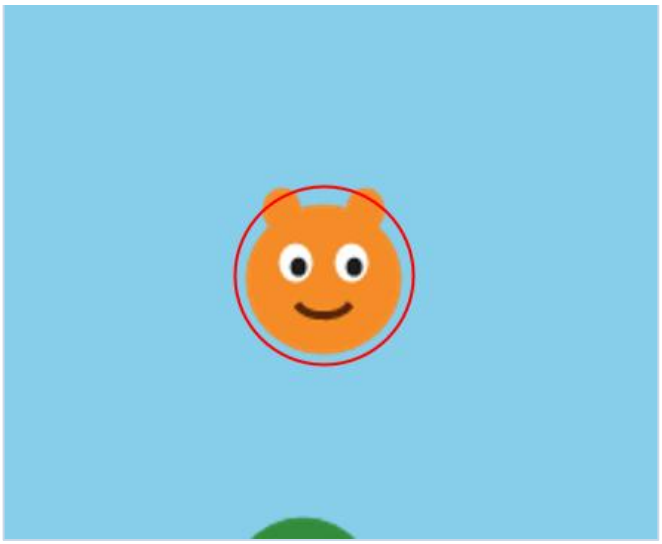
Step 4 - See your hitbox

The radius in `super(...)` is **still** your hitbox. The picture doesn't change that. So it helps to see where the hitbox is while you tune it.

Build it. At the end of `drawSprite()`, after `popStyle()`, draw the hitbox as a ring:

```
    parent.noFill();
    parent.stroke(255, 0, 0);
    parent.ellipse(pos.x, pos.y, 2 * radius, 2 * radius);   // the hitbox
```

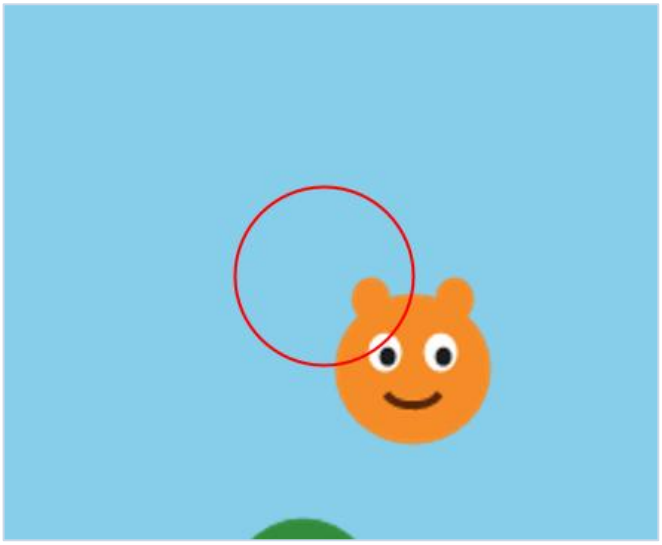
Check. A red ring around your picture. That's exactly the circle `collidesWith()` uses.



The hitbox, drawn as a ring.

Tune the `30` in `super(...)` until the ring fits. A ring slightly smaller than the picture usually feels fairest. Then take the three ring lines out, before anybody else plays.

> **Watch out.** Leave out `imageMode(CENTER)`, and the picture is placed by its corner. It sits down and to the right of your hitbox, and collisions feel broken, because they're happening where your character isn't drawn.



No `imageMode(CENTER)`. The hitbox is where the character is, and the picture is somewhere else.

Watch out. `player.collidesWith(tree1)` stops the sketch with:

```
The method collidesWith(Blob) in the type Blob is not applicable for the
arguments (ImageSprite)
```

That's Processing's way of saying a tree is an `ImageSprite`, and `collidesWith()` only works with Blobs. If your player has to touch it, make it a Blob with a picture inside, the way `MyImageCharacter` is.

The rule, side by side:

It needs to	Use
collide (your player, coins, hazards)	`extends Blob`, with a `PImage` inside
just **look nice** (trees, clouds, a background)	`ImageSprite`

A.5 Make it yours

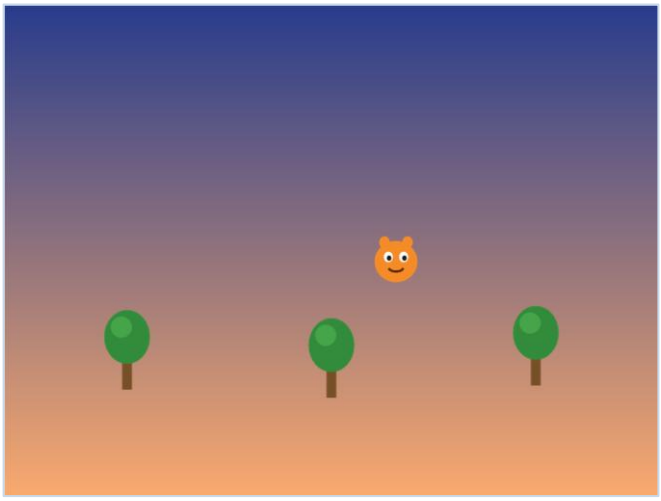
Your turn. Your own character picture. A PNG with a transparent background looks best by a mile.

Your turn. Fix the size, the `60, 60` in `image()`, so it matches your art's real shape.

Your turn. Optional: any of these.

- A **background picture**, as big as the window, drawn first in `draw()`, right after `background()`:

```
  bg = new ImageSprite(this, width/2, height/2, "background.png", width,
height);
```



A background picture, behind the trees.

- A decoration that drifts: give an `ImageSprite` a velocity, and `move()` it.
- A tilted one: `tree3.setRotation(PI/6);` tips it a little. `PI` is half a turn, so `PI/6` is a twelfth of one.
- Coins and hazards as pictures. They have to collide, so they need `extends Blob` with a picture inside, the same as `MyImageCharacter`.
- Fake an animation. Load **two** `PImage`s and swap between them every few frames. That's the whole idea behind Bonus C.

> **Watch out.** When you hand in your sketch, zip the whole folder, `data` and all. Without the `data` folder, nothing runs.

Chapter summary

You built	It's an example of
a `data` folder	where Processing looks for files
`new ImageSprite(this, x, y, "tree.png", 60, 100)`	a sprite that's a picture, for scenery
`MyImageCharacter extends Blob`	a picture inside a Blob, so it still collides
`imageMode(CENTER)`	placing a picture by its middle
the red ring	seeing your hitbox while you tune it

New syntax in this chapter

Syntax	Where you met it
`ImageSprite`	A.3, Step 1
`PImage`, `loadImage()`, `String`	Step 3
`image()`, `imageMode(CENTER)`	Step 3
`pushStyle()`, `popStyle()`	Step 3
`radius`	Step 4
`setRotation()`	A.5

Vocabulary

`data` folder -- a folder inside your sketch's folder, where Processing looks for pictures and sounds. (A.2)

`ImageSprite` -- a sprite from the library that shows a picture. It can't collide. (A.3)

`PImage` -- Processing's kind of variable for a picture. (Step 3)

transparent -- a part of a picture with no color at all, so whatever's behind it shows through. (A.1)

Check your understanding

1. Where does Processing look for your picture?
2. Why can't you use `collidesWith()` on an `ImageSprite`? What do you do when a picture has to collide?
3. Your character has a white box around it. What's wrong, and is it your code?
4. What does `imageMode(CENTER)` change?
5. Why does a picture character still need `extends Blob`?

Bonus B - Sounds

Add the click, the ding, and the music. It changes everything.



```
collectSound.play();
```

This is the loudest page in the book.

```
    collectSound.play();   // one-shot sound effect on collection
```

Your game is silent. This chapter gives it a ding when you collect a coin, a sound when the game ends, and music underneath the whole thing.

You can take it any time after Chapter 6, once your game has things to collect and a way to end. It pairs well with Bonus A, or works on its own. It takes one meeting.

This chapter does it in three moves.

Part	The idea
One	**A second library.** Install the Sound library, and put your sounds in `data`.
Two	**Declare, load, play.** The same three places as everything else.
Three	**Where the sound goes.** One line of code, and the real thinking is where to put it.

Part One - A second library

B.1 Installing the Sound library

SpriteGame is a library somebody else built, and you've used it since Chapter 1. Sound comes in a second one, from the people who make Processing. You install it once.

Build it. Choose **Sketch → Import Library → Add Library**. Search for "Sound", and pick the one by **The Processing Foundation**. Install it. Then **quit Processing and start it again**.

This is the step most likely to eat your time, so do it first.

Build it. At the top of your sketch, under the sprites import, add the second import:

```
import coxprogramming.processing.sprites.*;
import processing.sound.*;
```

B.2 Your sounds

Your turn. Find two sound files: one short effect, like a ding, a pop, a coin, or a beep, and, if you want one, a music loop about thirty seconds long. Free sounds are at opengameart.org, freesound.org, and zapsplat.com. Start with opengameart.org, which lets you download without an account. freesound.org needs a free account first. If a site won't let you log in, don't fight it. Switch sites, or ask your teacher.

Build it. Sketch → Show Sketch Folder, and drag both into the `data` folder, exactly as in Bonus A. Do it as soon as each one downloads. A sound that's only saved on a website can vanish, and then it's gone from your game too. MP3, WAV, OGG, and AIFF all work. **WAV is the most reliable**, especially for short effects.

Part Two - Declare, load, play

Step 1 - A sound for a coin

A sound goes in the same three places as everything else you've made. Declare it at the top, make it in `setup()`, and use it when you need it.

Build it. At the top, with your other variables:

```
SoundFile collectSound;
```

In `setup()`, load it, with **your** file's name:

```
  collectSound = new SoundFile(this, "collect.wav");
```

In `playGame()`, find the coin's `if`, and add one line after the score:

```
  if (player.collidesWith(coin)) {
    score = score + 1;
    coin.moveTo(random(width), random(height));
    collectSound.play();   // one-shot sound effect on collection
  }
```

Aside: `SoundFile`, and `play()`

A **`SoundFile`** is a variable for a sound, the way a `PImage` is for a picture. `new SoundFile(this, "collect.wav")` reads the file out of `data`. `collectSound.play()` plays it once, from the beginning. Call `play()` again while it's still going, and it starts over.

Run it, and collect a coin.

Check. A ding, every time you touch the coin.

Watch out. Get the file's name wrong, `"colect.wav"`, and Processing prints this under your code as the sketch starts:

```
Sound library error: unable to find file colect.wav
```

The game still runs, right up until the first coin. Then the sketch stops with `NullPointerException`, on the `play()` line. So when a `play()` line stops your sketch, check the folder and the file's name first, every time.

Step 2 - A sound for the end

Build it. Declare and load a second sound the same way, `gameOverSound`. Play it right where you set `gameOver = true;`:

```
  if (player.collidesWith(hazard)) {
    gameOver = true;
    gameOverSound.play();  // play the end sound
  }
```

Check. Touch the hazard, and the game ends with a sound.

Step 3 - Music

Build it. Declare and load a third sound, `bgMusic`, with your music file. Then, at the end of `setup()`, start it:

```
  bgMusic.amp(0.4);   // lower music volume so effects can be heard clearly
  bgMusic.loop();     // start background music - runs until stopped
```

And stop it when the game ends, in the hazard's `if`, before the end sound:

```
    bgMusic.stop();        // stop the music
```

Aside: `loop()`, `stop()`, and `amp()`

`loop()` plays a sound over and over until you call `stop()`. That's what music wants. `amp()` sets how loud: `0.0` is silent, and `1.0` is full volume. Music at full volume drowns out your effects, so turn it down.

Check. Music from the moment the game starts. It stops when the game ends, and the end sound plays.

Play it, and change the `0.4` until the balance feels right.

Part Three - Where the sound goes

B.3 The real skill

The code for playing a sound is one line. Deciding **where** that line goes is the actual thinking. A sound belongs at the exact moment its event happens.

The event	Where `play()` goes
collected something	inside its `if (player.collidesWith(...))`, next to the score
the game ended	right where you set `gameOver = true;`
music, all the time	once, in `setup()`, with `loop()`

That's Chapter 4's idea again. A sound effect is an event: it happens once, when something happens. Music is more like continuous input: it's going the whole time.

> **Watch out.** Put `collectSound.play();` in `draw()`, outside any `if`, and it starts over sixty times a second. It sounds like a machine gun.

> **Watch out.** Use `loop()` for a coin's ding, and it dings forever. `loop()` is for music.

B.4 Make it yours

Your turn. Choose sounds that fit your world. A cartoon boing in a space game is a choice. Make sure it's one you meant.

Your turn. Optional: any of these.

- A sound for a click, or for a key. It goes in `mousePressed()` or `keyPressed()`.
- A sound at a score milestone: `if (score == 10) { levelUpSound.play(); }`, inside the coin's `if`, right after the point.
- Fix the machine gun. When collisions come fast, `play()` restarts over and over, and the sound stutters. Guard it, so it only starts when it isn't already playing:

```
if (!collectSound.isPlaying()) collectSound.play();
```

Then decide which version you like. Sometimes the stutter is the right choice.

> **Watch out.** When you hand in your sketch, zip the whole folder, `data` and all, or nobody hears a thing. Short music loops keep the folder small. A four-minute song makes it huge.

Chapter summary

You built	It's an example of
`import processing.sound.*;`	a second library
`SoundFile collectSound;` and `new SoundFile(this, "collect.wav")`	declare, load
`collectSound.play()` in the coin's `if`	a sound at the moment of its event
`bgMusic.loop()` in `setup()`, and `bgMusic.stop()`	music, all the time, until the end
`bgMusic.amp(0.4)`	balancing the volume

New syntax in this chapter

Syntax	Where you met it
`import processing.sound.*;`	B.1
`SoundFile`, `new SoundFile(this, "...")`	Step 1
`play()`	Step 1
`loop()`, `stop()`, `amp()`	Step 3
`isPlaying()`	B.4

Vocabulary

`loop()` -- plays a sound over and over, restarting when it reaches the end, until `stop()`. (Step 3)

`play()` -- plays a sound once, from the beginning. (Step 1)

`SoundFile` -- a variable for a sound. (Step 1)

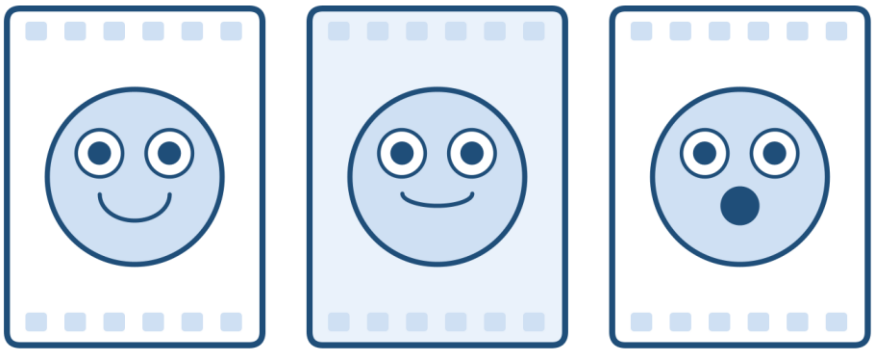
`stop()` -- stops a sound. (Step 3)

Check your understanding

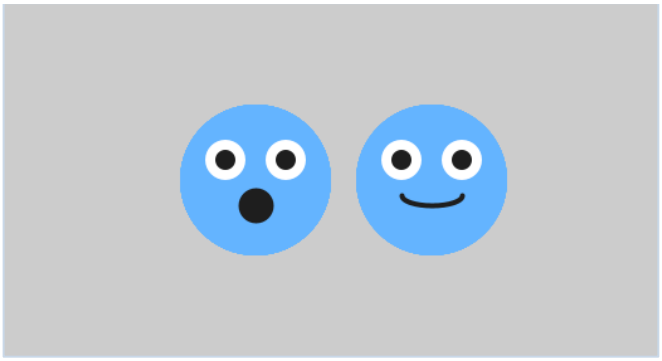
1. What two things does a sketch need before it can play a sound?
2. Where does the coin's sound go, and why there?
3. Why does music use `loop()`, and a coin use `play()`?
4. Your game runs, and stops with `NullPointerException` the first time you collect a coin. What's wrong?
5. What does `amp(0.4)` do?

Bonus C - Animate Your Character

Stop sliding a frozen drawing around. Make it move.



Same character. Same moment. Two places.



```
        .AddFrameSprite(new MyCharacterFrame(this, width/2 + 70, height/2, 1))
```

Your character moves around the screen, and it's the same drawing the whole time. This chapter makes it come alive: a mouth that talks, legs that walk, wings that flap.

You can take it any time after Chapter 2, once you have a character to animate. It shines most during Chapter 7, when you want your character to feel alive. It pairs well with Bonus A, because picture frames animate the same way. It takes one meeting.

This chapter does it in three moves.

Part	The idea
One	**A flipbook.** An animation is a few still pictures, shown fast, one after another.
Two	**One class, several poses.** Your character, drawn a few slightly different ways.
Three	**Build the flipbook.** An `AnimatedSprite` has the poses, and flips through them for you.

Part One - A flipbook

C.1 Pictures, shown fast

Have you ever drawn a flipbook in the corner of a notebook? A cartoon walks the same way: a few still drawings, each a little different, shown fast enough that your eyes see motion.

You met the idea in Chapter 1. `draw()` is already a flipbook: it draws a new page about 60 times a second. Today you add a second flipbook, inside your character, with a few **poses** to flip between.

That means the word *frame* has two jobs today.

- A **game frame** is one run of `draw()`, about sixty a second.
- An **animation frame** is one pose in your character's flipbook.

Part Two - One class, several poses

Step 1 - A frame class

Build it. Make a new tab called `MyCharacterFrame`. Copy your **whole character drawing** from your `MyCharacter` tab into it. Then rename the class, and the constructor, to `MyCharacterFrame`.

Build it. Add a `pose` number, the way the Canvas page's version does:

```
// MyCharacterFrame - ONE POSE of your character.
// It's just like your MyCharacter from Lesson 2, but the "pose" number lets you
// draw a few slightly different versions. AnimatedSprite flips between them.
class MyCharacterFrame extends Blob {

  int pose;  // which version to draw: 0, 1, 2, ...

  MyCharacterFrame(PApplet p, float x, float y, int pose) {
    super(p, x, y, 30, color(100, 180, 255));  // same handshake line as
MyCharacter
    this.pose = pose;
  }

  void drawSprite() {
    // --- Body (same in every pose) ---
    fill(100, 180, 255);
    noStroke();
    ellipse(pos.x, pos.y, 60, 60);

    // --- Eyes (same in every pose) ---
    fill(255);
    ellipse(pos.x - 12, pos.y - 8, 16, 16);
    ellipse(pos.x + 12, pos.y - 8, 16, 16);
    fill(30);
    ellipse(pos.x - 12, pos.y - 8, 8, 8);
    ellipse(pos.x + 12, pos.y - 8, 8, 8);

    // --- Mouth: CHANGES with the pose - this is what makes it animate! ---
    stroke(30);
    strokeWeight(2);
    noFill();
    if (pose == 0) {
      arc(pos.x, pos.y + 6, 24, 18, 0, PI);   // open smile
    } else if (pose == 1) {
      arc(pos.x, pos.y + 6, 24, 8, 0, PI);    // smaller smile
    } else {
      fill(30);
      ellipse(pos.x, pos.y + 10, 12, 12);     // round "talking" mouth
    }
  }
}
```

> **Aside: a property of your own, and `this.pose`**
>
> `int pose;` inside the class gives every frame a number of its own: which pose it draws. It's a property you added, the same way Blob gave your character `pos`. The constructor takes a `pose` too, and `this.pose = pose;` copies it in. Inside a class, `this` means the object itself, so `this.pose` is the frame's own pose, and plain `pose` is the number that was handed in.

Step 2 - Change one thing

Build it. Pick **one** thing on your character to animate. The mouth is easiest. Legs, arms, wings, or ears work well too. Wrap just that part in an `if (pose == 0) { ... } else if (pose == 1) { ... }`, the way the mouth is wrapped above.

> **Aside: `if`, if you're here before Chapter 6**
>
> Chapter 6 is where `if` gets explained properly. For now: `if (pose == 0) { ... }` runs the code in its braces only when `pose` is exactly `0`. `==` asks *exactly equal?* `else if (pose == 1) { ... }` is the next question, asked only when the first answer was no, and a last `else { ... }` catches whatever's left.

Make the difference small. Keep the body and the eyes **exactly** the same in every pose. Only change what should move, or it looks like flickering.

Part Three - Build the flipbook

Step 3 - An `AnimatedSprite`

Build it. In your main tab, your player becomes an **`AnimatedSprite`**, a sprite from the library that has several frames and shows one at a time. Here's the Canvas page's main tab:

```
// Bonus Lesson C - Animate Your Character
import coxprogramming.processing.sprites.*;

AnimatedSprite player;

void setup() {
  size(800, 600);

  // Build the flipbook from several poses - all at the SAME spot so it doesn't
jump.
  // AnimatedSprite shows one pose at a time and flips between them for you.
  player = AnimatedSprite.FromSprite(new MyCharacterFrame(this, width/2,
height/2, 0))
      .AddFrameSprite(new MyCharacterFrame(this, width/2, height/2, 1))
      .AddFrameSprite(new MyCharacterFrame(this, width/2, height/2, 2))
      .setFrameRate(8);   // hold each pose for 8 game frames; smaller = faster

  player.setVelocity(4, 0);  // gives followMouse() a speed to move at
}

void draw() {
  background(240);

  // --- Option A: follow the mouse ---
  player.followMouse();
  player.drawSprite();   // draws the current pose, then turns the page for you

  // --- Option B: arrow-key control ---
  // (comment out Option A above, uncomment these two lines, keep keyPressed
below)
  // player.move();
  // player.drawSprite();
}

// Used with Option B (arrow keys)
void keyPressed() {
  player.keyboardControl();
}
```

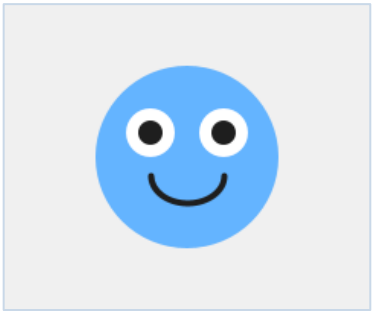
Aside: a chain of calls

`AnimatedSprite.FromSprite(...)` starts the flipbook with its first page. `.AddFrameSprite(...)` adds another page. `.setFrameRate(8)` sets the speed. Each one hands back the flipbook, so the next one can go right on the end. That's why it's one long line, split over four.

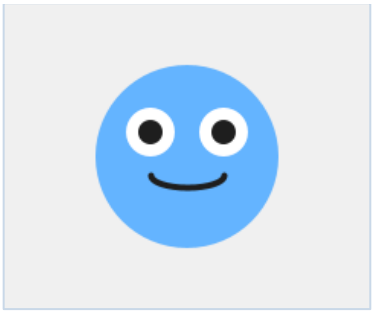
Every frame is made at the same `x` and `y`. That's the crucial part.

Run it.

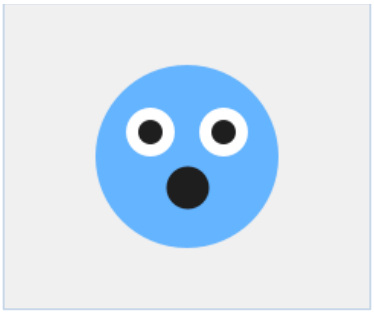
Check. Your character flips through its poses on its own, over and over.



Pose 0, game frames 1 to 8.



Pose 1, game frames 9 to 16.



Pose 2, game frames 17 to 24. Then back to pose 0.

`drawSprite()` draws the pose it's on, and keeps count. Every 8 game frames it turns the page.

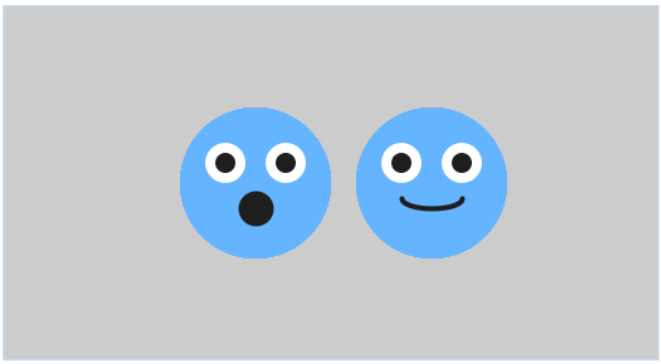
Step 4 - The speed

Build it. Try `setFrameRate(4)`, and then `setFrameRate(20)`. Which speed suits your character?

> **Check.** `4` flips fast and frantic. `20` flips slow and calm.

The number is how many game frames each pose stays on the screen. So a **smaller** number is **faster**. That's backwards from what most people expect.

> **Watch out.** Make one frame at a different `x` or `y`, and your character jumps every time the page turns. Here the second pose was made 70 pixels to the right.



With `background()` switched off: the same character, in two places.

> **Watch out.** Forget `this.pose = pose;`, and every frame's `pose` is `0`. Every page draws pose 0, so nothing seems to animate.

> **Watch out.** Copy the class and forget to rename its constructor, and the sketch stops with a message like this one, on the line that makes the frames:
>
> ```
> The constructor MyCharacterFrame(...) is undefined
> ```
>
> The constructor's name has to match the class's name exactly.

Step 5 - Move it

Build it. Your animated character moves exactly like before: `followMouse()` in `draw()`, or `keyboardControl()` in `keyPressed()` with `move()` in `draw()`. The starter has both options, the same way Chapter 4 did.

> **Check.** It moves and animates at the same time, with no extra work.

All the poses move together, because the `AnimatedSprite` carries every one of its frames along with it.

C.5 In your game

Here's the catch. An `AnimatedSprite` isn't a Blob, so it has no `collidesWith()`:

```
The function collidesWith(AnimatedSprite) does not exist.
```

For touching, use **`distanceTo()`**, which every sprite has. It hands back how far away the other sprite is, center to center. So in your Chapter 7 game, each `collidesWith()` becomes a distance check:

```
if (player.distanceTo(coin1) < 40) {   // 40 = how close counts as "touching"
  score = score + 1;
  coin1.moveTo(random(width), random(80, height - 150));
}
```

Pick a number that matches your character's size: about your character's radius plus the coin's.

Two more changes put it in your game. At the top, the player is an `AnimatedSprite`. And in `restartGame()`, the player is built from frames, the way Step 3 builds it, in place of `new MyCharacter(...)`.

> **Check.** Your animated character collects coins and runs into hazards, the same as before.

C.6 Make it yours

Your turn. Add a fourth pose. Try animating **legs or arms** in place of the mouth, and make it walk.

Your turn. Optional: any of these.

- More frames, for smoother motion. More pages make a smoother flipbook.
- A second animated creature, a pet, with its own `setVelocity()`, drifting across the scene.
- Poses by hand. Turn one page on a key, in `keyPressed()`:

```
  if (key == 'n') {
    player.nextFrame();          // turn one page
  }
```

- Or jump straight to a pose, for a jump or an attack: `player.setFrameNumber(2);`
- Picture frames. A frame can be an `ImageSprite` from Bonus A, so a set of drawings animates the same way: `AnimatedSprite.FromSprite(new ImageSprite(this, x, y, "walk0.png"))`, and `.AddFrameSprite(...)` for the rest.

Chapter summary

You built	It's an example of
`MyCharacterFrame`, with `int pose;`	one class, several poses, and a property of your own
`if (pose == 0) ... else if (pose == 1) ...`	changing one thing between poses
`AnimatedSprite.FromSprite(...).AddFrameSprite(...)`	a flipbook, built in a chain
`setFrameRate(8)`	game frames per pose: smaller is faster
`player.distanceTo(coin1) < 40`	touching, for a sprite that isn't a Blob

New syntax in this chapter

Syntax	Where you met it
a property of your own, `int pose;`	Step 1
`this.pose = pose;`	Step 1
`AnimatedSprite`, `FromSprite()`, `AddFrameSprite()`	Step 3
a chain of calls	Step 3
`setFrameRate()`	Step 4
`distanceTo()`	C.5
`nextFrame()`, `setFrameNumber()`, `key`	C.6

Vocabulary

animation frame -- one pose in a character's flipbook. (C.1)

`AnimatedSprite` -- a sprite from the library that has several frames and shows one at a time. (Step 3)

`distanceTo()` -- how far away another sprite is, center to center. (C.5)

game frame -- one run of `draw()`, about sixty a second. (C.1)

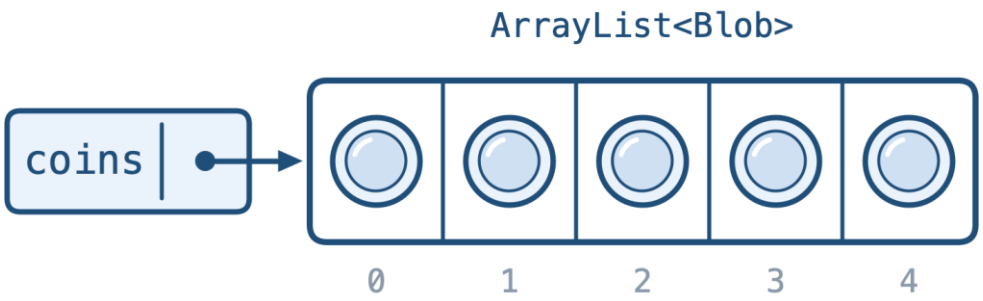
pose -- one version of your character's drawing. (C.1)

Check your understanding

1. What's the difference between a game frame and an animation frame?
2. What does `setFrameRate(4)` do, compared to `setFrameRate(20)`?
3. Your character jumps every time the pose changes. What do you check?
4. Why do the body and eyes have to be exactly the same in every pose?
5. Why can't an `AnimatedSprite` use `collidesWith()`? What does it use?

Bonus D - Lists and Loops

One coin is a variable. Twenty coins is a list.



Three coins, all in one spot. Walk the list forward, and you collect two of them.

```
for (int i = 0; i < coins.size(); i++) {
```

```
after frame 1:  score = 2, coins left = 1
```

Look at your game from Chapter 6 or 7. Each coin is a variable, a `new Blob(...)` line, a `move()`, a `drawSprite()`, and an `if`. Adding one more coin takes five pieces in four places. Twenty coins is about a hundred lines.

A hundred coins is about five hundred. `coin1`, `coin2`, `coin3`, typed one at a time, all the way up to `coin100`, until your fingers file a complaint. And somewhere in those five hundred lines is a `coin73` where you meant `coin74`. Good luck finding it.

This chapter fixes that with a **list**: one variable that holds as many coins as you give it, and loops that do the same thing to every one of them.

It's for the student who wants lots of coins. Take it during Chapter 7, or any time after Chapter 6. It takes one meeting. If you're working on your Chapter 7 game, save a copy first, and do this chapter on the copy.

This chapter does it in three moves.

Part	The idea
One	**A bag.** An `ArrayList`, one variable for many things.
Two	**Two loops that both say `for`.** One counts, to make things. One visits, to move and draw them.
Three	**Taking things out.** Removing a coin you collected, safely.

Part One - A bag

D.1 ArrayList

Think of a list as a **bag**. You keep putting things in, and it holds as many as you give it. There are three things you do with a list:

```
ArrayList<Blob> coins;                  // 1. declare it (what kind of bag)
coins = new ArrayList<Blob>();          // 2. create it (get an empty bag)
coins.add(new Blob(...));               // 3. add to it (put something in)
```

> **Aside:** `ArrayList<Blob>`
>
> An `ArrayList` is a list that grows as you add to it. The `<Blob>` says what kind of bag it is: a bag of Blobs. `coins.add(...)` puts one more thing in the bag.

Step 1 - An empty bag

Build it. Open your Chapter 6 sketch. At the top, replace `Blob coin;` with:

```
ArrayList<Blob> coins;      // a list that holds Blobs
```

In `setup()`, replace the line that makes your coin with:

```
  coins = new ArrayList<Blob>();   // an empty list
```

> **Watch out.** Forget that line, and there's no bag. The sketch stops with `NullPointerException` the first time you try to put something in it.

Part Two - Two loops that both say for

Java uses the same word, `for`, for two loops that feel completely different. One **counts**, and you use it to make things. The other **visits**, and you use it to do something to everything already in the list. Keeping them straight is the real skill in this chapter.

Step 2 - Fill it: the counting loop

Build it. In `setup()`, right after the empty list, fill it:

```
  // Counting loop - runs 12 times to create 12 coins
  for (int i = 0; i < 12; i++) {
    coins.add(new Blob(
      this,
      random(width),          // random x position
      random(height),         // random y position
      random(8, 18),          // random radius
      random(-2, 2),          // random horizontal speed
      random(-2, 2),          // random vertical speed
      color(255, 220, 50)     // gold
    ));
  }
```

> **Aside: the counting loop**
>
> `for (int i = 0; i < 12; i++)` says *do this 12 times*. `int i = 0` starts a counter at zero. `i < 12` keeps going while the counter is under 12. `i++` adds one to the counter each time around. So the braces run with `i` at 0, 1, 2, and on up to 11: twelve times.

Run it.

> **Check.** Nothing new on the screen.

The bag has twelve coins in it now. Nothing draws them.

Step 3 - Draw them: the for-each loop

Build it. Where you used to move and draw your coin, **visit** every coin in the bag. Delete your old coin's collision `if` for now.

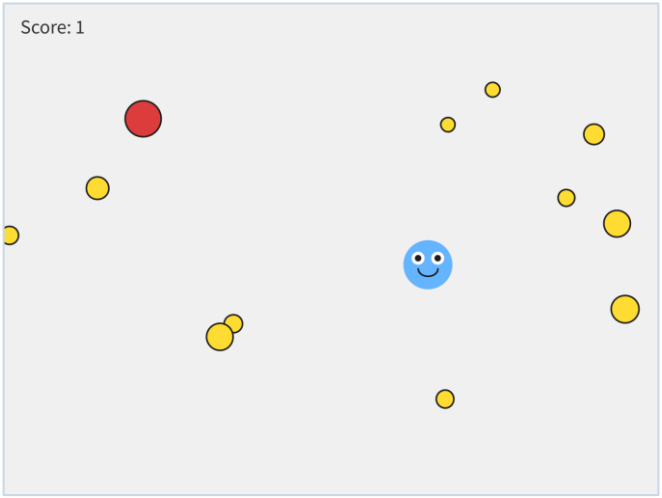
```
  for (Blob c : coins) {
    c.move();
    c.drawSprite();
  }
```

Aside: the for-each loop

`for (Blob c : coins)` reads *for each Blob `c` in `coins`*. It hands you the coins one at a time, calling each one `c` while it's your turn with it, and runs the braces once for every coin in the list.

Run it.

Check. The world is full of coins.



Twelve coins, from one loop.

Watch out. The bag is a bag. It can't draw itself. `coins.drawSprite();` stops the sketch with:

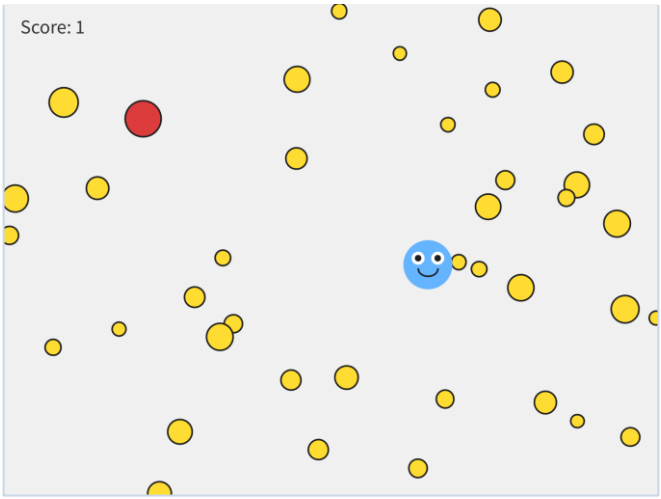
```
The function drawSprite() does not exist.
```

Reach into the bag and ask each coin, with the for-each loop.

Watch out. Mix the two loops up, and the messages get strange. `for (int i = 0; i < coins; i++)` gets `The operator < is undefined for the argument type(s) int, ArrayList<Blob>`, because you can't count up to a bag. `for (Blob c : 12)` gets `Can only iterate over an array or an instance of java.lang.Iterable`, because you can't visit a number. One counts. One visits.

Step 4 - How many?

Build it. Change the `12` in the counting loop to `40`, and run. Then try `3`. How many feels right?



Forty. Same five lines of code.

Watch out. Put the counting loop in `draw()` by mistake, and it adds twelve coins every frame. After one second there are about 700, and the sketch grinds to a halt. Making things goes in `setup()`.

Part Three - Taking things out

D.5 The problem with removing

With a list, collecting a coin means taking it **out** of the list. Removing something from a list *while you're walking through it* causes a real problem.

Picture five coins in a row. You're standing on coin 2, and you take it out. Every coin after it shifts back one place to fill the gap. Then the loop steps you forward to place 3, and the coin that *was* at 3 is at 2 now, behind you. You just skipped it.

It's not just a picture. Put three coins right on top of your player and walk the list forward, removing each one you touch: you collect two of them, and one gets skipped.

The fix is to walk **backwards**, from the end of the list toward the front. Taking a coin out then only shifts coins you've already passed, so every coin gets its turn. You don't have to work out why. Recognize it as *the safe way to remove things while looping*, and copy it when you need it.

Step 5 - Walk backwards

Build it. Replace your for-each loop with the backwards loop, with no collision check yet:

```
// Backwards loop - the safe way to remove things while looping
for (int i = coins.size() - 1; i >= 0; i--) {
  Blob c = coins.get(i);       // grab the coin at position i
  c.move();
  c.drawSprite();
}
```

Run it.

Check. Nothing looks different. The two loops do the same thing until you take something out.

Aside: `size()`, `get()`, and counting from 0

`coins.size()` is how many things are in the list right now. Every item has a number for its place, called its **index**, and a list starts counting at **0**. So twelve coins are numbered 0 to 11, and the last one is `coins.size() - 1`. `coins.get(i)` hands you the coin at place `i`. And `i--` counts **down**, one at a time.

Step 6 - Collect

Build it. Inside the backwards loop, after the coin is drawn, add the collision:

```
    if (player.collidesWith(c)) {
      coins.remove(i);           // take it out of the list
      score = score + 1;
    }
```

Run it, and collect a few.

Check. Touch a coin, it's gone, and the score goes up.

Watch out. Take a coin out inside a **for-each** loop, and the sketch stops with `ConcurrentModificationException`. That's a big word for *you changed the list while you were walking through it*. The backwards loop is the fix.

Watch out. Start the backwards loop at `coins.size()`, without the `- 1`, or count up with `i++` where `i--` belongs, and the sketch stops with:

```
IndexOutOfBoundsException: Index 12 out of bounds for length 12
```

Twelve coins are numbered 0 to 11, so there's no coin 12 to get.

Step 7 - A new wave

`coins.size()` shrinks every time you collect one. When it hits zero, the board is clear, and you get to decide what that means.

Build it. After the backwards loop, send a fresh wave:

```
  // An empty list means the board is clear:  send a new wave
  if (coins.size() == 0) {
    for (int i = 0; i < 12; i++) {
      coins.add(new Blob(this, random(width), random(height),
                         random(8, 18), random(-2, 2), random(-2, 2),
                         color(255, 220, 50)));
    }
  }
```

Clear the board, and watch it fill up again. Or set `playerWon = true;` in there, and the game ends. Your call.

Step 8 - A coin on a click

Build it. Add a coin wherever you click:

```
void mousePressed() {
  coins.add(new Blob(this, mouseX, mouseY,
                     random(8, 18), random(-2, 2), random(-2, 2),
                     color(255, 220, 50)));
}
```

Step 9 - A second list

Your turn. Make your hazards a list too: `ArrayList<Blob> hazards;`, filled with a counting loop, and visited with a **for-each** loop. You never take a hazard out, so the for-each loop is all you need.

D.7 Make it yours

Your turn. Change how many coins are in a wave, how big they are, and how fast.

Your turn. Optional: any of these.

- Make each wave bigger than the last, with a variable that counts waves, used in the counting loop.
- Add one more hazard to the hazards list every time a wave clears.
- Put a list into your Chapter 7 game. Convert your coins, and keep the hazards as they are. In `restartGame()`, `coins = new ArrayList<Blob>();` and then the counting loop give you a fresh, full board every restart: the same rebuild-from-scratch idea as the rest of `restartGame()`.
- Make a collected coin **split** into two smaller coins, added to the list.

Chapter summary

You built	It's an example of
`ArrayList<Blob> coins;`	a list: one variable for many things
`coins = new ArrayList<Blob>();`	an empty bag
`for (int i = 0; i < 12; i++) { coins.add(...); }`	the **counting** loop, to make things
`for (Blob c : coins) { ... }`	the **for-each** loop, to visit things
the backwards loop, with `coins.remove(i)`	the safe way to take things out while looping
`if (coins.size() == 0)`	an empty list, and a new wave

New syntax in this chapter

Syntax	Where you met it
`ArrayList<Blob>`, `new ArrayList<Blob>()`	D.1, Step 1
`add()`	D.1, Step 2
`for (int i = 0; i < 12; i++) { }`, `i++`	Step 2
`for (Blob c : coins) { }`	Step 3
`size()`, `get(i)`, `i--`	Step 5
`remove(i)`	Step 6

Vocabulary

`ArrayList` -- a list that holds many objects of the same kind, and grows as you add more. (D.1)

counting loop -- a loop that repeats a set number of times. Used to **make** things. (Step 2)

for-each loop -- a loop that visits every item already in a list, one at a time. Used to **visit** things. (Step 3)

index -- an item's place in a list. Lists start counting at 0. (Step 5)

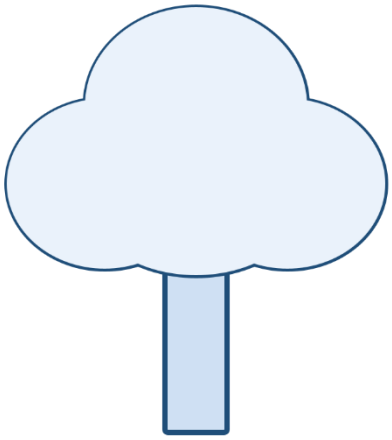
`size()` -- how many items are in the list right now. (Step 5)

Check your understanding

1. What's the difference between the counting loop and the for-each loop? When do you use each one?
2. Why doesn't `coins.drawSprite();` work?
3. Twelve coins. What's the index of the last one?
4. Why do we loop backwards when we take things out?
5. Your sketch stops with `ConcurrentModificationException`. What did you do?

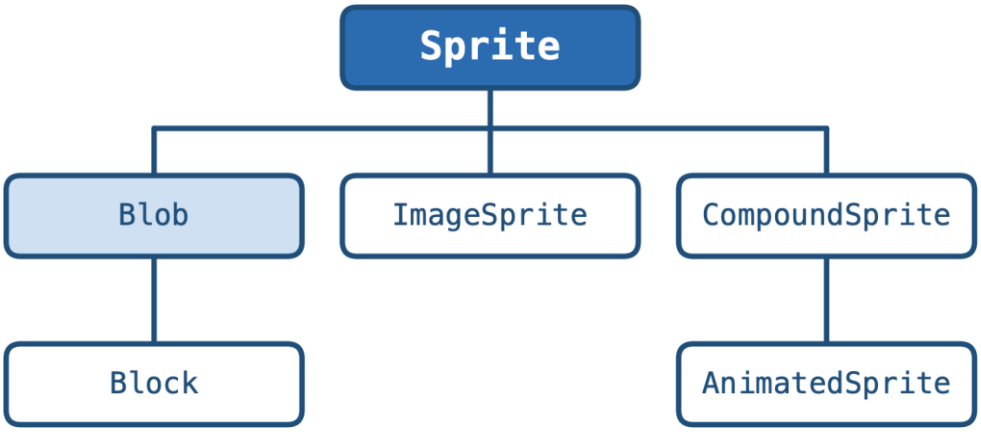
For Teachers

What's inside the toolbox.



Teacher Reference - The SpriteGame Library

What's inside the toolbox, for the person teaching from it.



This part of the book is for teachers. The chapters treat SpriteGame as a magic toolbox, on purpose: Guiding Principle 5 says students *"call methods the way they push buttons on a machine,"* and the button works. This reference opens the box. It documents every class and every public member, it shows how `Sprite` works on the inside, and it explains why the course builds every student class on `Blob`, along with what that choice costs.

Every behavior this reference describes was run against the library as it is in this repository, `library/SpriteGame.jar`, with Processing 4.5.2. The checks are in `lessons/chapters/planning/verify_api.sh`, and the messages and numbers below are what they printed. The generated Javadoc is in the repository too, under `reference/`.

It does it in five parts.

Part	What it covers
One	**The shape of the library.** The family tree, and where the students' classes hang on it.
Two	**`Sprite`, the base.** Its fields, its movement, and the vectors it hands out.
Three	**`Blob`, and why the course builds on it.** What the choice buys, and what it costs.
Four	**The rest of the family.** `Block`, `ImageSprite`, `CompoundSprite`, `AnimatedSprite`, `Vector2D`, `ScreenBehavior`, `Event`.
Five	**Known issues.** Every rough edge found, with a suggested fix.

Part One - The shape of the library

R.1 The family tree

```
Sprite  (abstract:  pos, vel, move(), followMouse(), follow(),
keyboardControl(), distanceTo())
 |
 +-- Blob  (radius, myColor, collidesWith(Blob), a circle for drawSprite())
 |    |
 |    +-- Block  (w, h, a rectangle for drawSprite())
 |    |
 |    +-- the students' classes:  MyCharacter, Cloud, MyImageCharacter,
MyCharacterFrame
 |
 +-- ImageSprite  (a picture, with a size and a rotation)
 |
 +-- CompoundSprite  (a list of child sprites, moved together)
      |
      +-- AnimatedSprite  (shows one child at a time, as a flipbook)

Helpers:  Vector2D (a pair of floats), ScreenBehavior (an enum), Event
(callbacks)
```

Everything a student puts on the screen, from Chapter 1's first circle to Chapter 7's game, is a `Blob` or a subclass of one. `ImageSprite` and `AnimatedSprite` arrive only in Bonus A and Bonus C. `CompoundSprite`, `Vector2D` by name, and `Event` appear in this reference alone.

R.2 How a student class fits

A class in a Processing tab is compiled as a nested class inside the sketch. That explains three things students do without knowing why.

- **Why they pass `this`.** Library classes live in their own package, so they reach the sketch only through a reference to it. Every constructor takes the `PApplet` and keeps it in `parent`. The handshake in Chapter 2, `super(p, x, y, 30, color(...))`, passes it up.
- **Why `MyCharacter` can call `fill()` directly, and the library calls `parent.fill()`.** A nested class can call the sketch's methods as its own. `MyImageCharacter` in Bonus A uses `parent.image(...)`, which works too. Both styles appear in the course, and both are correct.
- **Why `pos.x` works in a student's `drawSprite()`.** `pos` is `protected` in `Sprite`, so any subclass, including a nested class in a tab, can use it.

Part Two - `Sprite`, the base

R.3 Fields

Field	Type	Notes
`parent`	`PApplet` (`protected`)	the sketch. Used for drawing, `width`, `height`, `mouseX`, `mouseY`, `keyCode`.
`pos`	`Vector2D` (`protected`)	the sprite's position. For a `Blob` it's the center; for a `Block` it's the top-left corner.
`vel`	`Vector2D` (`protected`)	pixels per frame. Every constructor without a velocity starts it at `(0, 0)`.
`screenBehavior`	`ScreenBehavior` (`public static`)	what `move()` does at the window's edges. One setting, shared by every sprite. See R.12.

R.4 Constructors

Constructor	Notes
`Sprite(PApplet parent, float x, float y)`	velocity `(0, 0)`
`Sprite(PApplet parent, float x, float y, float dx, float dy)`	with a velocity
`Sprite(PApplet parent, Vector2D s)`	stores `s` itself as the position, so the caller's vector and the sprite's position are the same object. See R.7.

`Sprite` is `abstract`, so these are only ever called through `super(...)`.

R.5 Moving

Method	What it does
`move()`	`pos = pos.add(vel)`, which builds a new vector every frame, and then applies `screenBehavior` to the center.
`moveTo(float x, float y)`	sets `pos.x` and `pos.y` **in place**.
`moveTo(Vector2D p)`	stores `p` itself as the position. See R.7.
`setVelocity(float dx, float dy)`	sets `vel.x` and `vel.y` **in place**.
`setVelocity(Vector2D v)`	stores `v` itself as the velocity. See R.7.
`getPosition()`, `getVelocity()`	hand back the sprite's own vectors, not copies. See R.7.
`distanceTo(Sprite other)`	center to center: `pos` to `pos`. For a `Block`, that's its corner.
`drawSprite()`	`abstract`. Every subclass supplies one.

`move()` is the only method that looks at the window's edges. `followMouse()` and `moveTo()` skip them entirely, so a character steered by the mouse can be parked anywhere, edges included, whatever `screenBehavior` says.

Wrapping is by the **center**. When the center crosses an edge, the position jumps by the window's width or height in one frame. So half a character hangs off the edge before it wraps, and then the whole character pops across. Chapter 3 shows the stripe this makes.

R.6 The two input methods

`keyboardControl()` is a `switch` on `parent.keyCode`:

Key	Velocity
UP	`(0, -5)`
DOWN	`(0, 5)`
LEFT	`(-5, 0)`
RIGHT	`(5, 0)`
any other key	`(0, 0)`

Three consequences, all verified, and all taught in Chapter 4. The speed is always 5, whatever `setVelocity()` said. Movement is locked to one axis: UP sets `x` to zero. And every key that isn't an arrow is a brake, including Shift and the space bar. It reads `keyCode`, so it belongs in

`keyPressed()`. Called from `draw()`, it reapplies the last key's velocity every frame, and before any key has been pressed `keyCode` is 0, the `default` case, so the sprite sits still.

`followMouse()` is the other kind of control:

```
float speed = vel.mag();
Vector2D mLoc = new Vector2D(parent.mouseX, parent.mouseY);
Vector2D mDir = mLoc.subtract(pos);
pos = pos.add(mDir.unitVector().scale(speed));
```

It steps the **size** of the velocity straight toward the mouse, every frame, and leaves `vel` alone. So the velocity's direction is ignored: `(5, 0)`, `(0, 5)`, `(3, 4)` and `(-3, -4)` give byte-identical frames. A zero velocity is a zero step, which is why Chapter 4's Watch out says to set one. Within one step of the mouse it overshoots, and jitters back and forth by up to the speed.

It had one real bug, **fixed on 2026-10-30** with a guard clause: when the sprite is exactly on the mouse, `followMouse()` now returns without moving. Before the fix, `mDir` was `(0, 0)` there, `unitVector()` divided by zero, and the position became `(NaN, NaN)`. Every later step added to NaN and got NaN, so the sprite was gone for good:

```
frame 49:  pos = (596.0, 300.0)
frame 50:  pos = (600.0, 300.0)
frame 51:  pos = (NaN, NaN)
```

In a classroom it's exceedingly rare. The sprite has to land on the mouse exactly: level with it, a whole number of steps away, with the mouse held perfectly still the whole way. A hand on a real mouse doesn't do that. A test script can, and this book's screenshots hit it once that way, in Chapter 5, with a scripted mouse parked 200 pixels straight right of a player moving at 4. If a vanished sprite turns up now, that computer has an older copy of the library. The guard doesn't change the jitter within one step of the mouse. Part Five has a fuller fix that removes it too.

`follow(Sprite other)`, added on 2026-10-30, is `followMouse()` for a sprite target, with one difference: it only turns. It points the velocity straight at the other sprite, keeps the speed, and leaves the step to `move()`:

```
Vector2D toOther = other.pos.subtract(pos);
float distance = toOther.mag();
if (distance == 0) return;
setVelocity(toOther.scale(vel.mag() / distance));
```

So a chaser takes two lines a frame, `enemy.follow(player);` and then `enemy.move();`, and `move()` still handles the edges. It goes through `setVelocity()`, so a `CompoundSprite` carries its children along. A speed of 0 doesn't chase, and two sprites on exactly the same spot leave the velocity as it was. Verified with a sketch: a speed-3 chaser turned after a target moving down the screen, stayed at speed 3.00, and closed from 497 pixels to 163 in 150 frames. A compound chaser kept its child 10 pixels to the right the whole way. No chapter teaches it. It's in the Lesson 7 Sparky, for students who want a hazard that chases them.

Calling `followMouse()` and `move()` together doesn't jitter. `followMouse()` pulls the sprite 4 toward the mouse, and `move()` pushes it `vel`, which is still `(4, 0)`. The two cancel where *toward the mouse* points straight left, so the sprite parks on the mouse's row, just to its right,

and stays perfectly still. Chapter 4 shows it, and the Lesson 4 Mouse Group page has students try it on purpose.

R.7 The vectors it hands out

`Vector2D`'s own documentation says *"Every method returns a new Vector2D rather than changing the original."* That's true of `Vector2D`'s methods. It isn't true of the sprites that have them. `x` and `y` are public, `Sprite` changes them in place, and it hands its own vectors out. So two names can end up meaning one vector, and changing it through one name changes it through the other. Verified:

You write	What happens
`Vector2D p = a.getPosition(); p.x = 500;`	`a` moves to `x = 500`
`b.setVelocity(v); c.setVelocity(v);`, then `b.keyboardControl()` with UP	`c`'s velocity is `(0, -5)` too
`d.moveTo(spot); e.moveTo(spot);`, then `d.moveTo(50, 60)`	`e` moves to `(50, 60)` as well
the same, then `d.move()` and `d.moveTo(70, 80)`	`e` stays at `(50, 60)`: `move()` built `d` a new vector, and the sharing ended

The chapters use only the `float` versions of `moveTo()` and `setVelocity()`, and leave the getters alone, so the course itself steers clear of all of this. A curious student can find it, though, and it's the C# course's Chapter 1, aliasing, living in this library. The suggested fix is in Part Five.

Part Three - Blob, and why the course builds on it

R.8 Blob

Member	Notes
`radius`	`float` (`protected`). The collision circle. It doesn't change the drawing of a student's class.
`myColor`	`int` (`protected`). Used only by `Blob`'s own `drawSprite()`.
`Blob(PApplet parent, float x, float y, float r, int c)`	Chapter 1's five things
`Blob(PApplet parent, float x, float y, float r, float dx, float dy, int c)`	with a velocity: Chapter 5's coin
`collidesWith(Blob other)`	`distanceTo(other) <= radius + other.radius`. Circles only, and touching counts: centers exactly 20 apart, with radii 10 and 10, is `true`, and 21 apart is `false`.
`drawSprite()`	`fill(myColor)`, then a circle of `2 * radius`. It sets no stroke, so it uses whatever stroke is current. See R.10.

R.9 Why Blob

The course's own guiding principles say it: *"`collidesWith()` is on `Blob`, not on the base `Sprite`. Students' custom characters should use `class MyCharacter extends Blob` (not `extends Sprite`) so that collision detection works automatically against other Blobs in the world."*

So every class a student writes extends `Blob`: the character, the cloud, a picture character, an animation frame. One rule, the same words every time. The rule has costs, and each one buys something. Each is a trade-off, and all six are worth knowing when you're the one in the room.

1. A misspelling fails silently. `Sprite.drawSprite()` is `abstract`, and `Blob.drawSprite()` isn't. So a student who writes `void drawsprite()` in a class that extends `Blob` gets no error. The sketch runs and draws Blob's own circle, in the handshake's backup color. The same misspelling in a class that extends `Sprite` stops the sketch:

```
The type SpriteMisspelled.Thing must implement the inherited abstract method
Sprite.drawSprite()
```

What it buys: a sketch that always runs, and a picture of what's underneath. Chapter 2 turns the silent fallback into its first discovery. If you'd rather have the error, `@Override` on the line above `drawSprite()` brings it back, and keeps `Blob`:

```
The method drawsprite() of type OverrideCatch.Thing must override or implement a
supertype method
```

No chapter uses `@Override`. Adding it would put one more copied line in every class a student writes.

2. Only Blobs can collide. `collidesWith()` lives on `Blob`, and it takes a `Blob`. So only Blobs can use it, from either side. A plain `Sprite` subclass has no `collidesWith()`, and a Blob can't collide with one either:

```
The function collidesWith(Blob) does not exist.
The method collidesWith(Blob) in the type Blob is not applicable for the
arguments (BlobVsSprite.Thing)
```

`ImageSprite` and `AnimatedSprite` hit the same wall, in Bonus A and Bonus C. The way around it is `distanceTo()`, which lives on `Sprite` and works between any two sprites. What it buys: one type for the whole world. The character, the clouds, the coins, and the hazards are all Blobs, so one kind of variable fits any of them, and Bonus D's `ArrayList<Blob>` takes every one.

3. Collisions are circles. `collidesWith()` compares two radii, so every hitbox is a circle, and a tall character or a long platform can only be approximated. The radius is also separate from the drawing, so a student's art can drift far from its hitbox, which is a pitfall the Lesson 6 Canvas page names. What it buys: one number as the design lever. Chapter 6 asks *"generous or precise?"*, and the answer is the radius. The rule fits in one sentence a sixth grader can check.

4. The handshake carries two things the student doesn't use. `super(p, x, y, 30, color(...))` passes the sketch, which students only ever meet as `this`, and a color, which is dead the moment `drawSprite()` is replaced. What it buys: they're the same five things Chapter 1's Blob took, so Chapter 2 can say *"you've seen these five before."* The color is also what shows up when a misspelling falls back to Blob's circle, which makes trade-off 1 easier to explain.

5. `Block` is a `Blob`. A rectangle isn't a circle, and `Block` inherits anyway, to reuse `myColor` and to fit wherever a Blob fits. So a `Block` inherits a collision circle of radius 0 at its top-left corner. Verified: a Blob sitting in the middle of a 200-by-100 `Block` is `false`, and a Blob near its corner is `true`. `distanceTo()` measures to the corner too. That's why Chapter 5 calls a `Block` scenery. What it buys: one more thing that goes anywhere a `Blob` goes, at no cost in new code.

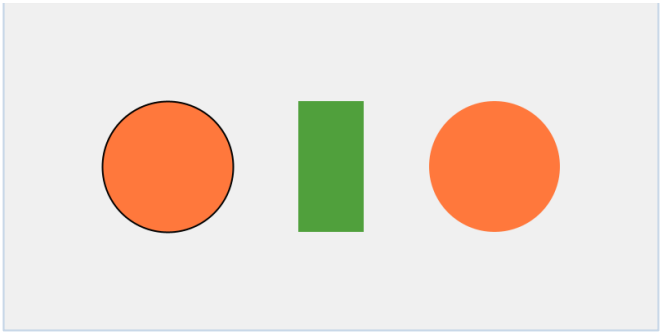
6. `pos` and `vel` are open to students. Because they're `protected`, a student's class can change `pos` directly, skipping `moveTo()`, and the getters hand out the live objects (R.7). What it buys: the belly button. A sixth grader writes `ellipse(pos.x - 12, pos.y - 8, 16, 16)`, with no getter and no copy, and Chapter 2's whole Part Three rests on it.

R.10 Style leaks from one sprite to the next

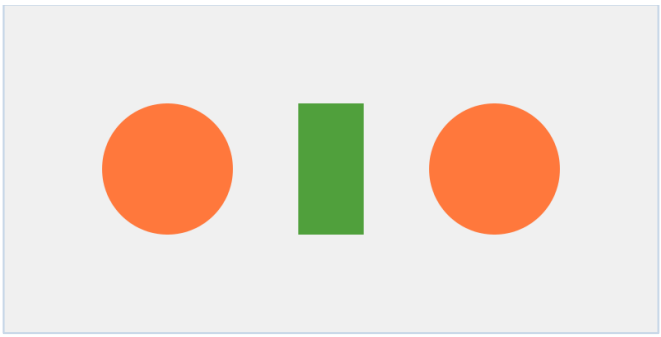
Processing keeps one set of drawing settings, `fill`, `stroke`, `strokeWeight`, and the rest, and each one carries on until something changes it: from shape to shape, sprite to sprite, and frame

to frame. The library's `drawSprite()` methods mostly don't save and restore them. `ImageSprite` is the exception: it wraps its drawing in `pushStyle()` and `popStyle()`.

So settings leak. `Block.drawSprite()` calls `noStroke()`, and every shape drawn after it loses its outline, including, from the second frame on, the shapes drawn before it, because the setting carries into the next frame.



Frame 1: a Blob, a Block, and a Blob. The first Blob has its outline, and the one drawn after the Block doesn't.



Frame 3: neither Blob has an outline. The `noStroke()` from frame 1 is still on.

In the course this shows up quietly. Chapter 1's blob has an outline, and Chapter 5's coin doesn't, because the ground is drawn first. A student's character sets its own fill and stroke inside `drawSprite()`, so it looks the same wherever it's drawn, and that's the habit to teach: a `drawSprite()` sets everything it needs.

Part Four - The rest of the family

R.11 Block

Member	Notes
`w`, `h`	`float` (`protected`). Width and height, stored as absolute values.
`Block(PApplet parent, float x, float y, float w, float h, int c)`	`x, y` is the **top-left corner**. The radius is 0.
`drawSprite()`	`fill(myColor); noStroke(); rect(pos.x, pos.y, w, h)`. The `noStroke()` leaks (R.10).

A `Block` has `move()` and `setVelocity()`, and starts still. Its collision is R.9's trade-off 5: a point at its corner.

R.12 ScreenBehavior

An enum with four values, set through the static `Sprite.screenBehavior`, so one line changes every sprite in the sketch, immediately, mid-game included. It applies only inside `move()`, and only to each sprite's center.

Value	At an edge	Verified
`Wrapping`	jumps to the opposite edge. The default.	Chapter 3
`HardWalls`	stops at the edge, and zeroes the velocity on that axis	a Blob moving right at 5 from `x = 780`: at frame 10, `x = 800`, `vx = 0`
`BouncyWalls`	reverses the velocity on that axis	then `vx = -5` two frames later
`NoWalls`	nothing: the sprite leaves the window	`x = 830` after ten frames from 780

Chapter 7 offers `BouncyWalls` as a one-line extra. Because it's static, the clouds bounce too, and a student can't make the hazards bounce and the clouds wrap at the same time.

R.13 ImageSprite

`ImageSprite` extends `Sprite` directly. It isn't a `Blob`, so it has no radius and no `collidesWith()`.

Member	Notes
`ImageSprite(PApplet parent, float x, float y, String fileName)`	100 by 100
`ImageSprite(PApplet parent, float x, float y, String fileName, float size)`	square
`ImageSprite(PApplet parent, float x, float y, String fileName, float w, float h)`	`x, y` is the **center**
`setSize(float size)`, `setSize(float w, float h)`	absolute values
`setRotation(float rotation)`, `rotate(float amount)`, `getRotation()`	radians, around the center
`drawSprite()`	`pushMatrix`, `pushStyle`, `translate`, `rotate`, `imageMode(CENTER)`, `image`, and both pops. The one `drawSprite()` in the library that puts every setting back.

The image loads once, in the constructor, from the sketch's `data` folder. A missing file prints Processing's *"is missing or inaccessible"* message and leaves the image `null`. The constructor carries on, and the sketch stops with `NullPointerException` the first time `drawSprite()` passes `null` to `image()`. A name with the wrong capitals gets Processing's own clear message instead: *"This file is named tree.png not Tree.PNG. Rename the file or change your code."*

R.14 CompoundSprite

A sprite made of other sprites, its **children**. The compound's own `pos` is a reference point. It isn't drawn, and it needn't be anywhere near its children. Children keep their own absolute positions and velocities.

Member	Notes
`CompoundSprite(PApplet parent, float x, float y)`, and with `dx, dy`	
`addSprite(Sprite s)`	adds the compound's **current** velocity to the child's, then adds it. Returns the compound, so calls chain.
`removeSprite(Sprite s)`	takes the compound's velocity back off the child
`getSprites()`	the live list of children
`setVelocity(...)`	changes the compound's velocity, and adds the same **change** to every child
`move()`	moves the compound, then calls `move()` on each child
`moveTo(...)`, `followMouse()`	move the compound, then shift every child by the same amount
`keyboardControl()`	sets the compound's velocity, and passes the change to every child
`drawSprite()`	draws every child, in the order they were added

Two consequences. **Each child wraps by itself**, because `move()` reaches every child separately. Verified with two Blobs 50 pixels apart moving right: the right one wraps to `x = 2` while the left is at `752`, and the compound's own point wraps later still. A compound that crosses an edge splits for a few frames, and comes back together on the other side. And **the compound's `distanceTo()` measures from its own point**, which is wherever it was made, so for collisions measure from a child.

No chapter uses `CompoundSprite` directly. The library's `FractalExample` builds its branches from it, and `AnimatedSceneExample` builds its clouds and sun from its subclass, `AnimatedSprite`.

R.15 AnimatedSprite

A `CompoundSprite` that draws one child at a time: a flipbook. Bonus C.

Member	Notes
`AnimatedSprite(PApplet parent, float x, float y)`, and with `dx, dy`	starts on frame 0, at a rate of 8
`FromSprite(Sprite s)`, `fromSprite(Sprite s)`	`static`. A new `AnimatedSprite` at `s`'s position, with `s` as its first frame. `null` throws `IllegalArgumentException`.
`AddFrameSprite(Sprite s)`, `addFrameSprite(Sprite s)`	adds a frame, and returns the sprite, so calls chain
`setFrameRate(int n)`, `getFrameRate()`	game frames per animation frame, at least 1. Smaller is faster.
`setFrameNumber(int n)`, `getFrameNumber()`	jump to a frame. Wraps around, negatives included.
`nextFrame()`, `previousFrame()`	turn one page, by hand
`getFrameSprites()`, `getFrameCount()`	the frames, and how many
`drawSprite()`	draws the current frame, **then** advances the clock

Both spellings of the builder methods exist, `FromSprite` and `fromSprite`, so either capital letter works. The clock advances inside `drawSprite()`, so an `AnimatedSprite` that isn't drawn doesn't animate, and one drawn twice a frame animates twice as fast. Each frame is a full sprite with its own position, so frames made at different spots jump (Bonus C shows it), and every frame under the current one still carries its own unused radius and color.

R.16 Vector2D

A pair of floats, `x` and `y`, both public. Every method returns a new vector.

Member	Notes
`Vector2D(float x, float y)`, `FromXY(float x, float y)`	the same thing, two ways
`FromPolar(float r, float theta)`	`(r cos theta, r sin theta)`. With y pointing down, positive angles turn **clockwise on the screen**.
`add`, `subtract`, `scale`, `inverse`	the usual
`mag()`	length. For a velocity, the speed.
`dotProduct(Vector2D other)`	the usual
`unitVector()`	length 1. `(0, 0)` gives `(NaN, NaN)`, which the documentation warns about, and which is R.6's bug.
`perpendicular()`	`(y, -x)`. `(1, 0)`, right, becomes `(0, -1)`, which is **up** on the screen: a quarter turn counter-clockwise as you see it. The documentation says *clockwise*, which is true in math's y-up coordinates.
`rotate(float rads)`	the standard formula. `(1, 0)` rotated by `HALF_PI` is `(0, 1)`, **down** on the screen: positive angles turn clockwise as you see it. The documentation says *counter-clockwise*, true in y-up coordinates.
`angleTo(Vector2D other)`	wrong for any vector pointing left. See Part Five.
`theta()`	`protected`. `atan(y / x)`.

`angleTo()` is `other.theta() - this.theta()`, and `theta()` is `atan(y / x)`, which can't tell left from right. Verified:

```
right.angleTo(left) = -0.0   (PI is 3.1415927)
right.angleTo(upLeft) = 0.7853982   (the true angle is -2.3561945)
```

No chapter uses it.

R.17 Event

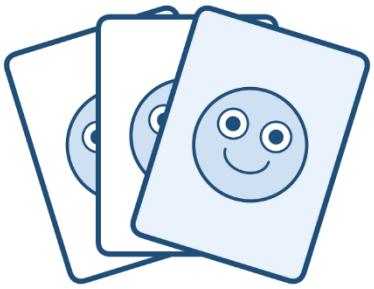
An observer: bind methods to an event by name, and `trigger()` calls every bound method. The original author is credited in the source as Stephcraft.

Member	Notes
`Event(Class<?>... args)`	the argument types, as classes: `new Event()`, or `new Event(Integer.class)`
`bind(Object listener, String name)`	finds the public method by name, with reflection, and adds it. A wrong name or wrong types prints `[Event] could not bind ...` and carries on.
`unbind(Object listener, String name)`, `bound(Object listener, String name)`	the reverse, and a check
`trigger(Object... arguments)`	calls every bound method. An exception inside one is printed, and the rest still run.

Two problems, both verified, mean `Event` is hard to use from a sketch.

- **The name collides.** Processing imports `processing.event.Event` into every sketch, so a bare `Event` stops the sketch with `The type Event is ambiguous`. It has to be written out: `coxprogramming.processing.sprites.Event`.
- **It can't call into the sketch's own classes.** Bound to a function on the sketch itself, `bind(this, "addPoint")`, it works. Bound to a method of a class in a tab, even a `public` one, every `trigger()` prints `IllegalAccessException: class coxprogramming.processing.sprites.Event$EventListener cannot access a member of class Events2$Counter with modifiers "public"`, and the method never runs. Classes in tabs are compiled as non-public nested classes, and reflection from another package can't call into them.

No chapter uses `Event`.



Part Five - Known issues

Everything this run found, with a suggested fix for each. The fixes are suggestions. Only the first was built and run, as an override in a sketch. The library itself took a smaller fix for the first, on 2026-10-30, and is otherwise unchanged.

Where	The issue	Suggested fix
`Sprite.followMouse()`	Landing exactly on the mouse divided by zero: the position became NaN, and the sprite vanished for good. **Fixed 2026-10-30** with a guard clause (R.6). Within one step it still jitters.	Land on the mouse when it's within one step (below). This would also end the jitter.
`Sprite.getPosition()`, `getVelocity()`, `moveTo(Vector2D)`, `setVelocity(Vector2D)`, `Sprite(PApplet, Vector2D)`	Share vectors with the caller (R.7).	Hand out and store copies: `return new Vector2D(pos.x, pos.y);`, `pos = new Vector2D(p.x, p.y);`.
`ImageSprite`	A missing file stops the sketch later, at `drawSprite()`, with `NullPointerException`.	In `drawSprite()`, `if (img == null) return;`, or draw a placeholder rectangle.
`Block`	Inherits a collision circle of radius 0 at its corner.	Give `Block` a rectangle check of its own, or say plainly that it's scenery (the chapters do).
`Blob.drawSprite()`, `Block.drawSprite()`	Leave fill and stroke changed for everything after them (R.10).	Wrap each in `parent.pushStyle()` and `parent.popStyle()`, the way `ImageSprite` does. This would also give Chapter 1's blob and Chapter 5's coin the same outline.
`Event`	Ambiguous by name in a sketch, and can't call methods in a sketch's classes.	Rename it, `GameEvent`, and call `method.setAccessible(true)` in `bind()`.
`Vector2D.theta()`	`atan(y / x)` can't tell left from right, so `angleTo()` is wrong for left-pointing vectors.	`(float) Math.atan2(y, x)`.

Where	The issue	Suggested fix
`Vector2D` documentation	*Clockwise* and *counter-clockwise* are y-up words, and the screen is y-down.	Say which way it turns on the screen.

The `followMouse()` fix, as it was run: overridden in a `Blob` subclass, with the mouse at the spot that gave the library's version a NaN at frame 51. It landed on `(600.0, 300.0)` at frame 51 and stayed there through frame 60.

```
public void followMouse() {
  float speed = vel.mag();
  Vector2D toMouse = new Vector2D(parent.mouseX, parent.mouseY).subtract(pos);
  float d = toMouse.mag();
  if (d <= speed) {
    pos = new Vector2D(parent.mouseX, parent.mouseY);
  } else {
    pos = pos.add(toMouse.scale(speed / d));
  }
}
```

In `CompoundSprite`, the same fix belongs in its override, which shifts the children by however far the compound moved.